

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева (Самарский университет)»

Институт _____ информатики и кибернетики _____

Кафедра _____ программных систем _____

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

_____ к курсовой работе по дисциплине «Программная инженерия»
_____ по теме «Приложение «Игра «Менеджмент» _____

Обучающийся _____ Д.О. Голобурдина

Обучающийся _____ Н.С. Некрасов

Обучающийся _____ Е.И. Скрипкин

Руководитель _____ Л.С. Зеленко

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева (Самарский университет)»

Институт _____ информатики и кибернетики _____

Кафедра _____ программных систем _____

ЗАДАНИЕ

на курсовую работу по дисциплине
«Программная инженерия»
обучающимся в группе № 6401-020302D
Д.О. Голобурдиной
Н.С. Некрасову
Е.И. Скрипкину

- 1 Тема проекта: «Приложение «Игра «Менеджмент»»
- 2 Исходные данные к проекту: см. приложение к заданию
- 3 Перечень вопросов, подлежащих разработке:
 - 3.1 Произвести анализ предметной области: изучить правила игры «Менеджмент»
 - 3.2 Выполнить обзор существующих систем-аналогов
 - 3.3 Разработать информационно-логический проект системы по методологии UML
 - 3.4 Разработать и реализовать программное и информационное обеспечение, провести его тестирование и отладку
 - 3.5 Оформить документацию курсовой работы
 - 3.6 Подготовить презентацию по разработанной системе
- 4 Перечень графических разработок:
 - 4.1 Структурная схема системы
 - 4.2 Канонические диаграммы UML
 - 4.3 Схемы основных алгоритмов

5 Календарный план выполнения работ

№ п/ п	Содержание работы по этапам	Объем этапа в % к общему объему проекта	Срок окончания	Фактическое выполнение
1	Оформление технического задания и его утверждение	5	17.09.2024	
2	Описание и анализ предметной области	10	24.09.2024	
3	Проектирование системы	40	10.12.2024	
3.1	Разработка структурной схемы системы	5	8.10.2024	
3.2	Разработка функциональной спецификации системы и прототипа интерфейса пользователя	10	22.10.2024	
3.3	Разработка информационно-логического проекта системы и его предъявление руководителю	25	10.12.2024	
4	Реализация проекта, разработка контрольных примеров. Предъявление реализации руководителю	40	10.12.2024	
5	Корректировка проекта и оформление документации проекта. Защита проекта с представлением презентации.	5	24.12.2024	

Задание принял

к исполнению _____ 10.09.2024 Д.О. Голобурдина
 _____ Н.С. Некрасов
 _____ Е.И. Скрипкин

ПРИЛОЖЕНИЕ

к заданию на курсовую работу
обучающимся в группе № 6401-020302D

Д.О. Голобурдиной

Н.С. Некрасов

Е.И. Скрипкин

Тема проекта: Приложение «Игра «Менеджмент»

Исходные данные к проекту:

1 Характеристика объекта автоматизации:

- 1) объект автоматизации: игра «Менеджмент»;
- 2) виды автоматизируемой деятельности:
 - процесс регистрации пользователей;
 - процесс моделирования хода игры;
 - процесс взаимодействия с игроками;
 - процесс расчёта и отображения финансовых показателей;
 - процесс визуализации действий игроков;
- 3) количество аватаров – 10;
- 4) минимальное количество игроков – 2;
- 5) максимальное количество игроков – 4;
- 6) количество старших игроков – 1;
- 7) минимальная длина логина – 4 символа;
- 8) максимальная длина логина – 8 символов;
- 9) максимальное количество кругов – 13;
- 10) начальное количество обычных фабрик – 2;
- 11) начальное количество сырья и материалов – 4;
- 12) начальное количество готовой продукции – 2;
- 13) стартовая сумма – 10000 долларов;
- 14) ежемесячные издержки за единицу ЕСМ – 300 долларов;
- 15) ежемесячные издержки за единицу ЕГП – 500 долларов;
- 16) издержки за владение обычной фабрики – 1000 долларов в месяц;

- 17) издержки за владение автоматизированной фабрики – 1500 долларов в месяц;
 - 18) минимальная цена ЕСМ – 300 долларов;
 - 19) максимальная цена ЕГП – 6500 долларов;
 - 20) максимальное общее количество автоматизированных и обычных фабрик – 6;
 - 21) ежемесячная переработка обычной фабрикой – 1 ЕСМ при затратах в 2000 долларов;
 - 22) ежемесячная переработка автоматизированной фабрикой – 2 ЕСМ при затратах в 3000 долларов;
 - 23) оплата от невыплаченных ссуд – 1%;
 - 24) ссуда под обычную фабрику – 5000 долларов;
 - 25) срок постройки обычной фабрики – 5 месяцев;
 - 26) ссуда под автоматизированную фабрику – 10000 долларов;
 - 27) срок постройки автоматизированной фабрики – 7 месяцев;
 - 28) максимальная общая ссуда – половина гарантированного капитала;
 - 29) срок погашения ссуды – 12 месяцев;
 - 30) реконструкция обычной фабрики в автоматизированную – 7000 долларов;
 - 31) срок реконструкции – 9 месяцев;
 - 32) максимальное время на подключение к игровой сессии – 60 с;
 - 33) максимальное время на совершение хода – 60 с.
- 2 Требования к информационному обеспечению:
- 1) информационное обеспечение разрабатывается на основе следующего источника:
 - финансовые воротилы, или Управление предприятиями и машинное моделирование [Электронный ресурс]. URL: <https://fb2.top/etyudy-dlya-programmistov-636815/read/part-8> (дата обращения: 17.09.2024);
 - 2) в системе должны быть реализованы правила игры «Менеджмент».

3 Требования к техническому обеспечению:

3.1 Требования к техническому обеспечению серверной части:

- 1) тип ЭВМ – IBM PC совместимый;
- 2) объем ОЗУ – не менее 2 Гб;
- 3) объем свободного пространства на внешнем диске – не менее 50 Гб;
- 4) наличие подключения к сети Интернет;
- 5) манипулятор – мышь;
- 6) технические характеристики определяются в процессе выполнения проекта.

3.2 Требования к техническому обеспечению клиентской части:

- 1) тип ЭВМ – IBM PC совместимый;
- 1) монитор с разрешающей способностью не ниже 800 x 600;
- 2) манипулятор – мышь;
- 3) технические характеристики определяются в процессе выполнения проекта.

4 Требования к программному обеспечению:

4.1 Требования к программному обеспечению серверной части:

- 1) тип операционной системы – Windows 7 и выше.

4.2 Требования к программному обеспечению клиентской части:

- 1) тип операционной системы – Windows 7 и выше;
- 2) браузер – Google Chrome 86.0.4240.183 (64-битный) и выше, Firefox 83.0 (64-битный) и выше.

4.3 Требования к программному обеспечению рабочего места разработчика:

- 1) тип операционной системы – Windows 7 и выше;
- 2) язык программирования – Java Script, HTML;
- 3) среда программирования – Visual Studio Code 2019;
- 4) фреймворки определяются в процессе разработки;
- 5) среда проектирования – StarUML 5.2.0.

5 Общие требования к проектируемой системе:

5.1 Функции, реализуемые системой:

1) функции системы:

- контроль подключения игроков к сессии;
- контроль времени выполнения хода;
- контроль подачи заявок;
- реализация игровых правил и логики игрового процесса;
- обработка действий игроков и изменение состояния игры;
- предоставление интерфейса для выбора действий игроками;
- отображение актуальной игровой ситуации для каждого игрока;
- ведение учета активов и капитала каждого игрока;
- фиксация всех действий игроков в течение игры;
- оплата постоянных издержек;
- оплата ссудного процента;
- погашение ссуд;
- возможность просмотра открытой информации о других участниках – капитал, наличные, взятые ссуды, всё, что касается готовой продукции, имеющихся и строящихся фабрик;
- выдача справочной информации;

2) функции банка:

- формирование цен на ЕГП и ЕСМ;
- рассмотрение заявок игроков на продажу ЕГП и покупку ЕГП;
- рассмотрение заявок игроков на продажу ЕСМ и покупку ЕСМ;

3) функции игрока:

- регистрация пользователя в системе (ввод никнейма и выбор аватара);
- подключение к игровой сессии;
- создание собственной сессии;
- выбор доступных игровых действий, в зависимости от игровой

ситуации:

- 1) подача заявки на получение сырья и материалов (необходимое количество ЕСМ и предлагаемую цену);
 - 2) выбор количества для переработки ЕСМ в ЕГП;
 - 3) подача заявки на продажу продукции ЕГП и предлагаемую цену;
 - 4) выбор ссуды;
 - 5) подача заявки на строительство;
- просмотр лога игры после очередного хода;
 - пропуск хода;
 - признание себя банкротом;
 - просмотр справочной информации.

5.2 Технические требования к системе:

- 1) режим работы – диалоговый;
- 2) система должна удовлетворять санитарным правилам и нормам СанПин 2.2.2./2.4.2198-07;
- 3) условия работы средств вычислительной техники (содержание вредных веществ, пыли и подвижность воздуха) должны соответствовать ГОСТ 12.1.005, 12.01.007;
- 4) температура окружающего воздуха – 15-35°C;
- 5) влажность воздуха – 45-75%.

Руководитель

проекта _____ Л.С. Зеленко

Задание принял

к исполнению _____ Е.И. Скрипкин

_____ Н.С. Некрасов

_____ Д.О. Голобурдина

РЕФЕРАТ

Пояснительная записка: 114 с, 60 рисунков, 6 таблиц, 17 источников, 2 приложения.

Графическая часть: 13 слайдов презентации PowerPoint.

МЕНЕДЖМЕНТ, ВЕБ-ПРИЛОЖЕНИЕ, АВТОМАТИЗАЦИЯ БИЗНЕС-ПРОЦЕССОВ, УПРАВЛЕНИЕ ФАБРИКАМИ, СЫРЬЕ, ПРОДУКЦИЯ, ПОШАГОВАЯ СТРАТЕГИЯ, КЛИЕНТ-СЕРВЕРНАЯ АРХИТЕКТУРА.

Объектом автоматизации является игровая система "Менеджмент".

Во время курсового проектирования разработано веб-приложение, моделирующее управление виртуальной производственной компанией. Игроки управляют предприятиями, принимая решения о покупке сырья, производстве продукции, продаже готовых товаров, а также ведении финансов. Система реализует игровые правила, включает аналитику финансовых результатов и отображение игровой информации. Разработана возможность подключения от двух до четырех участников, реализован удобный пользовательский интерфейс с регистрацией и визуализацией.

Архитектура программы основана на двухуровневой клиент-серверной модели с использованием протоколов HTTP и WebSocket. Серверная часть разрабатывается на ASP NET в Visual Studio 2022. Клиентская часть разрабатывается на JavaScript, HTML и CSS в Visual Studio Code 2019. Данные хранятся в формате JSON.

СОДЕРЖАНИЕ

1	Описание и анализ предметной области	14
1.1	Описание предметной области	14
1.1.1	Основные понятия и определения	14
1.1.2	Классификация экономических игр.....	15
1.1.3	Требования к экономической игре «Менеджмент»	17
1.2	Описание систем-аналогов	17
1.2.4	Компьютерная деловая игра «Бизнес-курс».....	18
1.2.5	«Монополия»	20
1.2.6	«Кондитерская фабрика».....	22
1.3	Диаграмма объектов предметной области	23
1.4	Постановка задачи.....	26
2	Проектирование системы.....	32
2.1	Выбор и обоснование архитектуры системы.....	32
2.2	Структурная схема системы	37
2.3	Разработка спецификации требований.....	41
2.3.1	Функциональная спецификация	42
2.3.2	Перечень исключительных ситуаций	47
2.4	Разработка прототипа интерфейса пользователя системы	47
2.5	Разработка информационно-логического проекта системы	52
2.5.1	Язык UML	53
2.5.2	Диаграмма вариантов использования.....	54
2.5.3	Сценарии	57
4	Игрок щелкает на кнопку с одной ЕСМ.....	58
2.5.4	Диаграмма классов	62
2.5.5	Диаграмма состояний.....	63
2.5.6	Диаграмма деятельности	69
2.6	Выбор и обоснование алгоритмов обработки данных	73
2.7	Выбор и обоснование комплекса программных средств	74
2.7.1	Выбор языка программирования	74

2.7.2	Выбор среды программирования	76
2.7.3	Выбор операционной системы	77
3	Реализация системы	78
3.1	Разработка и описание интерфейса пользователя	78
3.2	Диаграммы реализации	84
3.2.1	Диаграмма компонентов	84
3.2.2	Диаграмма развертывания	87
3.2.3	Диаграмма классов	88
3.3	Выбор и обоснование комплекса технических средств	89
3.3.1	Расчет объема занимаемой памяти	89
3.3.2	Минимальные требования, предъявляемые к системе	91
ЗАКЛЮЧЕНИЕ		93
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ		94
ПРИЛОЖЕНИЕ А Руководство пользователя		96
A.1	Назначение системы	96
A.4	Работа с системой	98
A.4.1	Работа с системой в режиме пользователя	98

ВВЕДЕНИЕ

Современный рынок труда предъявляет все более высокие требования к подготовке высококвалифицированных управленческих кадров, способных эффективно решать сложные задачи в условиях динамичной, высококонкурентной среды. Успех любой организации напрямую зависит от качества и своевременности принимаемых руководством управленческих решений, умения мотивировать персонал и выстраивать эффективные бизнес-процессы. Именно поэтому обучение современным управленческим практикам является одним из ключевых направлений подготовки будущих менеджеров [1].

Первые симуляционные игры, моделирующие бизнес-процессы, появились еще в 1950-х годах. Одной из наиболее известных и широко используемых в учебных целях является игра «Бизнес-симулятор», разработанная в 1956 году американскими учеными Алленом Нилсоном и Альбертом Шуллером. В дальнейшем концепция симуляционных бизнес-игр активно развивалась и совершенствовалась, появлялись все более реалистичные и технологичные модели, отражающие современные тренды в управлении организациями [2].

Переход от настольных игр к цифровым начался в середине 20 века с развитием компьютерных технологий. Первые попытки создать цифровые версии настольных игр появились в 1970-х годах, когда компьютеры стали более доступными. В 1980-х и 1990-х годах с ростом популярности домашних ПК и игровых консолей начали выходить классические настольные игры в цифровом формате.

С развитием интернета в 2000-х годах появились онлайн-платформы, позволяющие игрокам взаимодействовать друг с другом в реальном времени. Это привело к созданию многопользовательских игр и виртуальных настольных игр. Современные технологии, такие как мобильные приложения и виртуальная реальность, продолжают расширять границы цифровых версий настольных игр, делая их более доступными и интерактивными.

Во время курсового проектирования необходимо разработать игру «Менеджмент» – симуляционную бизнес-модель, позволяющую участникам в режиме реального времени применять на практике базовые управленческие концепции и оценивать последствия принимаемых ими решений. Данный инструмент активного обучения дает возможность сформировать у студентов и начинающих управленцев целостное понимание процессов, происходящих в организации, и выработать эффективные стратегии управления.

Разработка системы будет производиться по технологии быстрой разработки приложений RAD (Rapid Application Development), которая поддерживается методологией структурного проектирования и включает элементы объектно-ориентированного проектирования и анализа предметной области [3]. Данный подход позволяет в сжатые сроки создавать прототипы системы и получать обратную связь от пользователей, что способствует повышению качества и релевантности конечного продукта.

При проектировании системы будут использоваться методология ООАП (Object-Oriented Analysis/Design), в основу которой положена объектно-ориентированная методология представления предметной области в виде объектов, являющихся экземплярами соответствующих классов, и язык моделирования UML (Unified Modeling Language), который является стандартным инструментом для разработки «чертежей» программного обеспечения [4].

Таким образом, разработка игры «Менеджмент» с применением современных методологий проектирования и технологий разработки ПО позволит создать эффективный инструмент для обучения будущих менеджеров, способствующий формированию у них комплекса практических навыков, востребованных на рынке труда.

1 Описание и анализ предметной области

Деловая игра "Менеджмент" позволяет участникам почувствовать себя в роли руководителей компаний, управляющих производством и финансами. Игроки самостоятельно принимают решения, которые определяют успешность их бизнеса. Они управляют компаниями, покупая сырье, производя и продавая продукцию. Игроки решают, сколько и каких товаров выпускать, как финансировать расширение и как отчитываться. Игра ведется с шагом по времени в один месяц.

В начале игроки получают фабрики, сырье и определенную сумму денег. Далее они платят постоянные издержки и участвуют в торгах за сырье и материалы. Банк контролирует источники сырья и скупает готовую продукцию.

Игроки подают заявки на приобретение сырья и материалов. Они производят продукцию и продают ее банку. Игроки также выплачивают ссудный процент и погашают ссуды. Кроме того, они могут строить новые фабрики и автоматизировать существующие.

Игра заканчивается после фиксированного числа кругов или когда все игроки обанкротятся. Общий капитал компании подсчитывается по стоимости фабрик, сырья и продукции. Победителем становится игрок, набравший наибольший капитал.

1.1 Описание предметной области

1.1.1 Основные понятия и определения

Игра «Менеджмент» оперирует рядом ключевых понятий, характеризующих базовые элементы и процессы управления современной организацией:

- единицы сырья и материалов (ЕСМ) – это ограниченный ресурс, необходимый для производства готовой продукции;

- единицы готовой продукции (ЕГП) – это готовая продукция, которую банк закупает у игроков по определенной максимальной цене;
- постоянные издержки – ежемесячные выплаты, которые каждый игрок (президент компании) должен производить банку за владение ЕСМ, ЕГП, обычными и автоматизированными фабриками;
- старший игрок – игрок, который будет председательствовать в следующем раунде торгов;
- ссуда – это заем, предоставляемый банком игрокам для финансирования их деятельности.

1.1.2 Классификация экономических игр

Экономические игры представляют собой один из наиболее популярных и востребованных видов деловых игр. Они позволяют моделировать различные аспекты экономической деятельности и дают участникам возможность ощутить себя в роли руководителей предприятий, банкиров и инвесторов. В рамках этого обширного класса игр можно выделить несколько основных типологических групп [7].

Во-первых, это ролевые игры, в которых участники выступают в роли руководителей виртуальных компаний или организаций, принимая управленческие решения, где основной фокус делается на управлении финансами, ресурсами и производственными мощностями. Это наиболее распространенный тип экономических игр, позволяющий участникам освоить навыки оперативного и стратегического управления различными аспектами деятельности виртуальных предприятий. Примером такой игры может служить «Менеджмент». Это наиболее распространенный тип экономических игр, позволяющий участникам освоить навыки оперативного и стратегического управления различными аспектами деятельности виртуальных предприятий.

Второй тип – это игры с противоборством, предполагающие наличие соперничества между игроками за ограниченные ресурсы и рыночные доли.

Яркий пример такой игры – «Монополия», в которой участники ведут борьбу за монопольное доминирование на рынке недвижимости.

Помимо типа игры, экономические игры также различаются по характеру взаимодействия между участниками. Существуют многопользовательские игры, в которых несколько игроков взаимодействуют друг с другом, моделируя реальную конкурентную среду. Другие игры предполагают распределенное управление, когда каждый игрок самостоятельно принимает решения по управлению своей компанией, как в случае с «Менеджментом». Такие игры обычно предполагают центрального управляющего, когда «банк» или другой регулирующий орган выступает в качестве координатора.

Наконец, экономические игры могут различаться и по временной структуре. Пошаговые игры разделяют процесс на фиксированные временные интервалы, в рамках которых участники принимают решения, как в «Менеджменте». В других играх уровни спроса, предложения, цен и другие параметры изменяются в соответствии со случайными процессами, имитирующими реальные рыночные колебания.

Помимо этого, экономические игры можно классифицировать и по отраслевому признаку. Выделяются производственно-экономические игры, фокусирующиеся на управлении производственными ресурсами и процессами («Бизнес-курс», «Фабрика»), финансово-экономические игры, акцентирующие внимание на управлении финансовыми инструментами и операциями («Монополия», а также макроэкономические игры, моделирующие функционирование национальной или региональной экономики в целом («Градостроитель»)).

Данная классификация позволяет выделить основные типы экономических игр и определить место в ней конкретных игровых систем, таких как «Менеджмент», «Монополия», «Бизнес-курс» и других. Каждый тип игр имеет свои особенности и предназначен для решения определенных учебных или исследовательских задач.

1.1.3 Требования к экономической игре «Менеджмент»

Игровой процесс в «Менеджменте» должен иметь пошаговую структуру с фиксированными временными интервалами, например, месячными периодами. Это позволит участникам самостоятельно управлять различными функциональными аспектами деятельности виртуального предприятия, таких как производство, финансы и т.д., отслеживая взаимосвязи между ними.

Важным требованием к игре является необходимость имитации реальных бизнес-процессов и экономических механизмов. Это позволит участникам принимать решения в условиях, максимально приближенных к реальной экономической среде.

Эффективность игры "Менеджмент" должна обеспечиваться за счет наличия в ней развитого аналитического инструментария. Участники должны иметь возможность после окончания игры увидеть анализ деятельности предприятий, опираясь на систему ключевых показателей эффективности, а также средства визуализации и представления игровых результатов.

Не менее важно, чтобы экономическая игра "Менеджмент" обладала гибкостью и возможностью настройки под специфические требования.

Поддержка многопользовательского режима также является важным требованием для создания конкурентной игровой среды.

Совокупность перечисленных требований отражает ключевые аспекты, которым должна соответствовать экономическая игра «Менеджмент» для эффективного достижения поставленных учебных и игровых целей.

1.2 Описание систем-аналогов

Существует ряд экономических игр, схожих по своим характеристикам с «Менеджментом» и моделирующих различные аспекты управленческой деятельности. Среди них – «Бизнес-курс», которая позволяет участникам моделировать деятельность виртуального производственного предприятия. Схожие по типу задачи решает игра «Фабрика», имитирующая

производственно-хозяйственную деятельность и требующая от участников принятия решений по ключевым функциям.

Другое направление представляют финансово-ориентированные системы, такие как «Акционер», моделирующая деятельность на финансовых рынках. Также существует игра «Финансовый менеджмент», фокусирующаяся на управлении финансовыми ресурсами виртуальной компании.

Обязательно стоит упомянуть и, пожалуй, самую известную экономическую игру – «Монополию». Хотя она не имеет таких развитых управленческих механизмов, как рассмотренные выше, «Монополия» моделирует базовые экономические процессы – купля-продажа недвижимости, сдача в аренду, управление денежными потоками. Эта игра стала классикой в жанре экономических настольных игр.

Перечисленные игровые системы, каждая со своей спецификой, могут рассматриваться в качестве близких аналогов «Менеджмента» и использоваться для сравнения и оценки его возможностей в контексте обучения управленческим компетенциям.

1.2.4 Компьютерная деловая игра «Бизнес-курс»

«Бизнес-курс» является одной из ведущих экономических игр, схожих по своим характеристикам с «Менеджментом». Она была разработана специалистами Московского государственного университета им. М.В.Ломоносова в 1993 году и относится к классу производственно-экономических игр. Основная цель «Бизнес-курса» – дать участникам возможность смоделировать деятельность виртуального производственного предприятия и отработать навыки принятия управленческих решений в различных функциональных областях [8].

В процессе игры участники выступают в роли топ-менеджеров предприятия и принимают решения по широкому спектру вопросов – от планирования и организации производственного процесса до управления

финансами, закупок сырья и материалов, организации сбыта продукции. Одной из важных особенностей игры является возможность создания конкурентной среды между участниками, которые могут выступать в качестве руководителей конкурирующих предприятий.

Игровой процесс в «Бизнес-курсе» построен на пошаговой структуре с фиксированными временными интервалами, как правило, месячными периодами. Это позволяет участникам последовательно принимать управленческие решения, отслеживать их влияние на показатели деятельности предприятия и вносить необходимые корректировки. Важным элементом игры является развитая система ключевых показателей эффективности (KPI), которые используются для оценки результатов управленческих действий участников с различных сторон – финансовой, производственной, маркетинговой и других.

Таким образом, «Бизнес-курс» представляет собой комплексную производственно-экономическую игру, моделирующую деятельность виртуального предприятия [5]. На рисунке 1 приведена экранная форма модуля продажи готовой продукции программы «Бизнес-курс», на которой можно сформировать доходную часть игровой компании.

	Продукт А	Продукт В	Продукт С	Всего
Рынок продукции (прошлый месяц)				
Средняя цена единицы продукции (без НДС)	59,478	1,028	---	---
Среднее качество продукции	1,00	1,01	---	---
Средние расходы на рекламу (без НДС)	3 632	837	---	---
Средняя ставка комиссионных по сбыту, %	5,17	5,08	5,23	---
Производство предприятия				
Спрос на продукцию в прошлом месяце, ед	0	0	0	---
Запас продукции на начало тек. месяца, ед	1 560	0	0	---
Удельная производственная себестоимость	48,685	0,000	0,000	---
Уд. полная себестоимость в прошлом месяце	0,000	0,000	0,000	---
Качество продукции на складе	1,58	0,00	0,00	---
Решения				
Цена единицы продукции (без НДС)	80,000	0,000	0,000	---
Расходы на рекламу	10 000	0	0	---
Ставка комиссионных по сбыту, %	10,00	---	---	---
Период рассрочки оплаты, мес.	0	0	0	---
Доля факторинга, %	0,00	0,00	0,00	---
Прогноз				
Спрос на продукцию в текущем месяце, ед.	1 560	0	0	---
Объем продаж, ед.	1 560	0	0	---
Выручка от продажи продукции (без НДС)	124 800	0	0	124 800
Полная себестоимость продукции:	105 075	0	0	105 075
- производственная себестоимость	75 948	0	0	75 948
- коммерческие расходы	23 999	0	0	23 999
- управленческие расходы	5 128	0	0	5 128
Удельная полная себестоимость	67,356	0,000	0,000	---
Прибыль (убыток) от продажи продукции	19 725	0	0	19 725
Денежные поступления в текущем месяце (с НДС):	147 264	0	0	147 264
- от покупателей по продажам в текущем месяце	147 264	0	0	147 264
- от банка по факторингу в текущем месяце	0	0	0	0
- от покупателей по продажам в прошлых месяцах	0	0	0	0
Проценты по факторингу:	0	0	0	0
- учитываемые при налогообложении	0	0	0	0
- не учитываемые при налогообложении	0	0	0	0
Ставка процента по факторингу, % 51,0 Предельная ставка процента по НК, % 20,1				

Рисунок 1 – Экранная форма программы «Бизнес-курс»

К достоинствам данной системы относятся:

- комплексный подход к моделированию управленческой деятельности;

- развитая система ключевых показателей эффективности (KPI);
- практическая ценность для развития управленческих компетенций.

К недостаткам системы относятся:

- сложность для неподготовленных участников;
- ограниченная гибкость и адаптивность.

1.2.5 «Монополия»

«Монополия» – широко известная экономическая настольная игра, которую также можно отнести к числу аналогов для «Менеджмента». Она была разработана в 1935 году Чарльзом Дароу и с тех пор стала одной из самых популярных игр в мире. В «Монополии» игроки выступают в роли бизнесменов, ведущих свою предпринимательскую деятельность на виртуальном игровом поле, представляющем собой карту города с недвижимостью, улицами и другими элементами. Основная задача участников – приобретать недвижимость, развивать ее, сдавать в аренду и в конечном счете добиться полного банкротства остальных игроков.

Ключевые управленческие аспекты, с которыми приходится сталкиваться участникам «Монополии», включают выбор и приобретение наиболее выгодных объектов недвижимости, инвестирование в их развитие и улучшение, управление денежными потоками, кредитами и расчетами с другими игроками, принятие решений о сдаче недвижимости в аренду и установлении арендных ставок, а также реагирование на непредвиденные события, вызванные различными игровыми ситуациями.

Игровой процесс в «Монополии» основан на последовательном перемещении фишек игроков по полю и совершении действий в соответствии с выпавшими игровыми событиями. Это позволяет участникам неоднократно принимать управленческие решения, отслеживать их последствия и корректировать свою стратегию. Несмотря на относительную простоту правил, «Монополия» позволяет моделировать различные аспекты предпринимательской деятельности и развивать навыки экономического

мышления, принятия решений и управления ресурсами, что делает ее ценным аналогом для «Менеджмента» в части развития управленческих компетенций [6].

На рисунке 2 приведена главная экранная форма программы «Монополия», на которой приведен игровой полигон с основными игровыми элементами. Центральное место занимает карта города с различными объектами недвижимости, по которой передвигаются фишки игроков. На информационных панелях отображается текущее состояние участников, включая их денежные средства.

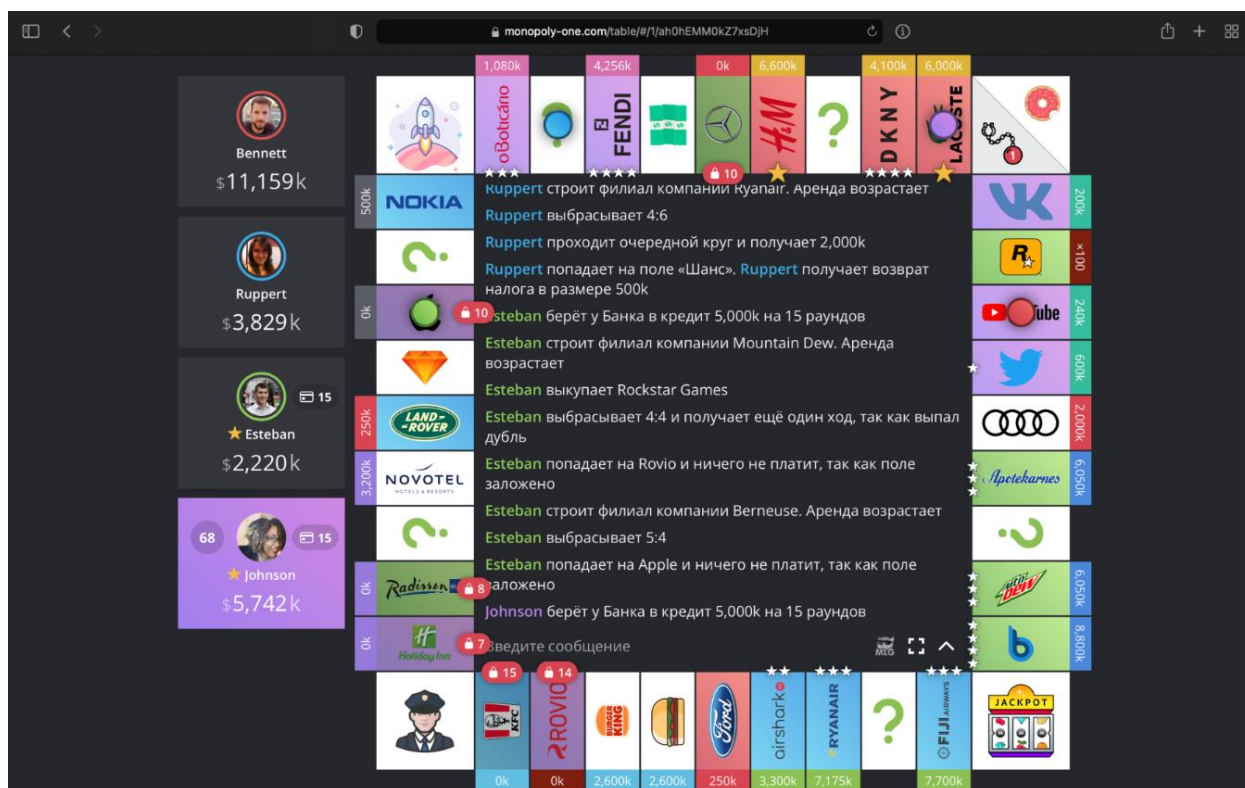


Рисунок 2 – Экранная форма программы «Монополия»

К достоинствам данной системы относятся:

- доступность и простота правил;
- реалистичное моделирование рыночных механизмов;
- высокая увлекательность и интерактивность.

К недостаткам системы относятся:

- высокая зависимость от случайности;
- длительность партий.

1.2.6 «Кондитерская фабрика»

«Кондитерская фабрика» – это симулятор управления семейным бизнесом по производству кондитерских изделий, который позволяет игрокам развить ключевые навыки менеджмента. Цель игры заключается в том, чтобы из небольшого семейного предприятия создать крупный кондитерский холдинг с высокой прибыльностью и множеством производственных линий.

В роли владельца кондитерской фабрики, игроки последовательно принимают стратегические и оперативные управленческие решения, направленные на развитие и масштабирование бизнеса. Они инвестируют в приобретение нового высокотехнологичного оборудования, автоматизируют производственные процессы, расширяют складские мощности и торговую сеть, а также нанимают и управляют командой высококвалифицированных менеджеров.

Ключевые управленческие аспекты, с которыми сталкиваются участники, включают разработку долгосрочных планов развития фабрики, определение приоритетных направлений инвестиций и постановку количественных целей. Кроме того, игрокам необходимо оптимизировать производственные линии, внедрять автоматизацию и контролировать выпуск продукции для максимизации объемов и минимизации издержек. Важной задачей является также управление персоналом – игроки должны нанимать, мотивировать и развивать команду профессиональных менеджеров, делегируя им полномочия по оперативному управлению. Наконец, участникам приходится отслеживать финансовые показатели, принимать решения об инвестициях, кредитовании и распределении прибыли, а также развивать торговую сеть, проводить рекламные акции и ценовую политику для увеличения объемов реализации.

Таким образом, «Кондитерская фабрика» является всесторонним симулятором управления предприятием, который позволяет игрокам приобрести практический опыт в области стратегического,

производственного, финансового и кадрового менеджмента, что делает её ценным инструментом для курса "Менеджмент" [7].

На рисунке 3 приведена главная экранная форма программы «Кондитерская фабрика», на которой приведен игровой полигон с основными игровыми элементами.

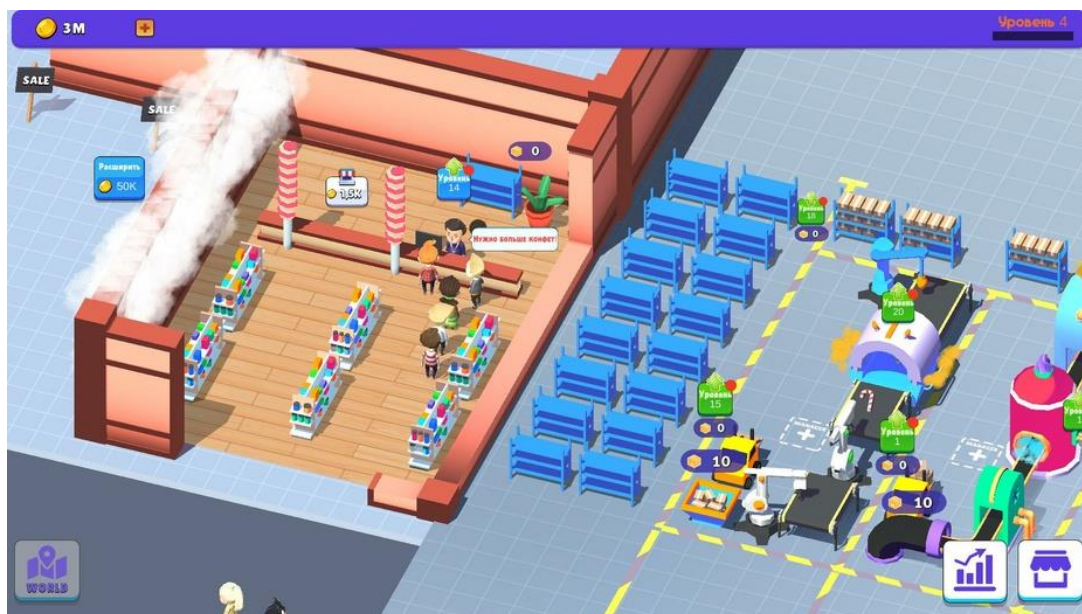


Рисунок 3 – Экранная форма программы «Фабрика»

К достоинствам данной системы можно отнести наглядность и интерактивность.

К недостаткам системы относятся:

- упрощение реальных бизнес-процессов;
- отсутствие острой конкуренции;
- ограниченная возможность анализа.

На основании анализа возможностей систем-аналогов были сформулированы требования к разрабатываемой системе (см. таблицу 1).

1.3 Диаграмма объектов предметной области

Объектно-ориентированный подход к проектированию программных систем основывается на моделировании реальных сущностей предметной области в виде объектов со своими атрибутами и методами.

Таблица 1 – Сравнительные характеристики систем-аналогов

Название системы Название показателя	«Бизнес-курс»	«Монополия»	«Кондитерская фабрика»	Разрабатываемая система
Предоставление справочной информации о правилах игры	+	—	—	+
Удобный интерфейс	—	+	+	+
Формирование статистики игроков завершившейся игры	+	—	—	+
Графика и визуализация	—	—	+	+
Сложность в освоении	+	—	—	—

Ключевые принципы объектно-ориентированного программирования (ООП), такие как инкапсуляция, наследование, полиморфизм и абстракция, позволяют создавать модульные, масштабируемые и легко поддерживаемые приложения. Объектный подход обеспечивает более наглядное представление логики программы, способствуя ее лучшему пониманию и упрощая процесс разработки.

В контексте разрабатываемого приложения, методология ООАП применяется для проектирования его объектной модели. Ключевые сущности предметной области и их отношения отображаются на диаграмме объектов, демонстрируя, как объектно-ориентированные принципы обеспечивают гибкость, масштабируемость и удобство поддержки архитектуры приложения.

На рисунке 4 приведена диаграмма объектов предметной области, которая демонстрирует основные взаимосвязи между ключевыми сущностями игры "Менеджмент", отражая ключевые управленческие решения, которые Игрок может принимать в ходе игрового процесса.

Ключевыми сущностями игры «Менеджмент» являются Игрок, Завод, ЕСМ, ЕГП и Банк. Игрок выступает в роли основного участника, принимающего управленческие решения в ходе игры.

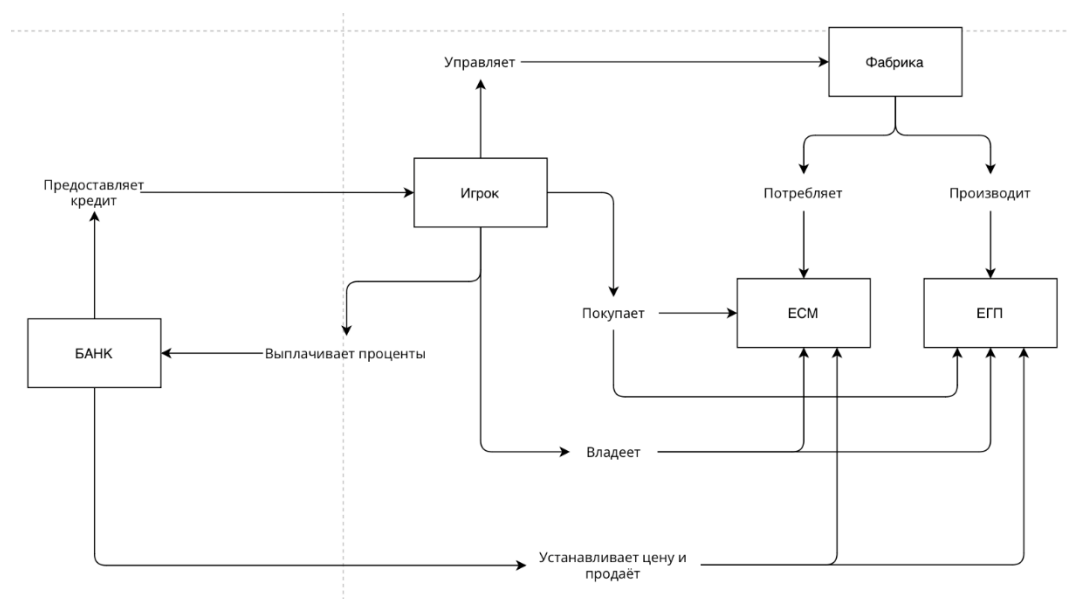


Рисунок 4 – Диаграмма объектов предметной области игры «Менеджмент»

Фабрика представляет собой производственную единицу, управляемую Игроком. Фабрика потребляет ЕСМ и производит ЕГП. Игрок может

принимать решения о том, как распоряжаться ресурсами Завода – потреблять ЕСМ для производства ЕГП или приобретать ЕСМ у Банка. Кроме того, Игрок может распределять произведенные ЕГП, продавая их Банку или используя для других целей.

Банк является финансовым учреждением, предоставляющим Игроку необходимые кредиты для финансирования деятельности Фабрики. Банк также определяет цены на ЕСМ и ЕГП, и сам может выступать в качестве покупателя или продавца этих ресурсов.

Игрок, являющийся основным участником игры, характеризуется своим именем, игровым балансом и кредитной историей. Он может управлять фабрикой, приобретать необходимые ЕСМ, распоряжаться произведенными ЕГП, а также получать кредиты в Банке.

1.4 Постановка задачи

Во время курсового проектирования необходимо разработать игру «Менеджмент» – виртуальную экономическую систему, в которой игроки принимают управленческие решения и реализуют стратегии для достижения финансового успеха. Система должна быть реализована в виде веб-приложения.

Игрок будет выступать в роли руководителя предприятия по производству продукции. Он будет управлять работой завода, контролируя необходимые для производства сырьевые материалы, процесс изготовления готовой продукции и продажу этой продукции на рынке.

Моделирование ведётся с шагом по времени в один месяц. В начале игры каждый игрок получает две обычные фабрики, четыре единицы сырья и материалов две единицы готовой продукции и 10000 долларов наличными. Игроки занумерованы от 1 до N, и в первом круге игрок 1 – старший. С каждым кругом (то есть ежемесячно) роль старшего переходит к следующему по порядку номеров игроку, после N-го старшим становится опять первым. На торгах при прочих равных условиях выигрывает самый старший игрок.

Описанные ниже сделки происходят каждый месяц и именно в таком порядке. Если в какой-то момент компания не может выполнить своих финансовых обязательств, она немедленно объявляется банкротом. Её имущество пропадает, и она выходит из игры. Все денежные расчёты происходят между отдельными игроками и одним общим банком. Невозможна передача денег прямо от игрока к игроку, что исключает сговор, направленный против какой-либо компании. Банк кроме того, контролирует источники сырья и скупает всю готовую продукцию.

Каждый игрок (в порядке убывания старшинства, начиная со старшего) платит 300 долларов за каждую имеющуюся у него ЕСМ, 500 долларов за каждую наличную ЕГП, 1000 долларов за владение каждой обычной фабрикой и 1500 долларов – за владение автоматизированной. Это постоянные ежемесячные издержки каждого игрока, даже если он в этом круге не предпринимает никаких других действий.

Банк решает и сообщает игрокам, сколько ЕСМ продаст в этот раз и какова их минимальная цена. Объявляется также, сколько ЕГП в общей сложности будет закуплено и какова максимальная цена. В табл. 2 приведены пять уровней предложения ЕСМ и спроса на ЕГП, а также верхние и нижние границы цен для каждого случая.

Таблица 2 – Уровни цен на ЕСМ и ЕГП

Уровень	ЕСМ	Минимальная цена	ЕГП	Максимальная цена
1	1.0P	\$800	3.0P	\$6500
2	1.5P	650	2.5P	6000
3	2.0P	500	2.0P	5500
4	2.5P	400	1.5P	5000
5	3.0P	300	1.0P	4500

В число игроков P не включены те, кто обанкротился, и P может, таким образом, быть меньше N. Произведения 1.5P и 2.5P округляются до ближайшего целого с недостатком. В табл. 3 приведена матрица вероятностей перехода, в соответствии с которой банк определяет новый месячный уровень

спроса и предложения, исходя из прежнего. Предполагается, что в нулевом месяце уровень равен 3.

Таблица 3 – Уровни цен на ЕСМ и ЕГП

Старый уровень	Новый уровень				
	1	2	3	4	5
1	1/3	1/3	1/6	1/12	1/12
2	¼	1/3	¼	1/12	1/12
3	1/12	¼	1/3	¼	1/12
4	1/12	1/12	¼	1/3	¼
5	1/12	1/12	1/6	1/3	1/3

Каждый игрок тайно от других готовит заявку на ЕСМ на текущий месяц. В заявке указывается требуемое число ЕСМ и предлагается цена не ниже банковской минимальной (запрос нуля ЕСМ или предложение цены ниже минимальной автоматически исключает игрока из торгов в этом месяце). Все заявки раскрываются одновременно, и имеющиеся ЕСМ распределяются между игроками в порядке убывания предложенной цены. Если сырья не хватает на всех, заявки с предложением более низкой цены не удовлетворяются. При прочих равных условиях сырьё достаётся самому старшему игроку. Игроки платят за сырьё при его получении.

Все игроки по очереди (по убыванию старшинства, начиная со старшего) объявляют, сколько ЕСМ они собираются переработать в ЕГП в текущем месяце и на каких фабриках. Каждый игрок тут же обязан покрыть расходы на производство. Обычная фабрика может за месяц переработать одну ЕСМ при затратах 2000 долларов. Автоматизированная фабрика может либо сделать то же, либо переработать 2 ЕСМ при затратах в 3000 долларов. Конечно, чтобы переработать ЕСМ, их надо иметь.

При покупке банком у игроков ЕГП организуются примерно такие же торги, как и при продаже ЕСМ. Заявленные цены не должны превышать максимальную цену, установленную банком, причём банк покупает ЕГП в первую очередь у тех, кто заявил более низкую цену. При прочих равных

условиях предпочтение отдаётся старшему игроку. Если предпочтение превышает спрос, наиболее дорогие ЕГП остаются непроданными. Игроки получают деньги за продукцию при её продаже.

Каждый игрок платит один процент от общей суммы непогашенных ссуд, в том числе и тех, которые будут погашены в текущем месяце.

Каждый игрок, получивший ссуду до текущего месяца, должен её погасить. Поскольку возвращение ссуд предшествует получению новых, платить надо наличными.

Теперь каждый игрок может получить ссуду. Ссуды обеспечиваются имеющимися у игрока фабриками; под обычную фабрику даётся ссуда 5000 долларов, под автоматизированную – 10000 долларов. Общая сумма непогашенных ссуд не может превышать половины гарантированного капитала, но в этих пределах можно свободно занимать. Банк немедленно выплачивает ссуду игроку. Срок погашения ссуды истекает через 12 месяцев например, ссуду, взятую в 3-м месяце, возвращать надо в 15-м. Нельзя погашать ссуды раньше срока.

Игроки могут строить новые фабрики. Обычная фабрика стоит 5000 долларов и начинает давать продукцию на 5-й месяц после начала строительства; автоматизированная фабрика стоит 10000 долларов и даёт продукцию на 7-й месяц после начала строительства. Обычную фабрику можно автоматизировать за 7000 долларов, реконструкция продолжается 9 месяцев, всё это время фабрика может работать как обычная. Половину стоимости фабрики надо платить в начале строительства, вторую половину – за месяц до начала выпуска продукции в этой же фазе цикла. Общее число имеющихся и строящихся фабрик у каждого игрока не должно превышать шести.

Игра оканчивается после некоторого фиксированного числа кругов (13 и более) или когда обанкротятся все игроки, кроме одного. Чтобы подсчитать общий капитал компании, надо сложить стоимость всех фабрик (по цене, по которой их можно было бы построить заново), стоимость имеющихся у неё

ЕСМ (по минимальной банковской цене текущего месяца), стоимость имеющихся ЕГП (по максимальной банковской цене текущего месяца) и имеющиеся у компании наличные. После этого надо вычесть общую сумму ссуд и предстоящих расходов по уже начатому строительству. Если к концу игры приходят несколько игроков, результаты считаются по их капиталам.

Любой игрок в любой момент может узнать состояние дел другого игрока – его капитал, наличные, взятые ссуды, всё, что касается готовой продукции, имеющихся и строящихся фабрик. Во время торгов игроки ничего не знают о заявках, сделанных другими, но, как только банк собрал все заявки, они обнародуются, и количество купленных или проданных каждым игроком единиц становится известно всем. Игроки могут сами вести любые записи, но банк не предоставляет им никакой информации, кроме той, которая предусмотрена правилами игры.

Таким образом, система должна решать следующие задачи:

1) функции системы:

- контроль подключения игроков к сессии;
- контроль времени выполнения хода;
- контроль подачи заявок;
- реализация игровых правил и логики игрового процесса;
- обработка действий игроков и изменение состояния игры;
- предоставление интерфейса для выбора действий игроками;
- отображение актуальной игровой ситуации для каждого игрока;
- ведение учета активов и капитала каждого игрока;
- фиксация всех действий игроков в течение игры;
- оплата постоянных издержек;
- оплата ссудного процента;
- погашение ссуд;
- возможность просмотра открытой информации о других

участниках – капитал, наличные, взятые ссуды, всё, что касается готовой продукции, имеющихся и строящихся фабрик;

- выдача справочной информации;

2) функции банка:

- формирование цен на ЕГП и ЕСМ;
- рассмотрение заявок игроков на продажу ЕГП и покупку ЕГП;
- рассмотрение заявок игроков на продажу ЕСМ и покупку ЕСМ;

3) функции игрока:

- регистрация пользователя в системе (ввод никнейма и выбор аватара);
- подключение к игровой сессии;
- создание собственной сессии;
- выбор доступных игровых действий, в зависимости от игровой ситуации:

6) подача заявки на получение сырья и материалов (необходимое количество ЕСМ и предлагаемую цену);

7) выбор количества для переработки ЕСМ в ЕГП;

8) подача заявки на продажу продукции ЕГП и предлагаемую цену;

9) выбор ссуды;

10) подача заявки на строительство.

- просмотр лога игры после очередного хода;
- пропуск хода;
- признание себя банкротом;
- просмотр справочной информации.

2 Проектирование системы

На этапе проектирования системы решаются несколько ключевых задач, которые помогают сформировать четкое представление о том, как будет выглядеть и функционировать конечный продукт. В первую очередь, важно определить требования, как функциональные, так и нефункциональные. Это включает в себя сбор информации от пользователей и заинтересованных сторон, что позволяет понять их потребности и ожидания.

Следующим шагом является архитектурное проектирование, где выбирается подходящий архитектурный стиль, например, клиент-сервер или микросервисы. На этом этапе также определяются основные компоненты системы и их взаимодействие, что закладывает основу для дальнейшей разработки.

Моделирование данных играет важную роль в проектировании, так как необходимо разработать структуру базы данных и определить связи между сущностями. Параллельно с этим идет работа над пользовательскими интерфейсами. Создаются прототипы и макеты, чтобы визуализировать, как пользователи будут взаимодействовать с системой.

Выбор технологий также является важной задачей на этом этапе. Необходимо определить стеки технологий, включая языки программирования и фреймворки, которые будут использоваться в проекте. При этом стоит обратить внимание на безопасность: разрабатываются меры по защите данных и определяются механизмы аутентификации.

Таким образом, этап проектирования системы включает в себя комплексный подход к созданию структуры и функциональности продукта, что является основой для его успешной разработки и внедрения.

2.1 Выбор и обоснование архитектуры системы

Архитектура системы – это высокоуровневое структурное описание программного обеспечения, которое определяет его компоненты, их взаимодействие и организацию. Она включает в себя выбор технологий,

паттернов проектирования и принципов, которые будут использоваться для реализации приложения. Архитектура приложения обеспечивает основу для разработки, масштабируемости, производительности и поддержки, а также помогает в управлении сложностью системы, обеспечивая ее надежность и гибкость.

Существует несколько видов архитектуры, каждый из которых представляет собой определенный подход к организации и структуре программных систем. Опишем наиболее популярные виды архитектур.

Монолитная архитектура – это традиционный подход, при котором все компоненты приложения развернуты в одной единственной программе. Весь код находится вместе, и все компоненты взаимодействуют напрямую друг с другом. Такой подход может быть простым в начале разработки, но с течением времени и ростом приложения, монолитная архитектура может стать менее подходящей из-за сложности сопровождения и масштабирования.

Микросервисная архитектура – подход, при котором приложение разделяется на маленькие, автономные и независимые сервисы, которые взаимодействуют между собой посредством сетевых вызовов. Каждый сервис выполняет определенные функции и может быть разработан, развернут и масштабирован отдельно от других сервисов.

Одноуровневая архитектура (Single-tier) – вид архитектуры, в котором пользовательский интерфейс, бизнес-логика и хранение данных объединены в одной программе. Такой подход обычно используется для небольших приложений или прототипов.

Многоуровневая архитектура (Multi-tier) – это подход, в котором приложение разделяется на несколько слоев, обычно три основных слоя: представление (пользовательский интерфейс), бизнес-логика и хранение данных. Каждый слой выполняет определенные функции и взаимодействует с другими слоями через строго определенные интерфейсы.

Событийно-ориентированная архитектура – вид архитектуры, в котором компоненты приложения обмениваются сообщениями (событиями) для

уведомления друг друга о происходящих событиях. Это позволяет создавать распределенные и гибкие системы.

Слоистая архитектура (Layered Architecture) – подход разделяющий приложение на горизонтальные слои, где каждый слой выполняет определенные функции. Например, пользовательский интерфейс, бизнес-логика и слой доступа к данным.

Клиент-серверная архитектура (КСА) – это распространенный подход, при котором приложение разделяется на две основные части – клиентскую, которая обрабатывает пользовательский интерфейс, и серверную, которая управляет бизнес-логикой и хранит данные. Существует несколько видов клиент – серверных архитектур.

Так, модель, в которой клиент и сервер разделены на два уровня – это двухуровневая архитектура. Клиентская часть отвечает за пользовательский интерфейс и взаимодействие с пользователем, а серверная – за обработку данных и бизнес-логику. Особенности работы заключаются в том, что на сервер приходит определенный запрос, потом его обрабатывают и дают напрямую, без дополнительного применения группы внешних ресурсов.

На рисунке 5 представлена схема данной архитектуры.

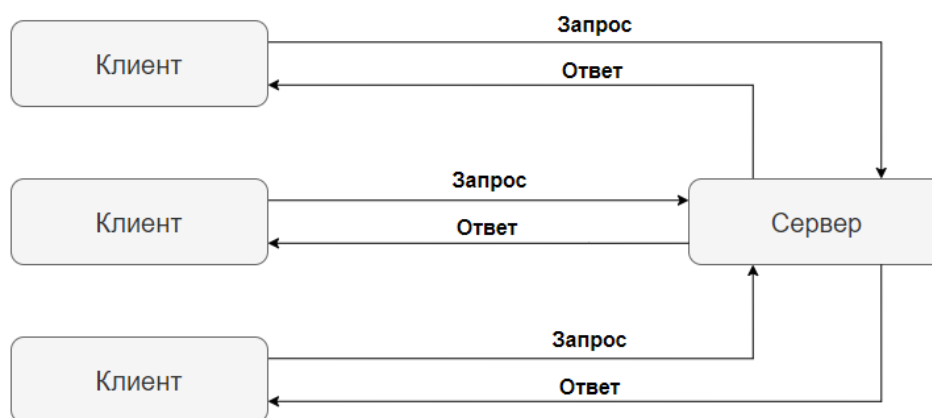


Рисунок 5 – Схема двухуровневой архитектуры

Так же существует трехуровневая модель, которая разделяет приложение на три уровня: клиентский (презентационный), серверный (бизнес-логика) и уровень данных (база данных). Сервер в этом случае занимается

исключительно обработкой сведений, обращаясь к БД за получением либо сохранением информации. В трехуровневой КСА клиентские запросы могут обрабатывать одновременно несколько серверов, благодаря чему значительно снижается нагрузка на систему. Схема трехуровневой архитектуры представлена на рисунке 6.

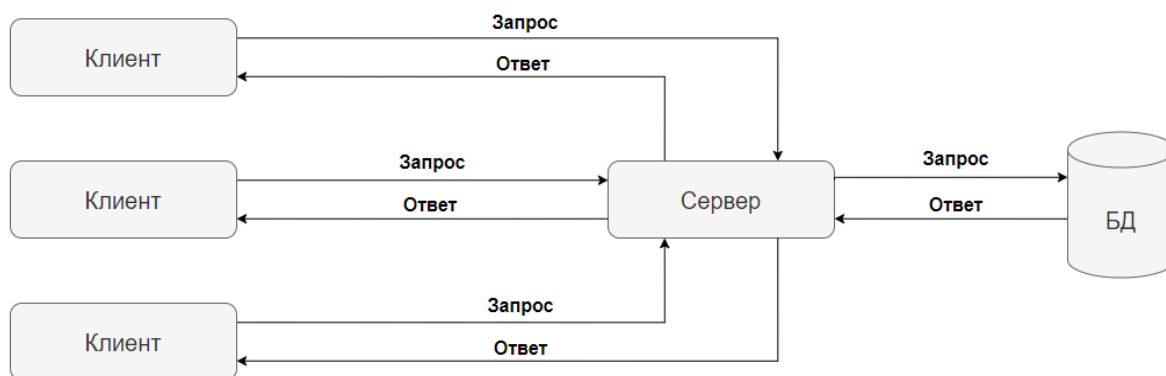


Рисунок 6 – Схема трехуровневой архитектуры

Пожалуй, самым распространенным примером клиент-серверной архитектуры является веб-приложение, где логика приложения распределена между сервером и клиентом, хранение данных осуществляется преимущественно на сервере, обмен информацией происходит по сети. Одним из преимуществ такого подхода является тот факт, что клиенты не зависят от конкретной операционной системы пользователя, поэтому веб-приложения являются межплатформенными службами [9].

Клиенты и серверы взаимодействуют по сети через определенные протоколы. Протокол – это набор правил, по которым передаются данные. С помощью протоколов связываются между собой компьютеры в сети, разные устройства и программы. Они выполняют определенные правила и поэтому понимают друг друга. Протоколы описывают, в каком формате будут передаваться данные, как будет выглядеть процедура их передачи и что делать с ошибками, если они возникнут.

Протоколов передачи данных существует огромное множество [8]. Самый распространенный протокол – HyperText Transfer Protocol (HTTP), создавался для передачи HTML-файлов, но впоследствии пригодился для всех

файлов в сети. Этот протокол клиент-серверного типа не сохраняет промежуточное состояние. Для обмена информацией обычно используется TCP/IP. HTTPS является защищённой версией протокола HTTP. Она поддерживает шифрование, а данные передаются поверх криптопротокола TLS.

Не менее распространенный протокол – Internet Protocol (IP, досл. «межсетевой протокол»), являющийся маршрутизируемым протоколом сетевого уровня стека TCP/IP. IP объединяет сегменты сети в единую сеть, обеспечивая доставку пакетов данных между любыми узлами сети через произвольное число промежуточных узлов.

File Transfer Protocol (FTP) – протокол прикладного уровня для передачи файлов, появившийся в 1971 году. Использует два канала для передачи данных. Первый, управляющий процессом передачи, называют командным. Второй, передающий информацию — транспортным. FTP — надёжный протокол. Сервер в данном случае называют удалённым хостом, а клиент — локальным. Работает по клиент-серверной модели.

Transmission Control Protocol (TCP) – протокол сквозной связи, созданный в 1974 году и до сих пор востребованный в мире. Надёжный, но медленный. Перед тем, как начинать передачу данных, достигается рукопожатие для установления соединения, и лишь затем начинается передача пакетов. При необходимости пакеты дублируются;

WebSocket – протокол связи поверх TCP-соединения, предназначенный для обмена сообщениями между браузером и веб-сервером, используя постоянное соединение. Протокол WebSocket позволил сделать соединение двунаправленным и постоянным. Сервер может не только отвечать на запрос клиента, но и самостоятельно передавать новую информацию по мере ее поступления. Обмен данными происходит в рамках одного установленного соединения в режиме реального времени.

Secure Shell (SSH) – защищённый протокол прикладного уровня, необходимый для удалённого управления ОС через протокол TCP. В SSH весь

трафик шифруется с использованием заданного вами алгоритма шифрования. Протокол позволяет обрабатывать другие сетевые протоколы передачи, включая аудио- или видеопоток.

User Datagram Protocol (UDP) – протокол передачи данных, не требующий предварительной установки соединения между хостами. Он быстрый, но часто теряет пакеты данных во время доставки.

В клиент-серверной архитектуре (КСА) можно выделить два типа клиента:

- КСА с “тонким” клиентом, где большая часть логики приложения выполняется на сервере, а клиентская сторона выполняет только представление данных и пользовательский интерфейс, таким образом сервер обрабатывает запросы от клиента и отправляет ему результаты;

- КСА с “толстым” клиентом, где клиентская сторона обрабатывает большую часть логики приложений, чем в предыдущем типе и выполняет множество операций независимо от сервера, однако сервер обрабатывает только специфические запросы от клиента.

Таким образом, для разработки игры «Менеджмент» наиболее подходящей можно назвать клиент-серверную архитектуру с “тонким” клиентом, так как она позволяет снизить требования к клиентским устройствам и упростить контроль над игровым процессом, минимизировав необходимость в обновлениях на стороне клиента. Кроме того, ввиду отсутствия необходимости хранить данные пользователей в базе данных было решено выбрать двухуровневую архитектуру, предоставляющую более быстрое время отклика и упрощающую разработку приложения. Для передачи файлов с гипертекстовой разметкой нам понадобится протокол HTTP, протокол WebSocket для организации двунаправленного соединения между клиентом и сервером и протокол IP для маршрутизации пакетов данных между устройствами, обеспечивая их доставку от источника к получателю.

2.2 Структурная схема системы

На этапе построения структурной схемы система по функциональному признаку разделяется на основные подсистемы, между ними указываются информационные связи и связи по управлению, описывается основное назначение подсистем. При разработке структурной схемы используется методология структурного проектирования, в основе которой лежит алгоритмическая декомпозиция и иерархия вида «часть-целое», учитывающая, что внутренние связи элементов внутри подсистем сильнее, чем связь между подсистемами. Декомпозиция системы может повторяться многократно, вплоть до уровня конкретных процедур, при этом должна быть обеспечена целостность системы, а все составляющие компоненты взаимоувязаны. Для этого используются такие принципы разработки, как «сверху-вниз», «разделяй и властвуй», «иерархическое упорядочивание» и другие [9]. Система – множество элементов, находящихся в отношениях и связях друг с другом, которое образует определённую целостность, единство. Систему можно рассматривать как набор подсистем, организованных для достижения определённой цели и описанных с помощью набора моделей, а подсистему – как группу элементов, часть которых составляет спецификацию поведения, представленного другими ее составляющими. На рисунке 7 приведена структурная схема разрабатываемой системы.

В состав клиентской части систем входят следующие подсистемы:

- подсистема вывода данных отвечает за отображение информации пользователю. Она получает данные из серверной части через API, обрабатывает их для удобства визуализации. Эта подсистема также обеспечивает динамическое обновление контента без перезагрузки страницы.

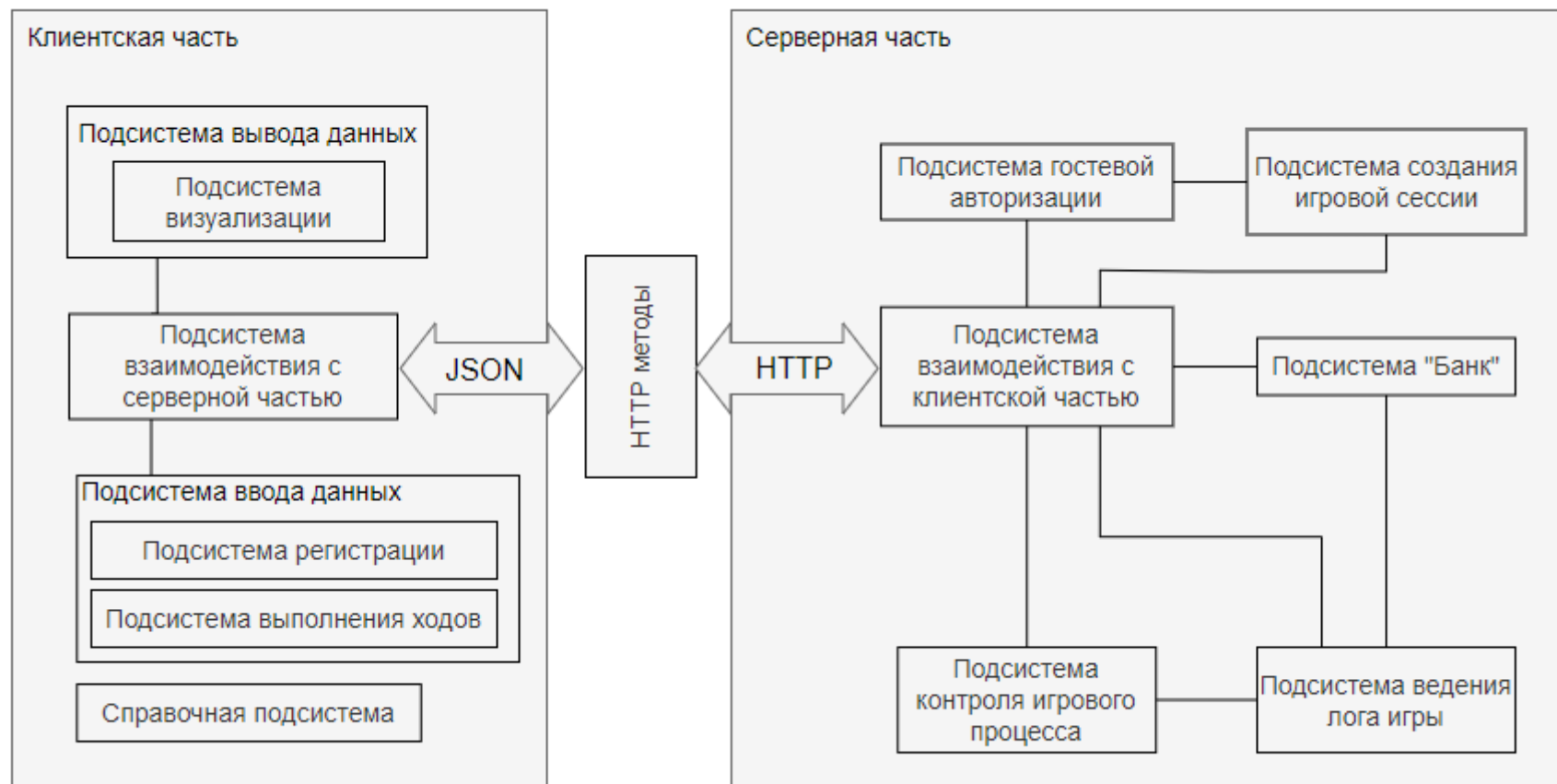


Рисунок 7 – Структурная схема системы

В подсистему вывода данных так же входит подсистема визуализации, которая использует HTML, CSS и JavaScript для создания интерфейса. Она включает интерактивные элементы, позволяющие пользователю взаимодействовать с данными, а также анимацию для улучшения восприятия информации;

- подсистема взаимодействия с серверной частью обеспечивает связь между клиентом и сервером, она отправляет запросы на сервер для получения или отправки данных, обрабатывает ответы и управляет состоянием приложения, обновляя пользовательский интерфейс в зависимости от текущего состояния;

- подсистема ввода данных занимается сбором информации от пользователей через формы ввода, она включает подсистему регистрации игроков и подсистему выполнения ходов, которая обрабатывает действия игроков в процессе игры. Обе подсистемы производят валидацию введенных данных на корректность и отправку их на сервер. Пользователи получают обратную связь о результате ввода, что может включать уведомления об успехе или ошибке;

- справочная подсистема предоставляет доступ к документации и поддержке пользователей. Она включает поиск по справочной информации, и информацию о разработчиках.

В состав серверной части входят следующие подсистемы:

- подсистема гостевой авторизации отвечает за генерацию временных учетных записей для взаимодействия пользователя с приложением;

- подсистема контроля игрового процесса отвечает за обработку игровых событий и изменение состояния игры в соответствии с ними. Кроме того, эта подсистема отвечает за синхронизацию состояний игры между всеми игроками, отправляя обновления о текущем состоянии всем пользователям;

- подсистема взаимодействия с клиентской частью обеспечивает связь между сервером и клиентом. Сервер предоставляет WebSocket API для обработки запросов от клиентской части. Это может включать запросы на получение данных, отправку действий игроков и так далее. Кроме того, сервер

обрабатывает входящие запросы и формирует соответствующие ответы, которые отправляются обратно на клиент;

- подсистема «Банк» отвечает за симуляцию работы игрового банка, который устанавливает цены на ЕСМ и ЕГП, определяет их количество в текущем месяце и выдает игрокам кредиты. Кроме того, игровой банк сам может выступать посредником в продажах, а значит подсистема должна обеспечивать симуляцию реалистичных торгов от лица банка, в которых он стремится к максимизации своей прибыли;

- подсистема ведения лога игры отвечает за сбор и хранение данных о ходе игры. Все значимые события в игре (действия игроков, изменения состояния, ошибки) записываются в лог-файлы или базу данных. Это позволяет отслеживать историю игрового процесса. Логи могут использоваться для анализа игрового процесса, выявления проблем или улучшения механик игры.

2.3 Разработка спецификации требований

В процессе разработки программного обеспечения ключевое значение имеет четко оформленная спецификация требований. Спецификация требований служит основой для проектирования, разработки и тестирования системы, а также играет важную роль в коммуникации между всеми заинтересованными сторонами. Правильная спецификация помогает избежать недопонимания и недочетов, гарантируя, что конечный продукт будет соответствовать ожиданиям пользователей [10].

Существует несколько типов требований, которые необходимо учитывать. Функциональные требования определяют поведение системы, описывая функции, которые должны быть внедрены. Они включают пользовательские функции, сценарии использования и обработку данных. Например, в контексте разрабатываемого приложения может быть описано, какие действия могут выполнять пользователи, какие данные они могут вводить и получать.

Нефункциональные требования касаются аспектов качества, производительности и надежности системы. К ним относятся такие характеристики, как скорость работы, безопасность, удобство использования и требования к поддерживаемым платформам.

Процесс сбора требований включает взаимодействие с заинтересованными сторонами, такими как пользователи, заказчики и разработчики, для выявления их ожиданий и потребностей. В этом процессе могут использоваться разнообразные методы, включая интервью, опросы, рабочие группы и мозговые штурмы. Анализ существующих систем также может выявить полезные идеи и улучшения.

Когда речь идет о спецификациях, можно выделить несколько их видов. Техническая спецификация представляет собой детальное описание технических аспектов системы, включая архитектуру приложения, используемые технологии, интерфейсы и методы интеграции с другими системами. Пользовательская спецификация фокусируется на потребностях конечного пользователя, описывая сценарии использования и пользовательские интерфейсы. Спецификация тестирования определяет, как будет проходить тестирование системы для проверки её соответствия требованиям, включая детальную информацию о тестовых сценариях и критериях приемки.

2.3.1 Функциональная спецификация

Функциональная спецификация описывает функциональные требования к системе, продукту или проекту. Она служит основой для разработки и определяет, какие функции и возможности должны быть реализованы, а также как эти функции будут взаимодействовать между собой. В таблице 4 приведен перечень функций, выполняемых системой на клиентской части системы.

Таблица 4 – Перечень функций, выполняемых системой (клиентская часть системы)

Название подсистемы	Название функции	Информационная среда			
		Входные данные		Выходные данные	
		Назначение (наименование)	Тип, ограничения	Назначение (наименование)	Тип, ограничения
1	2	3	4	5	6
Справочная	Выдать сведения о разработчиках	Сведения о разработчиках системы (ФИО, номер группы)	Текст	Визуальное отображение информации	–
	Выдать сведения о системе	Файл справки	Текстовый (*.HTML)		
	Выдать сведения о правилах игры	HTML-файл	Текстовый (*.HTML)		
				Код ошибки	Целое
Регистрации	Ввести никнейм	Набор допустимых символов	Латинские, русские буквы, цифры, «_»	Логин	Строка
		Допустимая длина	Целое 4..8	Код ошибки	Целое

Продолжение таблицы 4

1	2	3	4	5	6
Регистрации	Выбрать аватар	Список аватаров	Список строк (ссылки на графические объекты в системе)	Визуальное отображение информации	—
				Номер аватара	Целое
Выполнения хода	Продать ЕГП	Количество ЕГП	Целое	Информация о ЕГП	Файл JSON
		Стоимость ЕГП	Целое		
	Ввести количество ЕГП	Допустимый диапазон	Целое, от 1 до допустимого количества	Капитал	Целое
				Количество ЕГП	Целое
	Ввести стоимость ЕГП	Допустимый диапазон	Целое, больше 0	Код ошибки	Целое
				Стоимость	Целое
	Купить ЕСМ	Количество ЕСМ	Целое	Код ошибки	Целое
				Информация о ЕСМ	Файл JSON

Продолжение таблицы 4

1	2	3	4	5	6
Выполнения хода	Купить ЕСМ	Стоимость ЕСМ	Целое	Капитал	Целое
				Код ошибки	Целое
	Ввести стоимость ЕСМ	Допустимый диапазон	Целое, больше 0	Стоимость	Целое
				Код ошибки	Целое
	Ввести количество ЕСМ	Допустимый диапазон	Целое, больше 0	Количество	Целое
				Код ошибки	Целое
	Просмотреть лог игры	Файл с информацией об игре	Текстовый (*.HTML)	Визуальное отображение информации	—
				Код ошибки	Целое
	Построить завод	Количество заводов	Целое	Номер уровня завода	Целое
			Целое		
			Целое		
		Капитал	Целое		
			Код ошибки	Целое	

Продолжение таблицы 4

1	2	3	4	5	6
Выполнения хода	Произвести продукцию	Допустимый диапазон значения ЕСМ	От 0 до допустимого количества	Количество произведенной продукции	Целое
		Номер завода	Список		
		Взять ссуду	Список ссуд	Целое, 5000, 10000	Код ошибки
	Величина ссуды				Целое
	Капитал				Целое
	Код ошибки				Целое

2.3.2 Перечень исключительных ситуаций

Исключительная ситуация – это ситуация, при которой система не может выполнить возложенных на нее функций или которая может привести к денормализации работы системы.

В таблице 5 приведен перечень исключительных ситуаций для разрабатываемой системы и описаны реакции системы на их возникновение.

Таблица 5 – Перечень исключительных ситуаций

Название подсистемы	Название исключительной ситуации	Реакция системы
1 Справочная	1.1 Невозможно открыть файл справки	Выдача сообщения «Файл справки поврежден»
	1.2 Невозможно найти файл справки	Выдача сообщения «Отсутствует файл справки»
2 Вывода данных	2.1 Невозможно найти файл с логом игры	Выдача сообщения «Отсутствует файл с логом игры»
	2.2 Невозможно открыть файл с логом игры	Выдача сообщения «Файл лога игры поврежден»
3 Взаимодействие с серверной частью	3.1 Отсутствует соединение с сервером	Выдача сообщения «Отсутствует соединение с сервером. Повторите попытку позже»

2.4 Разработка прототипа интерфейса пользователя системы

Интерфейс — это соглашение, определяющее набор методов и свойств, которые должны реализовываться классами, унаследованными от этого интерфейса. Он служит своего рода контрактом, гарантируя, что любой класс,

реализующий интерфейс, предоставит реализацию всех его методов. Это обеспечивает согласованность и предсказуемость в использовании объектов различных классов [11].

Интерфейсы играют ключевую роль в создании модульного, гибкого и поддерживаемого кода, способствуя лучшему проектированию систем.

Рассмотрим прототипы всех основных форм приложения.

На рисунке 8 изображена приветственная страница приложения. Она является первой точкой контакта с пользователями и играет важную роль в создании первого впечатления о приложении. Страница должна быть привлекательной, чтобы заинтересовать посетителей и побудить их продолжить взаимодействие. На этой странице размещены логотип приложения, а также две ключевые кнопки: «Create Room», которая переводит пользователя на страницу создания комнаты, а также кнопка «Connect», позволяющая присоединиться к уже созданной игровой сессии.



Рисунок 8 – Прототип экранной формы приветственного окна приложения

На рисунке 9 приведен прототип регистрационного окна приложения. Здесь пользователям должно быть предложено несколько возможных вариантов аватаров. Важным элементом является легенда в формате письма, в котором игрока уведомляют о передаче в его владение бизнеса, что усиливает вовлеченность. В начале письма пользователю должно быть предложено

ввести свой никнейм. Для удобства предусмотрена кнопка в левом нижнем углу, возвращающая на предыдущую страницу, и кнопка в правом нижнем углу для перехода к следующему шагу.

После регистрации в игре система должна открывать окно ожидания, прототип которого изображенное на рисунке 10. Здесь будут отображены ключ для входа в комнату, количество игроков, присоединившихся к сессии, их аватары и никнеймы. Также должна быть предусмотрена кнопка для возврата на предыдущую страницу.

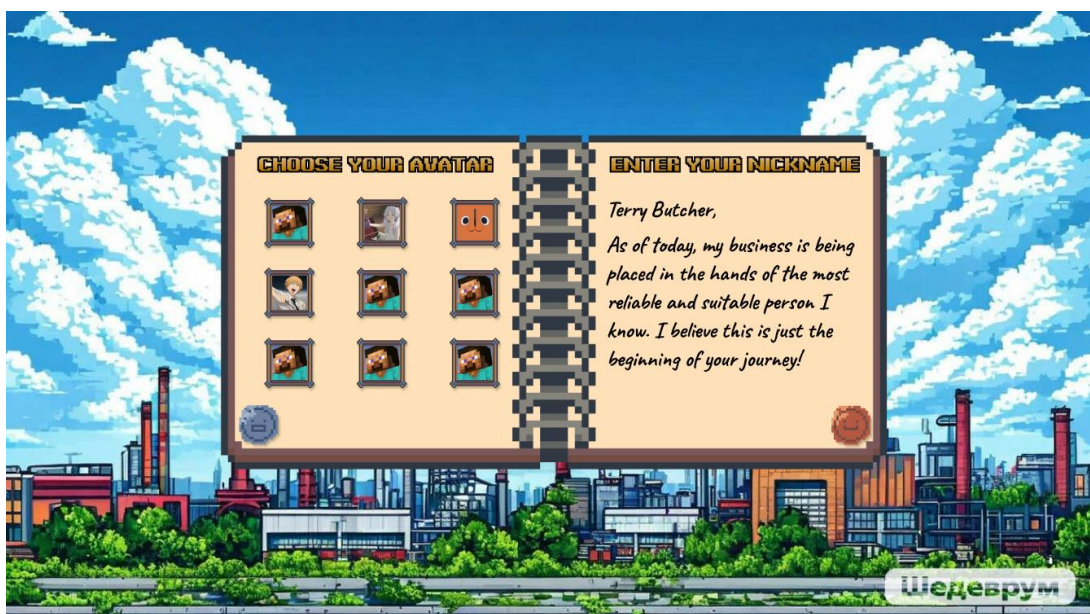


Рисунок 9 – Прототип экранной формы регистрационного окна приложения



Рисунок 10 – Прототип экранной окна ожидания приложения

Как только подключается минимум игроков – у старшего игрока должна появляться кнопка для старта игры, при нажатии на которую будет открываться страница с игровым окном, прототип которого изображен на рисунке 11. Если же старший игрок не нажимает на появившуюся кнопку в течение 60 секунд, то игра должна начинаться автоматически с тем составом игроков, который успел подключиться.

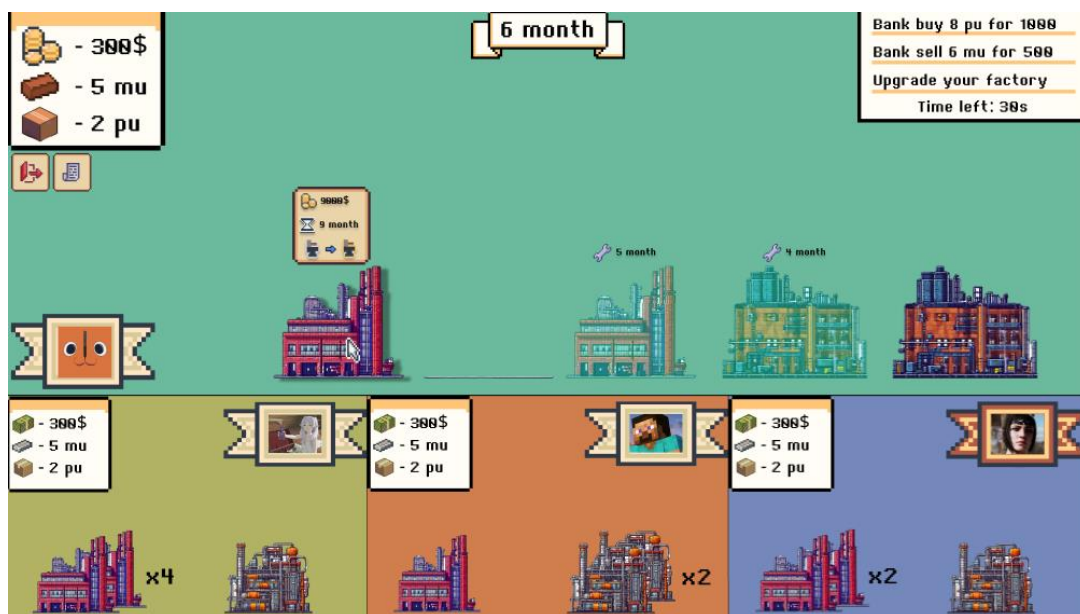


Рисунок 11 – Прототип игрового окна приложения

Игровое окно приложения будет представлять собой центральный элемент интерфейса и предоставлять игрокам всю необходимую информацию для взаимодействия с игровым процессом. Это окно должно быть интуитивно понятным и визуально привлекательным. Экран будет разделен на области, количество которых равно количеству игроков. Верхняя, самая большая область всегда должна принадлежать текущему игроку. На экране должна отображаться информация о каждом игроке: аватар, состояние, количество ЕГП и ЕСМ, а также число фабрик и их уровень технологичности. У старшего игрока текущего раунда рамка дополнительно будет выделяться оранжевым цветом. На экране также будет отображаться номер текущего круга.

В правом верхнем углу будет размещена информация о текущем ходе игры, включая количество и стоимость ЕГП и ЕСМ, текущий этап – преобразование фабрик, а также оставшееся время для совершения хода (30

секунд). При наведении курсора на свои фабрики пользователь должен получать детальную информацию о необходимых ресурсах для их преобразования, если фабрика доступна для реконструкции на данный момент.

Рассмотрим прототипы игровой формы в разные этапы игры подробнее.

На рисунке 12 изображен прототип игрового окна на этапе закупки сырья, где будет отображаться доступное игроку количество ЕСМ для закупки и её стоимость.

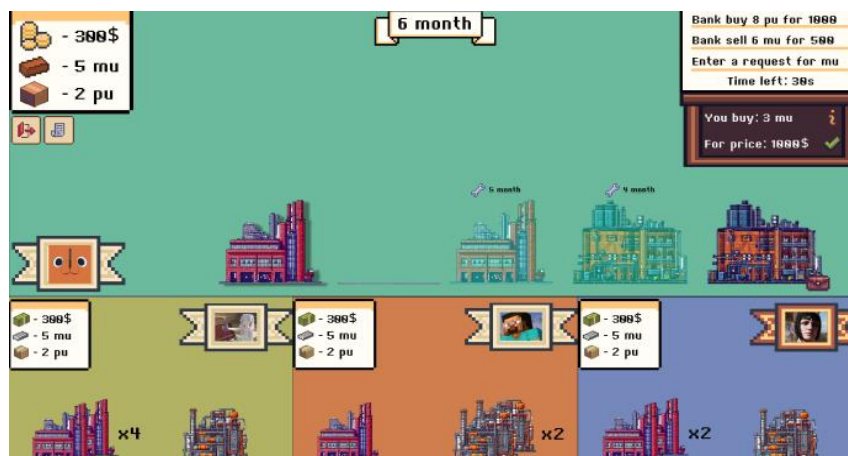


Рисунок 12 – Прототип игрового окна на этапе закупки сырья

На рисунке 13 изображен прототип игрового окна на этапе продажи продукции, где будет отображаться доступное количество ЕПП, которое доступно игроку для продажи и стоимость.

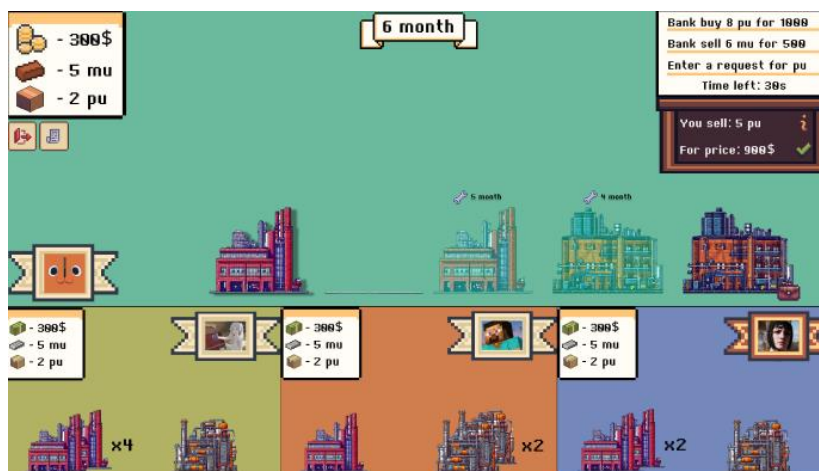


Рисунок 13 – Прототип игрового окна на этапе продажи продукции

На рисунке 14 изображен прототип игрового окна на этапе распределения сырья, где будет отображаться информация о том, сколько продукции, на каком заводе и за какую стоимость можно изготовить.

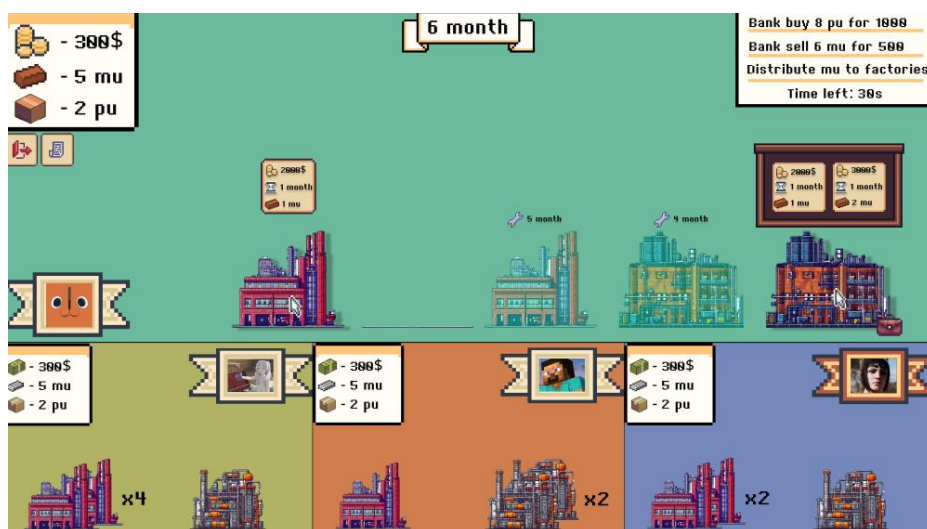


Рисунок 14 – Прототип игрового окна на этапе распределения сырья

На рисунке 15 изображен прототип игрового окна на этапе взятия ссуды, где будет отображаться о сумме ссуды, которая доступна игроку на данный момент, и под залог какого завода она может быть взята.

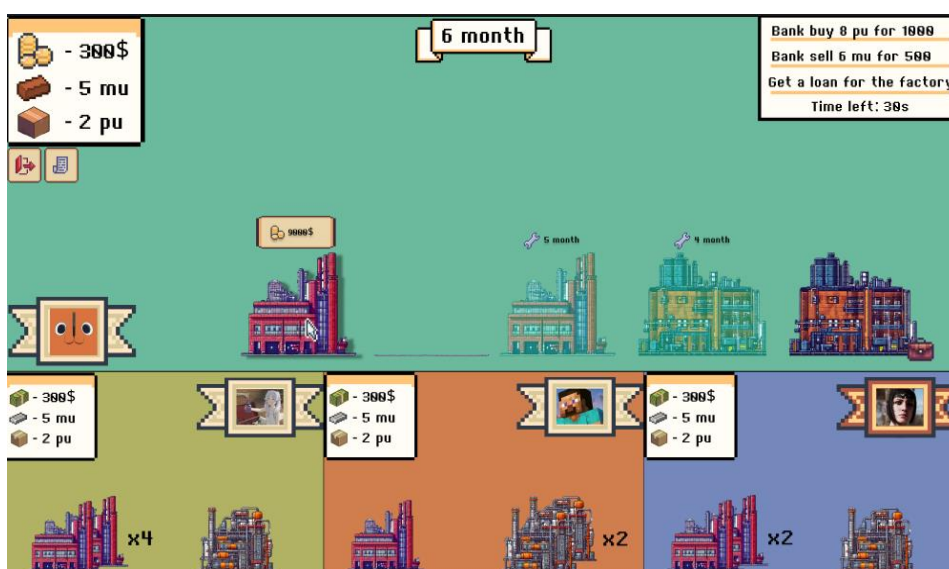


Рисунок 15 – Прототип игрового окна на этапе взятия ссуды

2.5 Разработка информационно-логического проекта системы

Разработка информационно-логического проекта системы является ключевым этапом, определяющим успешность будущей системы.

Качественное выполнение всех этапов – от анализа требований до документирования – обеспечивает эффективное функционирование системы и удовлетворение потребностей пользователей.

На начальном этапе важным шагом является анализ требований. Необходимо тщательно изучить потребности пользователей и заинтересованных сторон, а также определить функциональные и нефункциональные требования к системе.

Следующим этапом является моделирование логической структуры системы. На этом этапе создаются логические модели, представляющие собой абстракции элементов системы и их взаимосвязей. Это может включать в себя диаграммы классов, диаграммы взаимосвязей и другие модели, которые позволяют визуализировать архитектуру и структуру данных системы.

Важным аспектом в этом контексте является выбор архитектуры системы, которая должна соответствовать критериям масштабируемости, производительности и безопасности, обеспечивая надежность и эффективность в работе.

Также необходимо уделить внимание разработке интерфейсов, что включает в себя определение как пользовательских интерфейсов, так и интерфейсов взаимодействия с другими системами.

Удобство и интуитивность пользовательского опыта играют ключевую роль в успешной реализации проекта, поэтому важно продумать дизайн интерфейсов на данном этапе.

2.5.1 Язык UML

Для специфицирования (построения точных, недвусмысленных и полных моделей) системы и ее документирования используется унифицированный язык моделирования UML. Он был разработан в 1990-х годах как стандартный язык для графического описания объектов, их взаимоотношений и системы в целом, став общепринятым инструментом для разработки программного обеспечения и системного проектирования,

позволяя визуализировать, специфицировать, строить и документировать системы [12].

Основной целью UML является создание единого стандарта, который был бы понятен разработчикам и заинтересованным сторонам проекта. Это помогает устранить барьеры между различными участниками разработки и обеспечивает единообразие в представлении информации. UML предоставляет средства для создания различных моделей, демонстрирующих различные аспекты системы, такие как функциональные возможности, структура, поведение и взаимодействие компонентов.

Кроме того, UML поддерживает различные парадигмы разработки, что позволяет моделировать системы как в объектно-ориентированном, так и в функциональном или архитектурном стилях.

2.5.2 Диаграмма вариантов использования

Диаграмма вариантов использования представляет собой наиболее общую концептуальную модель сложной системы, которая является исходной для построения всех остальных диаграмм. На ней изображаются отношения между актерами и вариантами использования, что позволяет визуализировать взаимодействие пользователей с системой и понимает, какие функциональные возможности необходимы для удовлетворения потребностей пользователей [13].

Каждый вариант использования описывает конкретную функцию или действие, выполняемое системой в ответ на запрос актеров.

Актеры могут представлять как конечных пользователей системы, так и другие системы или внешние элементы, взаимодействующие с моделируемой системой.

При создании диаграммы вариантов использования важно определить все ключевые сценарии, которые будут реализованы в системе. Для достижения этого сначала необходимо провести идентификацию актеров, то есть определить всех участников, которые будут взаимодействовать с

системой. Это позволит лучше понять контекст функционирования системы и ее окружение.

На рисунке 16 приведена диаграмма вариантов использования, на которой представлено 3 актера: пользователь, игрок и банк, а также различные варианты использования, относящиеся к механике игры.

Пользователь может войти в игровую сессию двумя способами: присоединиться к игре по коду или создать новую игровую сессию. Чтобы войти в игровую сессию ему необходимо выбрать аватар и никнейм.

Пользователю и игроку доступен просмотр справки: сведения о разработчиках, об игре или о правилах.

Игрок является основным участником игры и взаимодействует со всеми функциями системы, принимая решения, которые влияют на ход игры. Он управляет процессом производства, принимая решения о том, что и когда производить, исходя из рыночного спроса и доступных ресурсов.

Игрок может инициировать строительство новых объектов, чтобы расширить свои возможности, а также принимать решения, связанные с финансами, такие как подача заявок на получение ссуд или погашение уже существующих обязательств, к тому же в его обязанности входит оплата постоянных издержек, таких как аренда и зарплаты, что необходимо для поддержания функционирования бизнеса. Он также может подать заявки на закупку ЕСМ, инициируя процесс приобретения необходимых ресурсов для производства, а также подать заявки на продажу ЕГП, в зависимости от своей стратегии.

Игрок вправе пропустить ход, или же посмотреть лог после очередного хода в игре.

В случае, когда управление становится неэффективным, он может принять решение о банкротстве. Это завершит его участие в текущей игре и изменит статус его бизнеса.

Банк выполняет функции, связанные с управлением ресурсами: формируя цены на ЕГП и ЕСМ, а также рассматривает заявки на покупку сырья и продажу продукции.

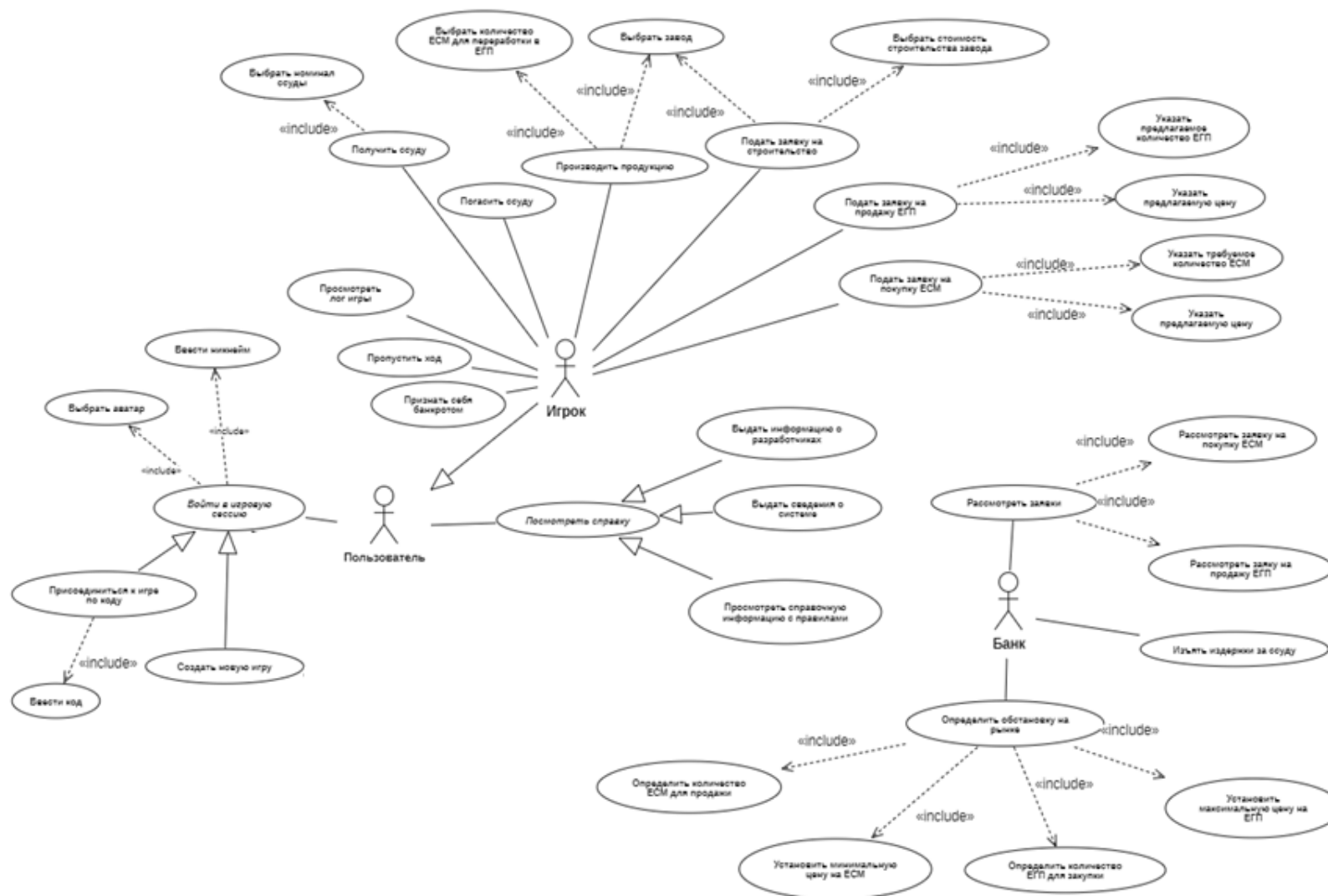


Рисунок 16 – Диаграмма вариантов использования системы

2.5.3 Сценарии

Сценарий (scenario) - определенная последовательность действий, которая описывает действия актеров и поведение моделируемой системы в форме обычного текста [14].

В контексте языка UML сценарий используется для дополнительной иллюстрации взаимодействия актеров и вариантов использования.

Рассмотрим несколько сценариев.

Вариант использования «Выбрать количество ЕСМ для переработки в ЕПП»

Краткое описание. Даёт возможность игроку выбрать необходимое количество ЕСМ, которое будет переработано на заводе в ЕПП. Включается в вариант использования «Производить продукцию».

Актант. Игрок.

Предусловия. Компьютер пользователя включён. Пользователь запустил веб-приложение «Менеджмент». Игровая сессия создана и успешно запущена. Предыдущие этапы игры были пройдены. Игрок должен сделать очередной ход.

Основной поток событий.

1 На игровой форме в верхнем правом углу появляется описание текущего этапа: «Распределение сырья» и оставшееся время до его завершения. На форме отображаются заводы, которыми владеет игрок. Если завод обычный, то отображается форма только с одной такой кнопкой. Кроме того на форме доступны кнопки выхода «» и получения справки «».

2 Игрок щелкает левой кнопкой мыши по улучшенному заводу.

A1: Игрок щелкает по кнопке «».

A2: Игрок щелкает по кнопке «».

A3: Игрок не совершает никаких действий.

A4: Игрок щелкает левой кнопкой мыши по обычному заводу.

3 Система отображает форму с двумя кнопками, на каждой из которых отображается информация, необходимая для выбора: длительность переработки, её стоимость и количество ЕСМ.

- 4 Игрок щелкает на кнопку с одной ЕСМ.
А3: Игрок не совершает никаких действий.
А5: Игрок нажимает на кнопку с двумя ЕСМ.
- 5 Система проверяет количество у игрока ресурсов для выбранных им переработок на корректность.
- 6 Система закрывает форму для распределения сырья, переходя на форму следующего этапа. Вариант использования завершается успешно.
А6: Ресурсов для переработки недостаточное количество.
Альтернативы.
- А1: Игрок щелкает по кнопке «».
А1.1 Переход к варианту использования вариант использования «Выйти из игровой сессии».
А1.2 Вариант использования завершается.
- А2: Игрок щелкает по кнопке «».
А2.1: Переход к варианту использования вариант использования «Посмотреть лог игры».
- А3: Игрок не совершает никаких действий:
А3.1: Система реагирует на действие игрока как на пропуск хода и выполняется вариант использования «Пропуск хода».
- А4: Игрок щелкает левой кнопкой мыши по обычному заводу.
А4.1: Система отображает форму с одной кнопкой, на которой отображается информация о необходимых ресурсах для переработки на этом заводе: длительность переработки, её стоимость и количество ЕСМ.
А4.2: Переход к п.4 из основного потока.
- А5: Игрок нажимает на кнопку с двумя ЕСМ.
А5.1: Переход к п.5 из основного потока.
- А6: Ресурсов для переработки недостаточное количество.
А6.1: Система выводит всплывающую подсказку, у заводов, где продукцию произвести не вышло: «Ресурсов для переработки недостаточно».

A6.2A1: Время на ход истекло. Вариант использования завершается.

A6.2A2: Переход к п.2 основного потока.

Постусловия. При успешном завершении форма этапа переработки продукции закрывается автоматически и открывается форма следующего этапа хода.

Вариант использования «Ввести код»

Краткое описание. Позволяет игроку ввести код комнаты и попытаться подключиться к ней. Включается в вариант использования «Присоединиться к игре по коду».

Актант. Пользователь.

Предусловия. Компьютер пользователя включён. Пользователь запустил веб-приложение «Менеджмент» и нажал на кнопку «Connect».

A4.2A1: Пользователь щелкает левой кнопкой мыши за пределами окна Основной поток событий:

1 Система выводит на экран форму пустой игровой комнаты. На форме отображаются кнопки «» (далее кнопка «Назад»), «» (далее кнопка «Вперёд») и «» (далее поле ввода кода).

2 Пользователь вводит код комнаты в поле ввода кода.

A1: Пользователь щелкает по кнопке «Назад».

A2: Пользователь щелкает по кнопке «Вперёд».

3 Пользователь щелкает по кнопке «Вперёд».

A3: Система не находит соответствующую коду комнату.

4 Система находит соответствующую коду комнату.

A4: В комнате недостаточно места для ещё одного игрока.

5 В комнате достаточно места для ещё одного игрока. Пользователь подключается к комнате. Вариант использования завершается успешно.

Альтернативы.

A1: Пользователь щелкает по кнопке «Назад».

A1.1 Переход к варианту использования «Войти в игровую сессию».

A2: Пользователь щелкает по кнопке «Вперёд».

A2.1: На форме под полем ввода выводится всплывающая подсказка с сообщением о том, что код не был введён.

A2.2A1: Пользователь щелкает левой кнопкой мыши за пределами окна. Система закрывает окно с сообщением.

A2.2A2: Переход к п.2 основного потока.

A3: Система не находит соответствующую коду комнату.

A3.1: На форме под полем ввода выводится всплывающая подсказка с сообщением о том, что комната не была найдена.

A3.2A1: Пользователь щелкает левой кнопкой мыши за пределами окна. Система закрывает окно с сообщением.

A3.2A2: Переход к п.2 основного потока.

A4: В комнате недостаточно места для ещё одного игрока.

A4.1: На форме выводится сообщение о том, что комната заполнена.

A4.2A1. Система закрывает окно с сообщением.

A4.2A2: Переход к п.2 основного потока.

Постусловия. На экранной форме отображается список игроков, находящихся в этой комнате (их аватары и никнеймы).

Вариант использования «Указать предлагаемую цену»

Краткое описание: Позволяет игроку указать цену, за которую он готов продать каждую ЕГП. Включается в вариант использования «Подать заявку на продажу ЕГП».

Актант: Игрок.

Предусловия. Компьютер пользователя включён. Пользователь запустил веб-приложение «Менеджмент». Игровая сессия создана и успешно запущена. Предыдущие этапы игры были пройдены. Игрок должен сделать очередной ход.

Основной поток событий.

1 Система выводит на экран форму ввода количества ЕГП и предлагаемой цены. На форме отображаются поля «You sell», «For price» с предустановленным по умолчанию значением, равным 0, справа от полей для ввода имеются кнопки для подтверждения заявки и выдачи справочной информации о данной операции.

2 Игрок щелкает левой кнопкой мыши поле «For price» и вводит предлагаемую цену за ЕГП.

A1: Игрок щелкает по кнопке .

A2: Игрок щелкает по кнопке .

A3: Игрок щелкает по кнопке .

A4: Игрок не совершает никаких действий.

A5: Игрок вводит предлагаемое количество ЕГП.

3 Игрок нажимает кнопку подтверждения заявки «».

4 Система проверят введенное значение на корректность по допустимому диапазону значений.

A6: Введенное значение не корректно.

5 Система ожидает подтверждения заявок от других игроков или окончания времени на текущий этап игры. После завершения этапа форма для продажи ЕГП скрывается. Вариант использования завершается успешно.

Альтернативы.

A1: Игрок щелкает по кнопке .

A1.1 Переход к варианту использования «Выдать справочную информацию с правилами».

A2: Игрок щелкает по кнопке .

A2.1 Переход к варианту использования «Выйти из игровой сессии».

A3: Игрок щелкает по кнопке .

A3.1 Переход к варианту использования «Просмотреть лог игры».

A4: Игрок не совершает никаких действий.

А4.1 Система регистрирует действие игрока как пропуск хода.

А4.2 Вариант использования завершается.

А5: Игрок вводит предлагаемое количество ЕГП.

А5.1 Переход к варианту использования «Указать предлагаемое количество ЕГП».

А6: Введенное значение не корректно.

А6.1 Открывается всплывающая подсказка под полем для ввода цены с сообщением об ошибке: «Введено не корректное значение цены».

А6.2А1: Пользователь щелкает левой кнопкой мыши за пределами окна. Система закрывает окно с сообщением.

А6.2А2: Переход к п.2 основного потока.

Постусловия. При успешном завершении форма для продажи ЕГП скрывается и запускается следующий этап хода.

2.5.4 Диаграмма классов

Диаграммы классов – это наиболее часто используемый тип диаграмм, которые создаются при моделировании объектно-ориентированных систем, они показывают набор классов, интерфейсов и коопераций, а также их связи.

На практике диаграммы классов применяют для моделирования статического представления системы, они служат основой для целой группы взаимосвязанных диаграмм – диаграмм компонентов и диаграмм размещения.

Кроме того, диаграммы классов служат инструментом для документирования системы, что делает их важным аспектом технической документации.

На рисунке 17 приведена диаграмма классов системы (этап проектирования).

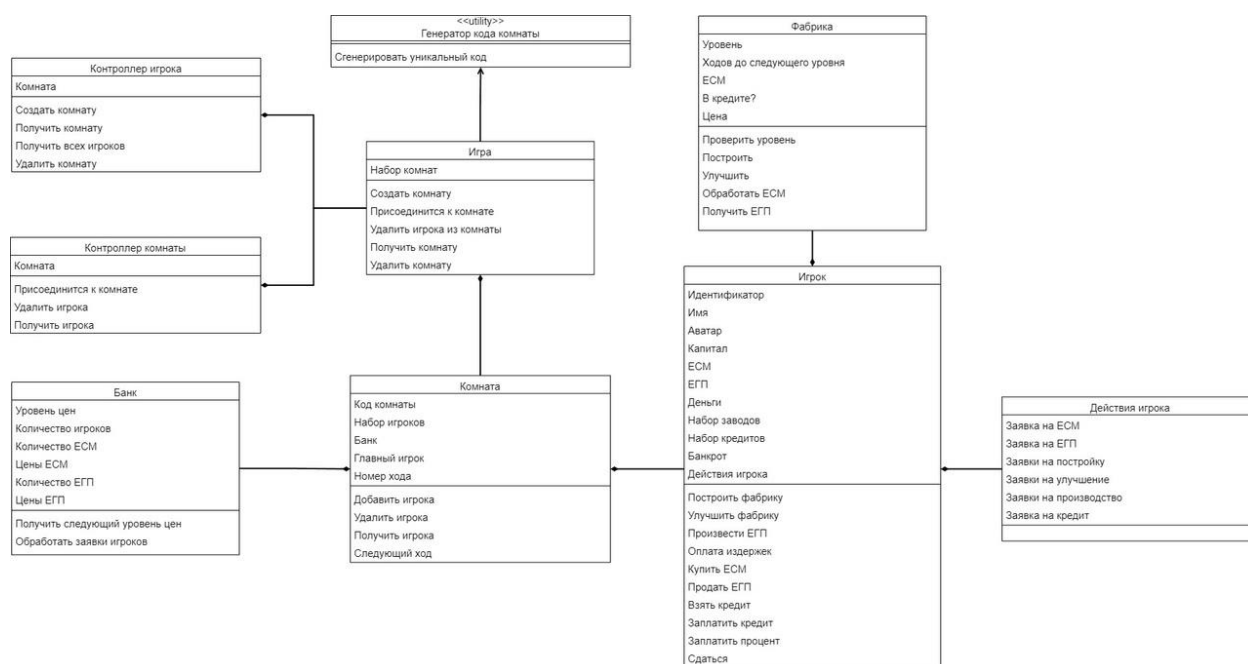


Рисунок 17 – Диаграмма классов системы

В таблице 6 приведено описание классов.

Таблица 6 – Описание классов системы

Название класса	Назначение
Игра	Управление игрой и хранение комнат.
Комната	Управление логикой комнаты, обработка ходов, хранение данных об игроках и банке.
Банк	Управление банковской логикой, обработка заявок игроков на покупку/продажу ЕСМ/ЕГП.
Игрок	Управление действиями игрока, хранение информации об игроке, логика обработки заявок пользователя.
Фабрика	Управление логикой фабрик, их улучшения, постройки и переработки ЕСМ.
Действия игрока	Содержит данные о действиях и заявках игрока.
Контроллер игрока	Обрабатывает запросы к игроку с клиента.
Контроллер комнаты	Обрабатывает запросы к комнате с клиента.

2.5.5 Диаграмма состояний

Диаграммы состояний предназначены для описания поведения элемента модели в течение его жизненного цикла. Они применяются для объяснения работы сложных объектов, полезны при моделировании их жизненного цикла и служат для анализа динамических аспектов системы. Эти диаграммы помогают визуализировать, как объекты переходят из одного состояния в другое, а также какие события вызывают эти переходы, что делает их важным инструментом в разработке и анализе систем.

На рисунке 20 приведена диаграмма состояний системы. Эта диаграмма состояний описывает этапы жизненного цикла клиента в сетевой многопользовательской игре. Стартовое состояние процесса – инициализация, а финальное состояние – конец игры.

Первый этап начинается с запуска приложения, когда клиент иницирует игру. На этом этапе устанавливается соединение с сервером, открывается приветственная форма для выбора дальнейших действий. Клиент может выбрать просмотреть справку по игре, зарегистрироваться или присоединиться к игровой сессии.

На следующем этапе происходит присоединение к игровой сессии. Клиент может выбрать существующую сессию, и если игроков недостаточно, он будет ждать, пока не наберется необходимое количество участников, после чего откроется игровая форма. Если подходящих сессий нет, клиент может создать новую, и процесс будет аналогичным: он также будет ждать, пока игроков не станет достаточно. Кроме того, клиент может присоединиться к сессии по коду, введя специальный код, чтобы подключиться по приглашению, и в случае нехватки участников сохранится ожидающее состояние до получения достаточного количества игроков.

На завершающем этапе клиент участвует в игровом процессе. Диаграмма состояний данного этапа изображена на рисунке 21.

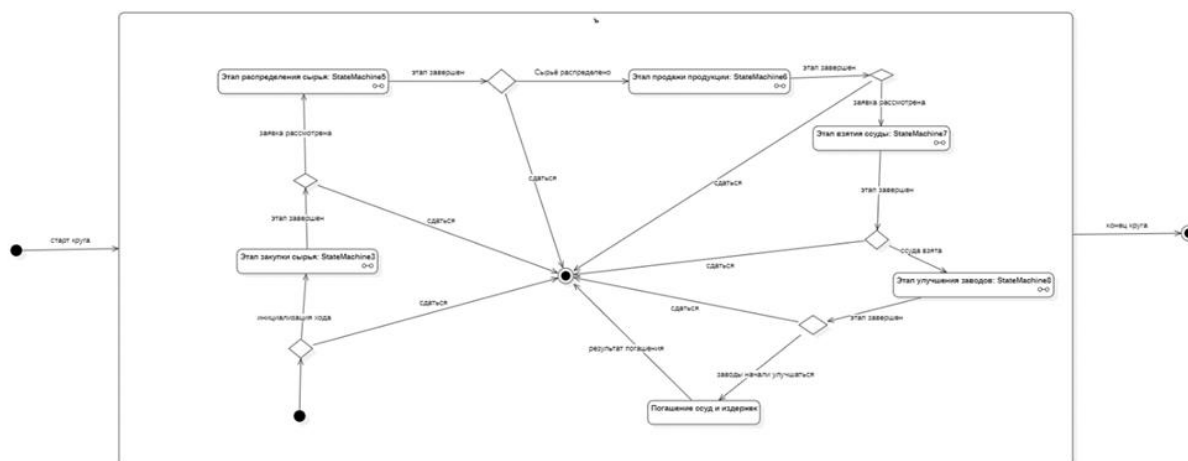


Рисунок 21 – Диаграмма состояний системы

Данная диаграмма состояний описывает процесс игры, описывающий целый игровой круг в которой игрок должен преодолеть несколько этапов. Стартовое состояние игры — «Старт круга», откуда начинается весь процесс.

Первый этап процесса называется «Закупка сырья». Подробная диаграмма состояний этого процесса изображена на рисунке 22.

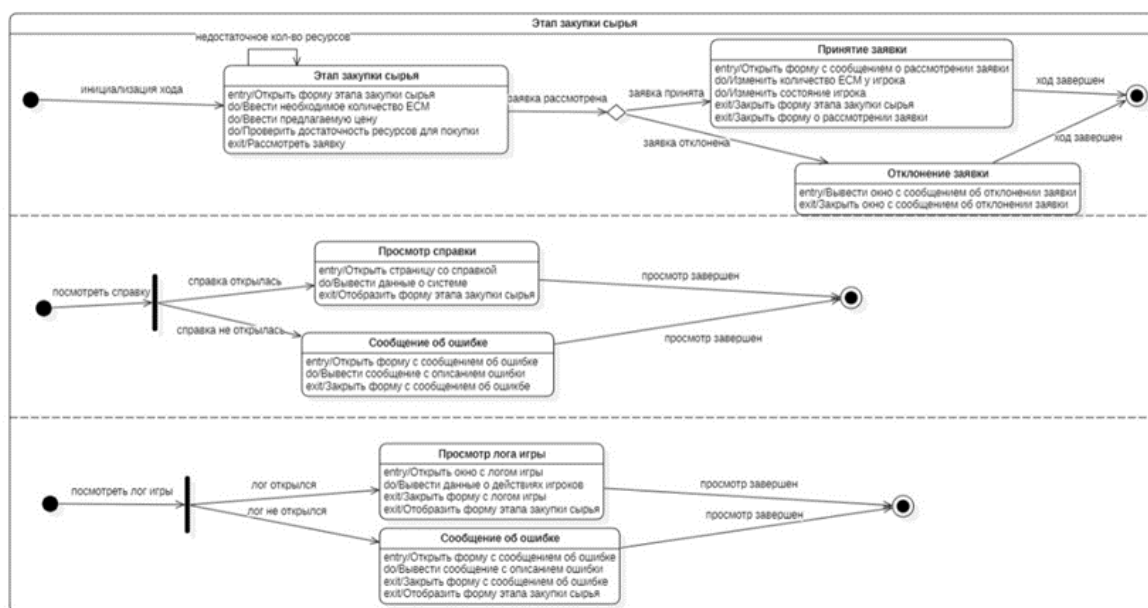


Рисунок 22 – Диаграмма состояний системы

На этом этапе игрок вводит необходимое количество ресурсов, которые ему нужно закупить, а также предлагает цену. Игра проверяет, достаточно ли у него ресурсов для выполнения закупки, и, если все в порядке, переходит ко второму этапу — «Распределению сырья».

На втором этапе, «Распределение сырья», игрок выбирает заводы, куда он хочет отправить сырье, и решает, сколько сырья будет перерабатываться на каждом заводе. Диаграмма состояний системы этого этапа изображена на рисунке 23.

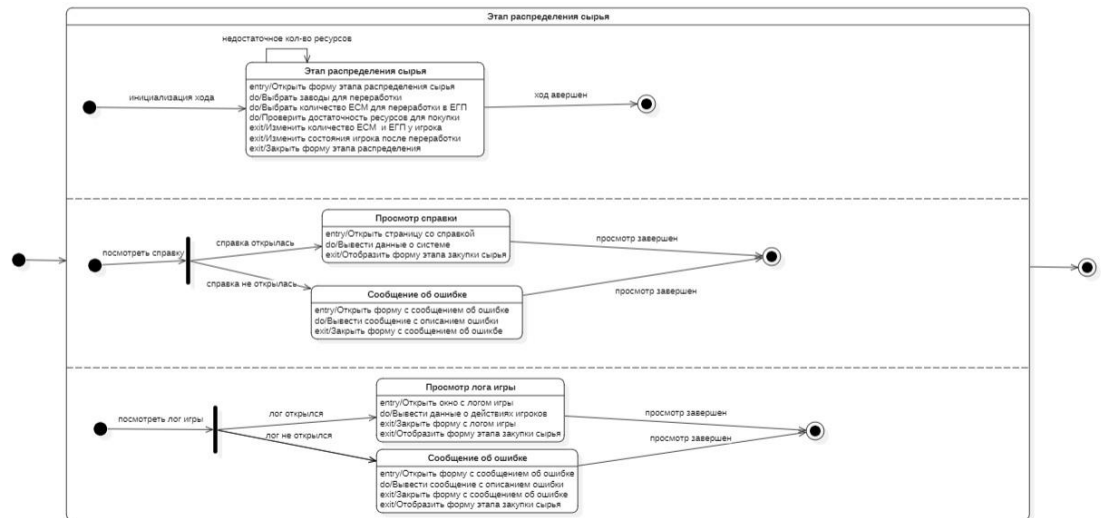


Рисунок 23 – Диаграмма состояний системы

Третий этап — «Продажа продукции», диаграмма состояний системы которого изображен на рисунке 24.

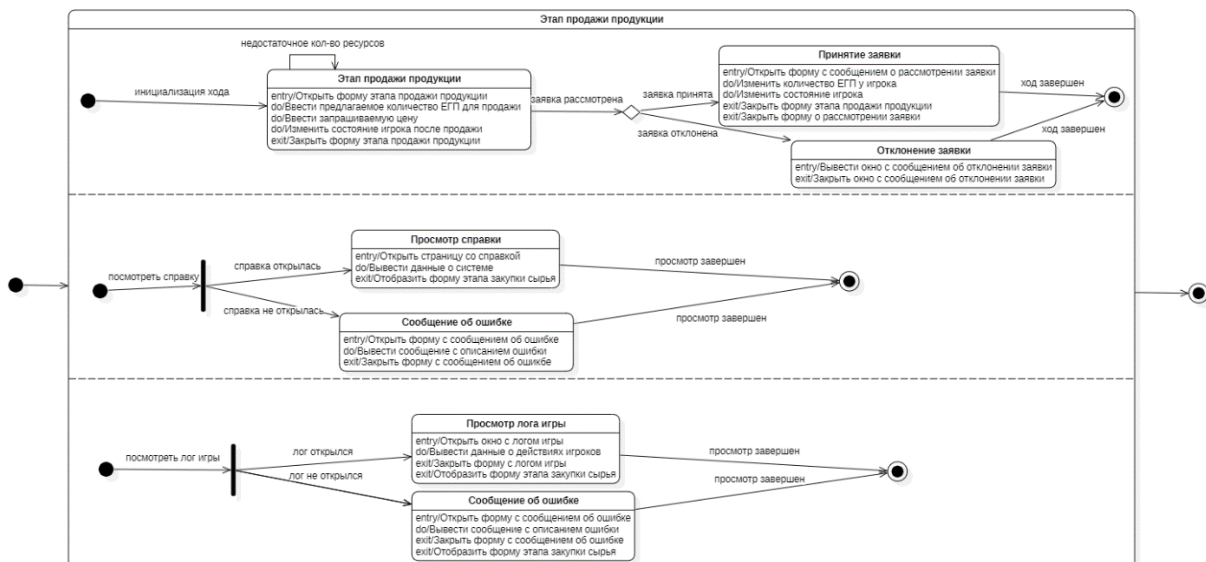


Рисунок 24 – Диаграмма состояний системы

Здесь игрок определяет количество продукции, которое он намерен продать, и предлагает цену. Игра проверяет наличие достаточного количества продукции для продажи, и, если это так, переходит к следующему этапу — «Взятию ссуды».

Четвертый этап называется «Взятие ссуды». Игрок может воспользоваться ссудой, если ему не хватает ресурсов для дальнейших действий, таких как закупка сырья или улучшение заводов. Диаграмма этого этапа изображен на рисунке 25.

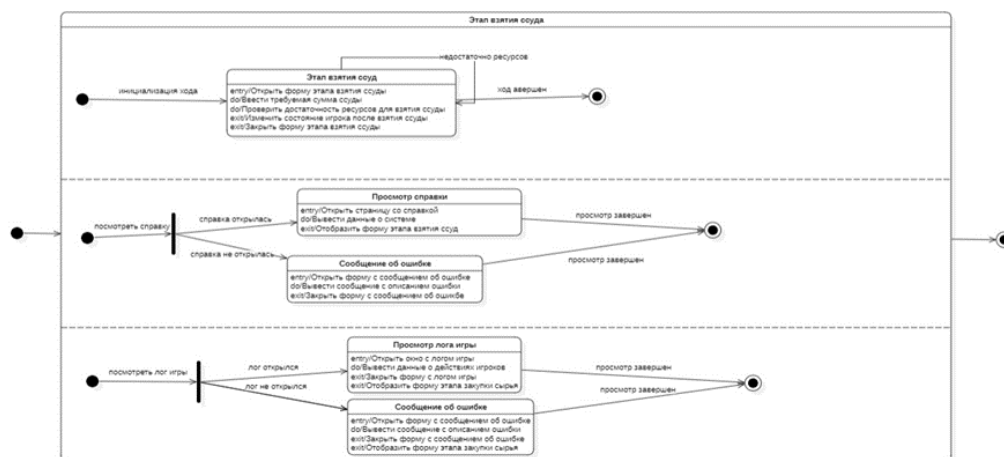


Рисунок 25 – Диаграмма состояний системы

На пятом этапе, «Улучшение заводов», игрок может использовать имеющиеся у него ресурсы для улучшения. Данная диаграмма состояний изображена на рисунке 26.

На последнем этапе игрок погашает ссуды, у которых срок погашения в этом месяце, а также постоянные издержки.

Игра завершается в конечном состоянии, обозначенном как "Конец круга", после которого процесс может быть перезапущен.

Дополнительно в диаграмме предусмотрены состояния, в которых игрок может «Сдаться» на любом этапе игры.

Также имеется возможность «Посмотреть справку», чтобы ознакомиться с правилами, или «Посмотреть лог игры», чтобы узнать историю своих действий.

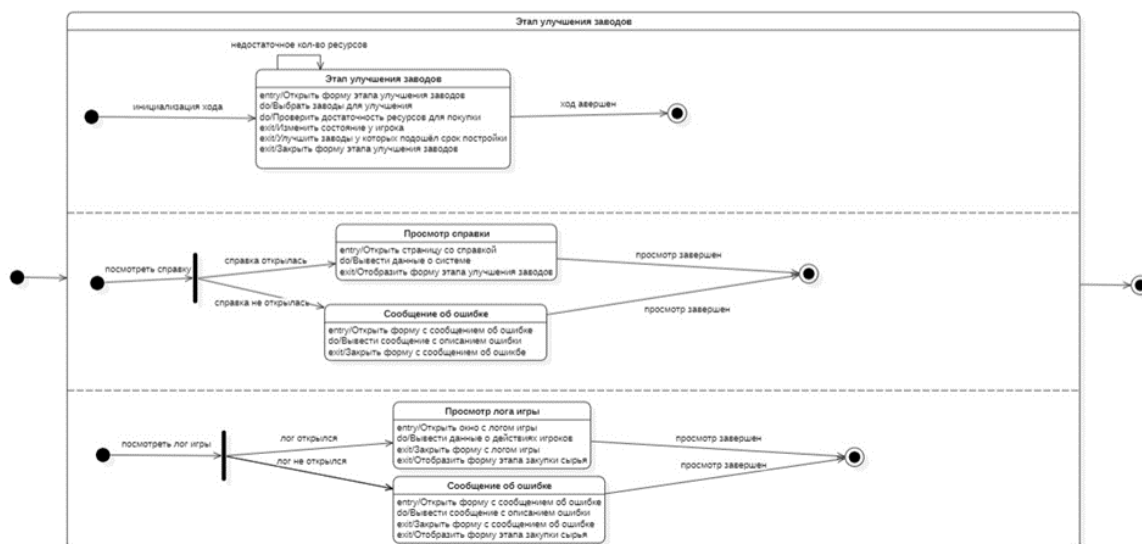


Рисунок 26 – Диаграмма состояний системы

2.5.6 Диаграмма деятельности

Диаграмма деятельности представляет собой один из основных инструментов моделирования бизнес-процессов, используемых в UML. Она позволяет визуализировать и описать поток работ и действий, выполняемых при реализации бизнес-процесса.

2.5.7 Диаграмма последовательности

Диаграммы последовательности принадлежат к семейству диаграмм UML и в большинстве случаев используются для представления взаимодействий между различными объектами в системе в определенном времени. На диаграмме последовательности представлены объекты и актеры, которые взаимодействуют друг с другом.

Временные линии, или линии жизненного цикла, показывают активность объекта в течение времени. Эти линии представляют собой вертикальные пунктирные линии, исходящие из объектов. Когда объект активен, линия жизни отображает прямую линию, а когда объект неактивен – пунктирную линию.

Взаимодействия между объектами отображаются в виде стрелок, которые указывают, какой объект отправляет сообщение и какому объекту оно

предназначено. Сообщения могут быть как синхронными (вызывающими ответ), так и асинхронными (без ожидания ответа).

На рисунке 27 приведена диаграммы последовательности системы для варианта использования «Ввод кода комнаты».

Диаграммы построены на основании сценариев, приведенных в п.2.4.3.

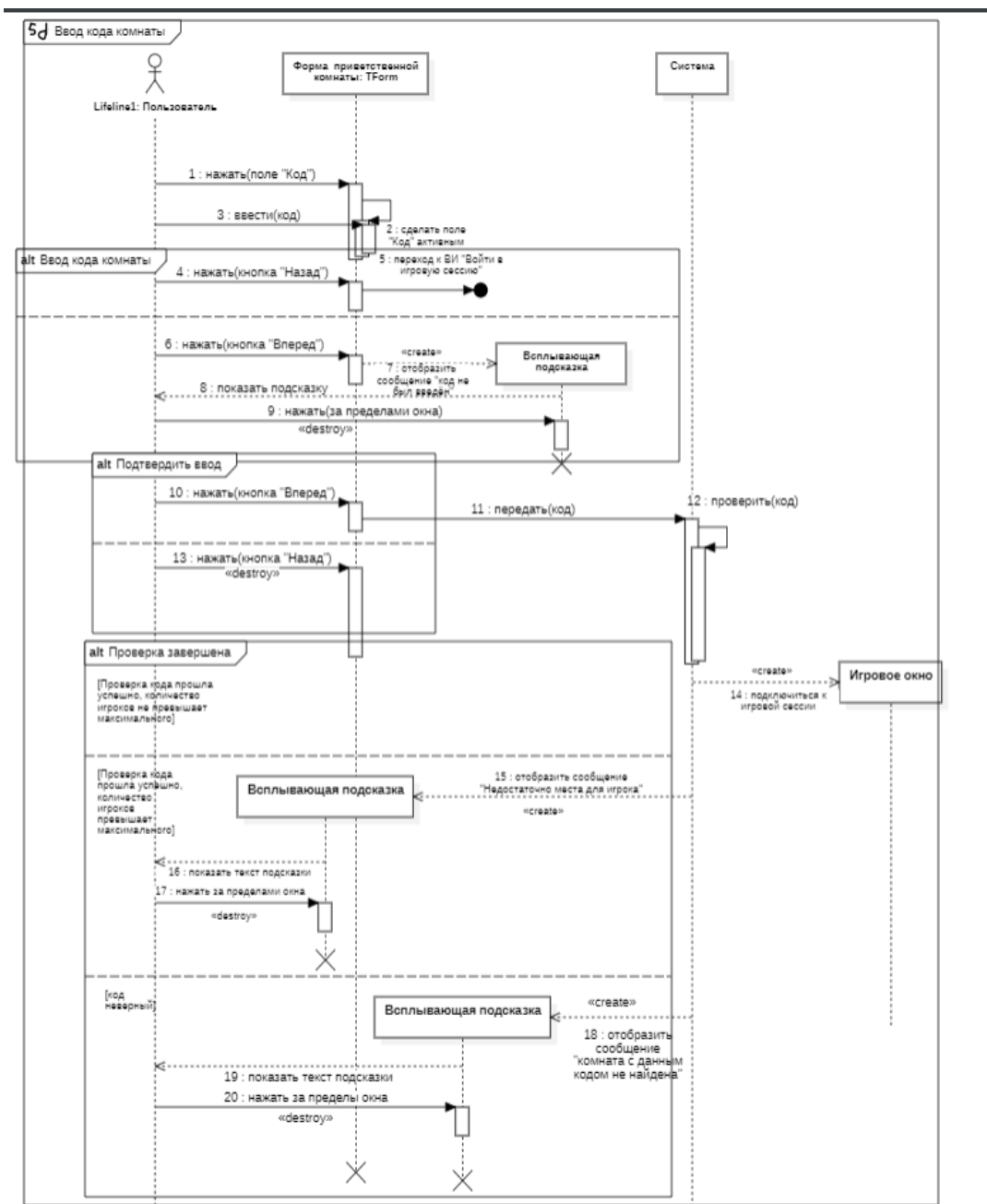


Рисунок 27 – Диаграмма последовательности для
ВИ «Ввод кода комнаты»

На рисунке 28 приведена диаграммы последовательности системы для варианта использования «Указать предлагаемую цену».

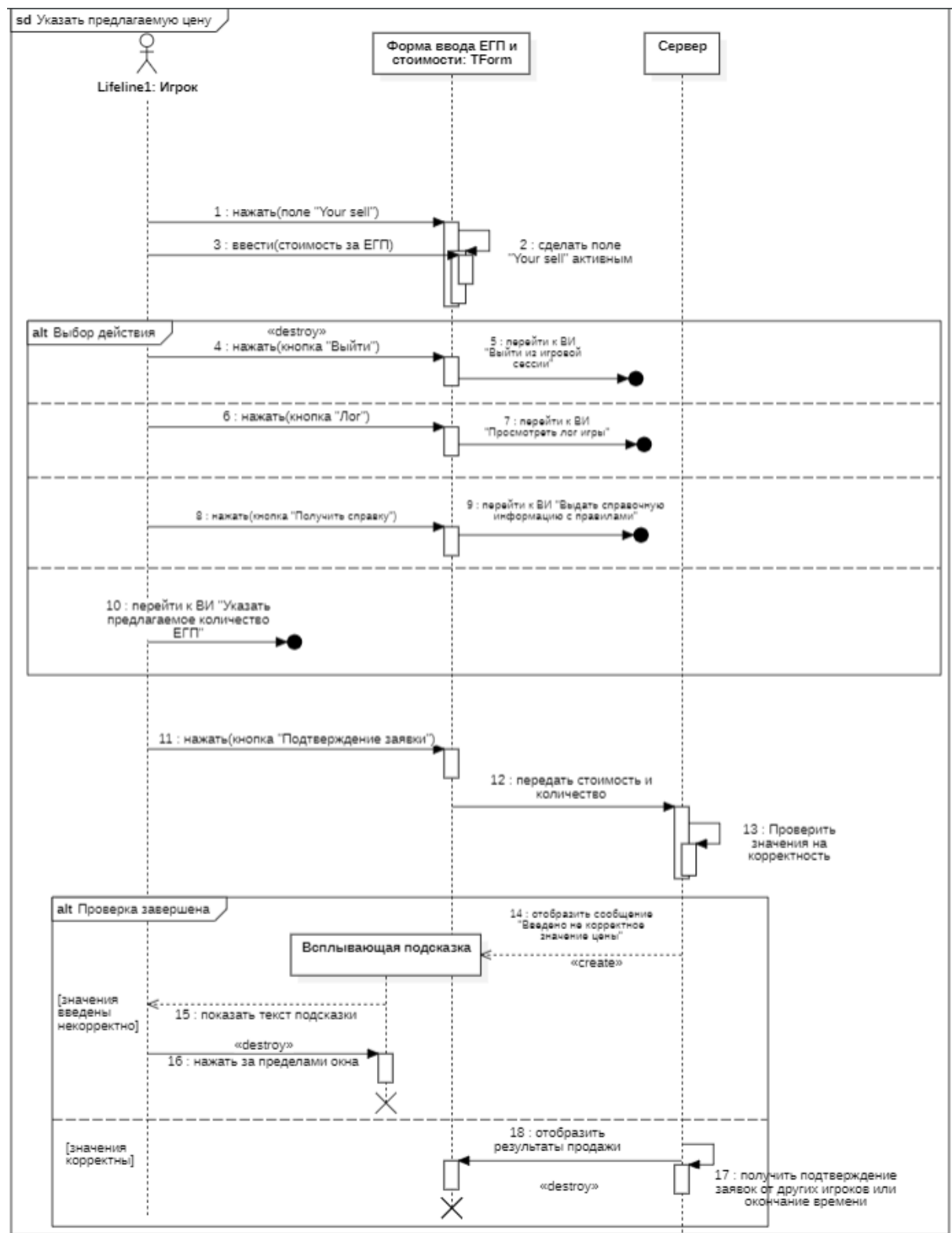


Рисунок 28 – Диаграмма последовательности для
ВИ «Указать предлагаемую цену»

На рисунке 29 приведены диаграммы последовательности системы для варианта использования «Выбрать количество ЕСМ для переработки в ЕГП».

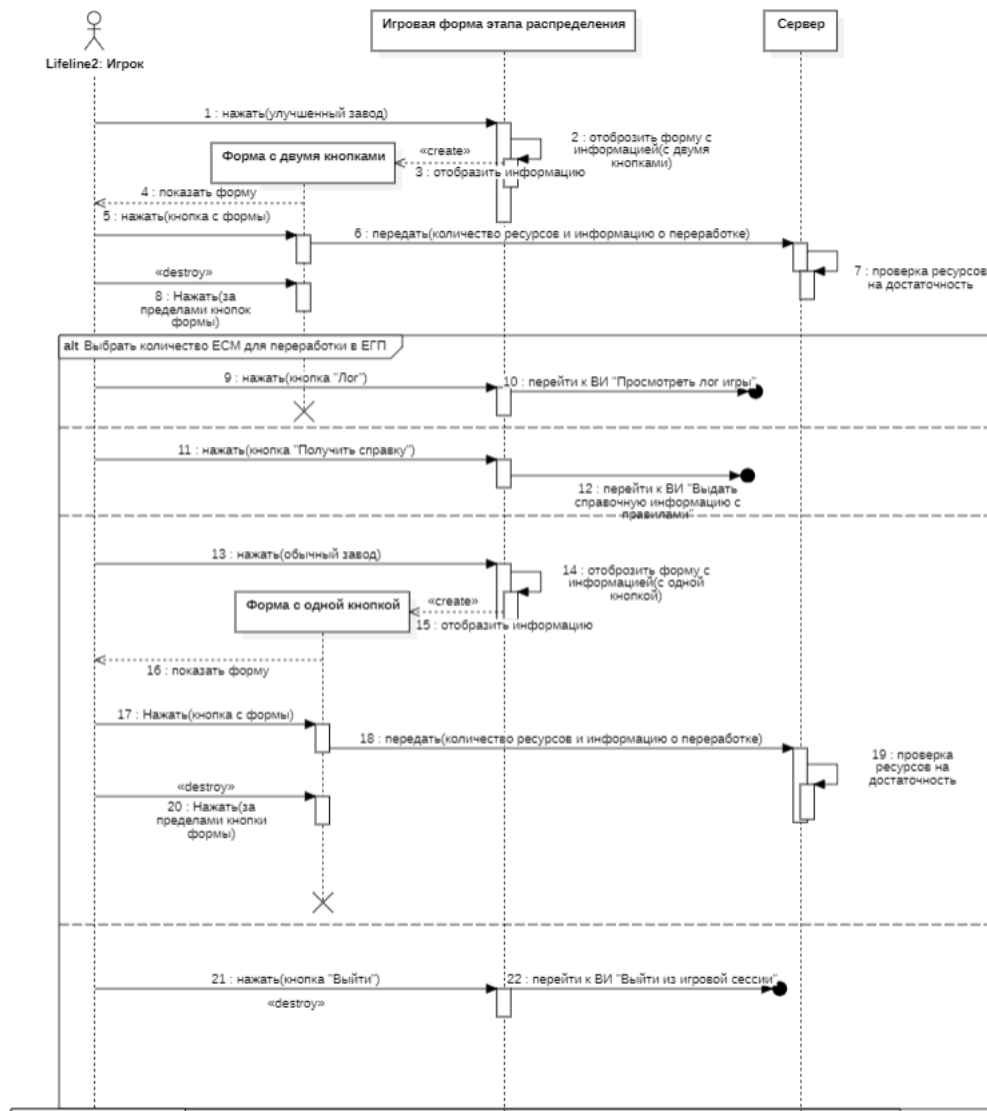


Рисунок 29 – Диаграмма последовательности для ВИ «Выбрать количество ЕСМ для переработки в ЕГП» (начало)

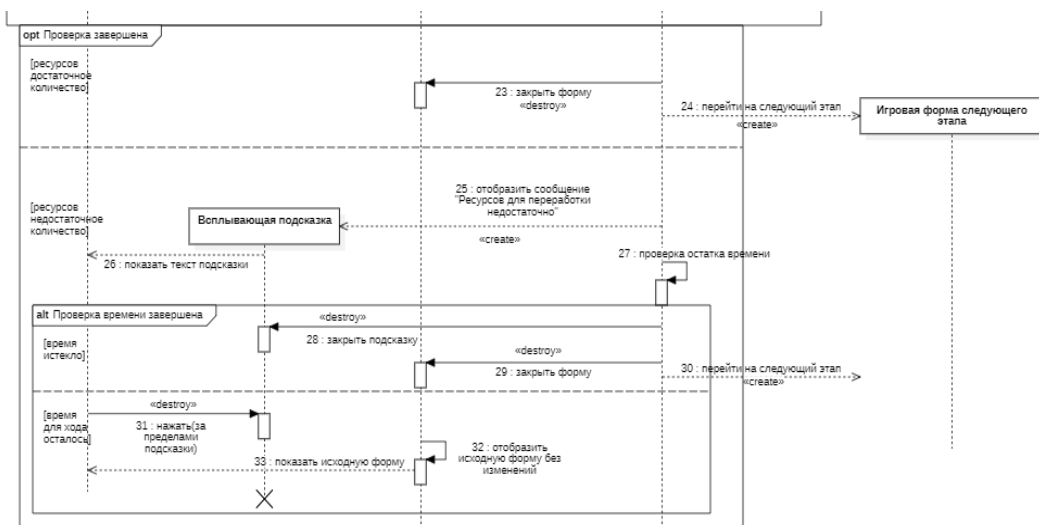


Рисунок 29 – Диаграмма последовательности для ВИ «Выбрать количество ЕСМ для переработки в ЕГП» (окончание)

2.6 Выбор и обоснование алгоритмов обработки данных

Алгоритм обработки данных — это метод, позволяющий решить задачу, который можно реализовать с использованием выбранного языка программирования. Процесс разработки алгоритма для конкретной задачи включает в себя определение ограничений на входные данные и разбивку задачи на более простые подзадачи. На рисунке 30 приведена схема алгоритма определения нового уровня цен в банке.

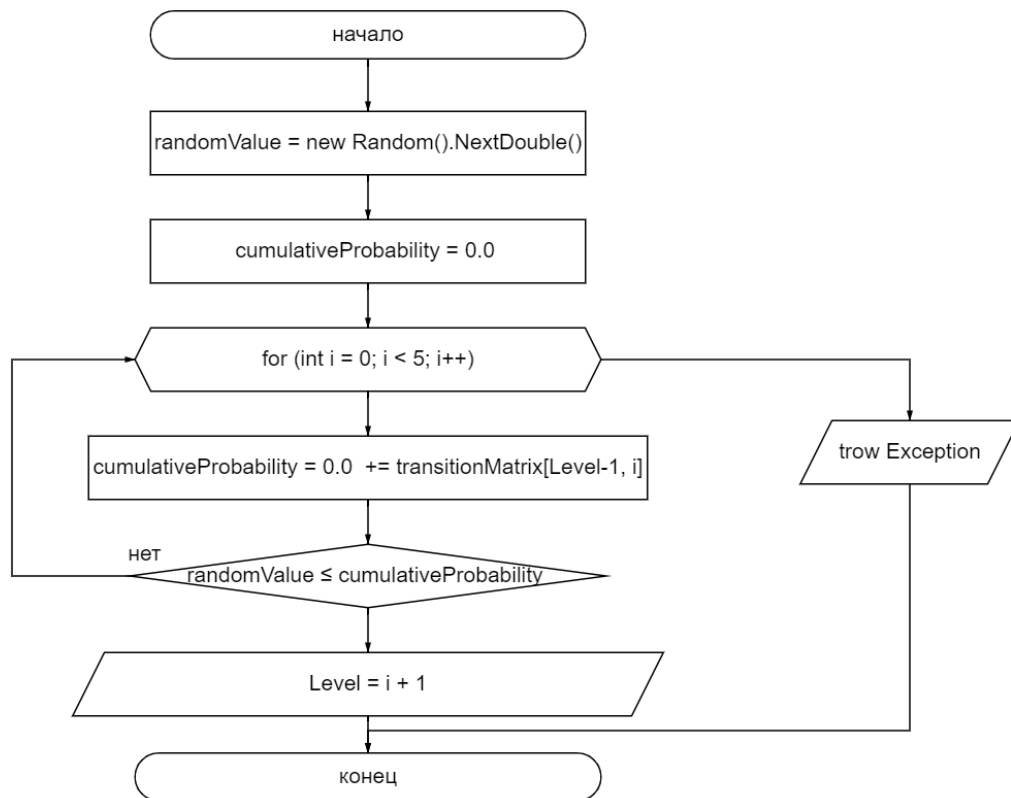


Рисунок 30 – Схема алгоритма установки нового уровня цен в банке

Алгоритм начинается с инициализации необходимых переменных. Сначала создается случайное число в диапазоне от 0 до 1 с помощью функции `random.NextDouble()`, которое будет использоваться для определения уровня. Затем инициализируется переменная `cumulativeProbability`, устанавливаемая в 0.0, которая будет накапливать вероятности из матрицы переходов.

Далее проверяется, меньше ли текущий индекс i количества уровней в матрице переходов. Если это так, выполнение продолжается; в противном случае происходит завершение.

На следующем этапе к переменной `cumulativeProbability` добавляется вероятность из матрицы переходов для текущего уровня, которая определяется по строке `currentLevel - 1` и столбцу `i`. После этого алгоритм проверяет, меньше ли или равно сгенерированное случайное число `randomValue` накопленной вероятности `cumulativeProbability`. Если условие выполняется, алгоритм переходит к следующему шагу, в противном случае увеличивается счетчик `i`, и выполнение возвращается к предыдущему шагу проверки условий цикла.

Если уровень найден, алгоритм возвращает индекс уровня, увеличенный на 1, так как индексация начинается с 0. В противном случае, если уровень не был найден, алгоритм завершает свою работу.

2.7 Выбор и обоснование комплекса программных средств

В настоящее время имеется множество программных решений, так что при выборе подходящего продукта учитывались различные критерии, позволяющие оценить его качество.

2.7.1 Выбор языка программирования

При разработке серверной части веб-приложения было решено использовать технологии ASP.NET и язык программирования C#. Это было обусловлено несколькими факторами.

Во-первых, ASP.NET Core обеспечивает быструю обработку запросов благодаря встроенному оптимизированному серверу Kestrel и эффективному использованию современных технологий компиляции.

Во-вторых, переход на ASP.NET Core делает разработку приложения независимой от конкретной операционной системы, что позволяет запускать сервер на различных платформах, таких как Windows, Linux и macOS, и тем самым упрощает процесс развертывания и масштабирования.

Кроме того, C# – мощный типобезопасный язык, который предлагает широкий спектр возможностей для объектно-ориентированного и

асинхронного программирования [15]. Эти функциональные особенности упрощают управление игровыми событиями, обработку действий пользователей и реализацию многопоточных операций. Важно отметить, что специфика приложения исключает необходимость работы с базой данных, так как все данные хранятся и обрабатываются в оперативной памяти. ASP.NET Core предлагает удобные инструменты для управления кэшем, коллекциями и сериализацией данных, что делает его идеальным выбором для легковесной архитектуры.

Наконец, ASP.NET Core и C# обладают богатой экосистемой библиотек и встроенных возможностей, включая работу с WebSocket, реализацию API и обработку JSON. Эти инструменты ускоряют процесс разработки и упрощают реализацию сетевого взаимодействия.

При разработке клиентской части системы был сделан выбор в пользу JavaScript как основного языка программирования, и это решение основывается на нескольких ключевых факторах.

Во-первых, JavaScript является универсальным языком. Это означает, что один и тот же язык можно применять для разработки всех слоев приложения, что упрощает процесс разработки, уменьшает потребность в переквалификации команды и обеспечивает более плавное взаимодействие между клиентом и сервером.

Кроме того, JavaScript имеет одно из самых больших сообществ разработчиков, что обеспечивает доступ к обширной коллекции библиотек и фреймворков. Эти инструменты значительно упрощают разработку.

JavaScript также обеспечивает высокую производительность благодаря своей неблокирующей модели ввода-вывода. Это позволяет приложениям обрабатывать множество запросов одновременно, что критически важно для обеспечения быстрой отзывчивости и возможности одновременной работы нескольких пользователей.

Кроме того, JavaScript основан на событийной модели, что означает, что он может эффективно реагировать на события (такие как HTTP-запросы) и

обрабатывать их асинхронно. Это позволяет максимально эффективно использовать ресурсы сервера и снижает время отклика.

Его универсальность, высокая производительность, активная экосистема, легкость в обучении и возможности интеграции делают JavaScript оптимальным выбором для построения современных, эффективных и масштабируемых веб-приложений.

В написании приложения так же использовался фреймворк React. Он предлагает компонентный подход, который позволяет разбивать интерфейс на переиспользуемые и независимые части. Это значительно упрощает процесс разработки, тестирования и последующей поддержки приложения, поскольку каждый компонент можно разрабатывать и отлаживать отдельно. Кроме того, React обеспечивает высокую производительность благодаря использованию виртуального DOM. Это означает, что изменения в интерфейсе обрабатываются более эффективно, так как React минимизирует количество операций с реальным DOM. В результате приложение работает быстрее и отзывчивее, что особенно важно для современных веб-приложений.

2.7.2 Выбор среды программирования

В качестве среды программирования используется Visual Studio Code (VS Code). Выбор обусловлен несколькими ключевыми факторами [16].

Во-первых, VS Code является легковесным и быстрым редактором, что позволяет разработчикам эффективно работать без значительных затрат ресурсов системы.

Во-вторых, он поддерживает множество языков программирования и предоставляет обширные возможности для настройки, что делает его универсальным инструментом для различных проектов.

Также стоит отметить удобный интерфейс и мощные инструменты, такие как интегрированный терминал, отладчик и система контроля версий, что значительно упрощает процесс разработки.

Наконец, активное сообщество пользователей и регулярные обновления от Microsoft обеспечивают постоянное улучшение функционала и исправление ошибок, что делает VS Code надежным выбором.

2.7.3 Выбор операционной системы

Создаваемая система предназначена для операционной системы Windows. Во-первых, она обладает широкой распространенностью и является одной из самых популярных ОС в мире, что обеспечивает значительное количество пользователей и разработчиков, что, в свою очередь, облегчает поиск поддержки и ресурсов [17].

Во-вторых, Windows предлагает обширный набор программного обеспечения и инструментов, что делает её идеальной для решения различных задач.

Кроме того, Windows обеспечивает хорошую совместимость с различным аппаратным обеспечением, что позволяет использовать широкий спектр устройств и периферии.

Также стоит отметить удобный интерфейс и высокую степень пользовательской настройки, что делает систему доступной как для новичков, так и для опытных пользователей. Наконец, регулярные обновления и поддержка со стороны Microsoft гарантируют, что операционная система остается актуальной и безопасной, что является критически важным.

3 Реализация системы

3.1 Разработка и описание интерфейса пользователя

Интерфейс — это набор средств и методов взаимодействия между пользователем и системой, машиной или программным обеспечением. Это может включать графические элементы, такие как кнопки, меню, иконки, а также текстовые команды и другие способы ввода и вывода информации. Интерфейс служит связующим звеном, через которое пользователи могут выполнять задачи и получать результат работы системы, обеспечивая удобство, доступность и эффективность взаимодействия.

Одной из ключевых особенностей интерфейсов является абстракция. Они позволяют скрыть детали реализации и предоставить только необходимую информацию о функционале объектов, что упрощает взаимодействие с ними. Кроме того, интерфейсы способствуют полиморфизму: объекты различных классов могут использоваться единообразно, если они реализуют один и тот же интерфейс, что упрощает расширение системы.

Важно отметить, что интерфейсы не могут содержать состояния и состоят только из объявлений методов и свойств, что делает их легкими и декларативными. Множественное наследование — еще одна особенность интерфейсов, поскольку класс может реализовывать сразу несколько интерфейсов, что создаёт гибкие и многоуровневые иерархии, что помогает избежать проблем, связанных с множественным наследованием в некоторых языках программирования.

Интерфейсы действуют как контракты: они определяют, какие методы должны быть реализованы. Если класс не реализует все методы интерфейса, он не будет скомпилирован, что обеспечивает строгий контроль. Эта контрактность также упрощает тестирование: используя интерфейсы, разработчики могут легко подменять реальные реализации на заглушки или моки для тестирования, что делает тестирование более гибким и модульным.

Интерфейсы предоставляют дополнительные возможности для расширяемости: легко добавляя новые методы, можно избежать нарушения старого кода. Если интерфейс изменяется, разработчики могут создавать новые классы, реализующие обновленный интерфейс, не затрагивая существующий функционал. Наконец, использование интерфейсов помогает минимизировать зависимости между классами, так как объекты взаимодействуют друг с другом через интерфейсы, а не напрямую, что делает архитектуру приложения более чистой и управляемой.

На рисунке 30 изображена главная страница приложения. На ней размещены логотип приложения, а также ключевые кнопки: «Create Room», которая переводит пользователя на страницу создания комнаты, а также кнопка «Connect», позволяющая присоединиться к уже созданной игровой сессии по коду, также кнопка «Help», по нажатию на которую будет открываться страница со справочной информацией.

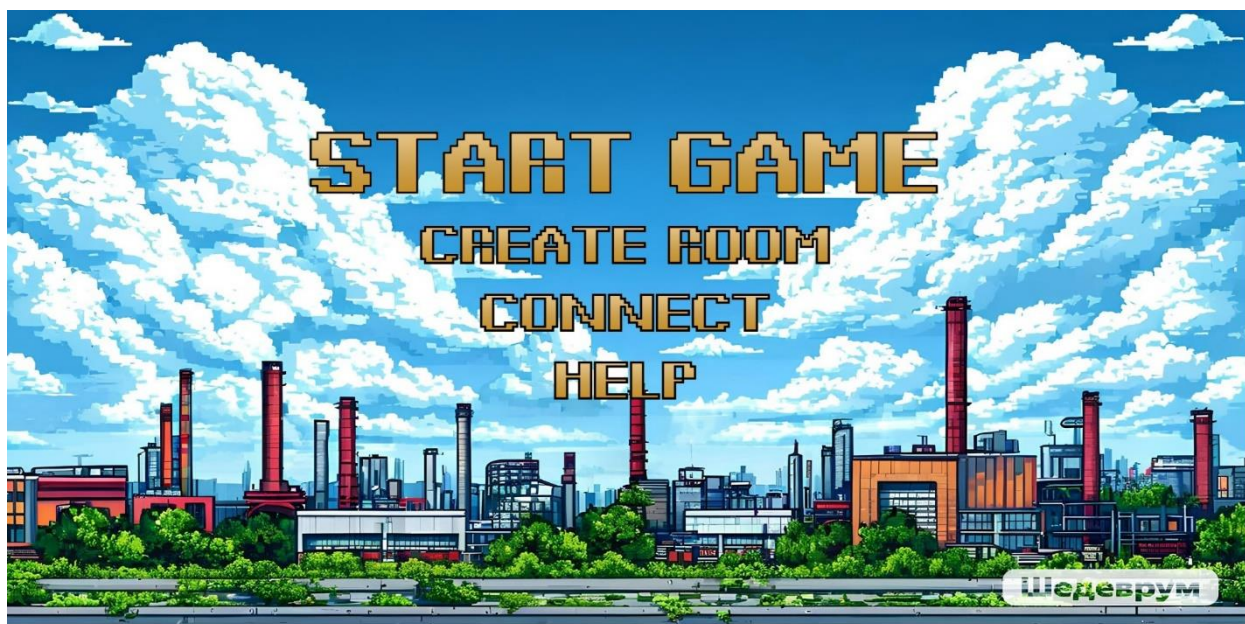


Рисунок 30 – Главная страница приложения

На рисунке 31 приведено регистрационное окно приложения. Здесь пользователям предложено несколько возможных вариантов аватара, обязательный для выбора, а также поле для ввода никнейма



Рисунок 31 – Регистрационное окно приложения

После регистрации в игре система открывает окно ожидания, изображение которого представлено на рисунке 32. Здесь отображены ключ для входа в комнату, количество игроков, присоединившихся к сессии, их аватары и никнеймы. Также предусмотрена кнопка для возврата на предыдущую страницу.

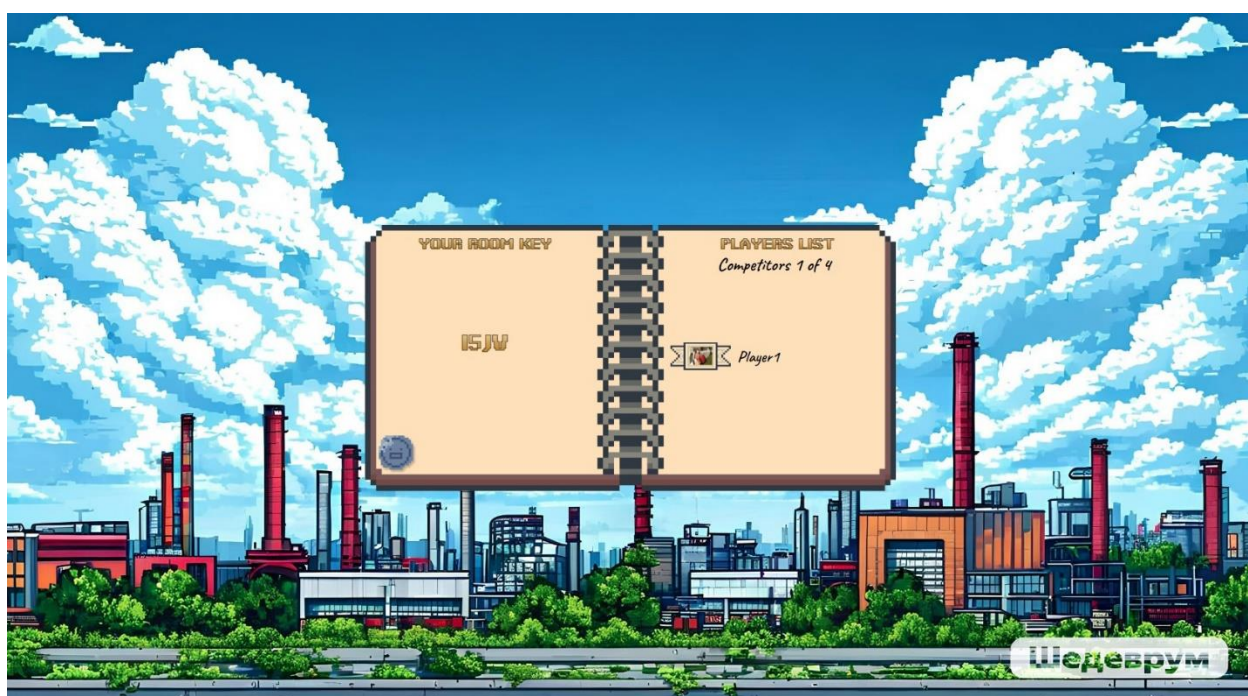


Рисунок 32 – Окно ожидания приложения

Как только подключается минимум игроков – у старшего игрока появляется кнопка для старта игры, при нажатии на которую открывается страница с игровым окном, которое изображено на рисунке 33. Если же старший игрок не нажимает на появившуюся кнопку в течение 60 секунд, то игра начинается автоматически с тем составом игроков, который успел подключиться.

Игровое окно приложения представляет собой центральный элемент интерфейса и предоставлять игрокам всю необходимую информацию для взаимодействия с игровым процессом. Верхняя, самая большая область принадлежит текущему игроку. На экране отображается информация о каждом игроке: аватар, состояние, количество ЕГП и ЕСМ, а также число фабрик и их уровень технологичности. У старшего игрока текущего раунда рамка дополнительно выделяется оранжевым цветом. На экране также отображается номер текущего круга.

В правом верхнем углу размещена информация о текущем ходе игры, включая количество и стоимость ЕГП и ЕСМ, текущий этап – преобразование фабрик, а также оставшееся время для совершения хода (30 секунд).

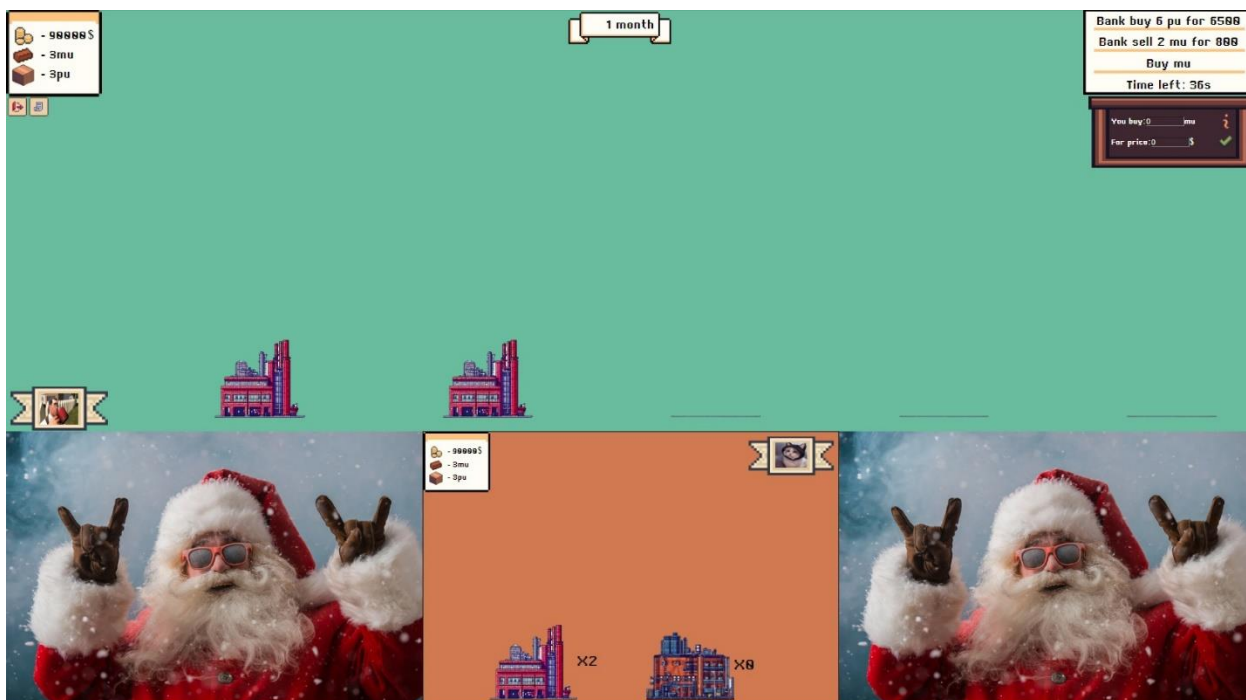


Рисунок 33 – Игровое окно приложения

На рисунке 34 изображено игровое окно на этапе закупки сырья, где отображается доступное игроку количество ЕСМ для закупки и её стоимость.

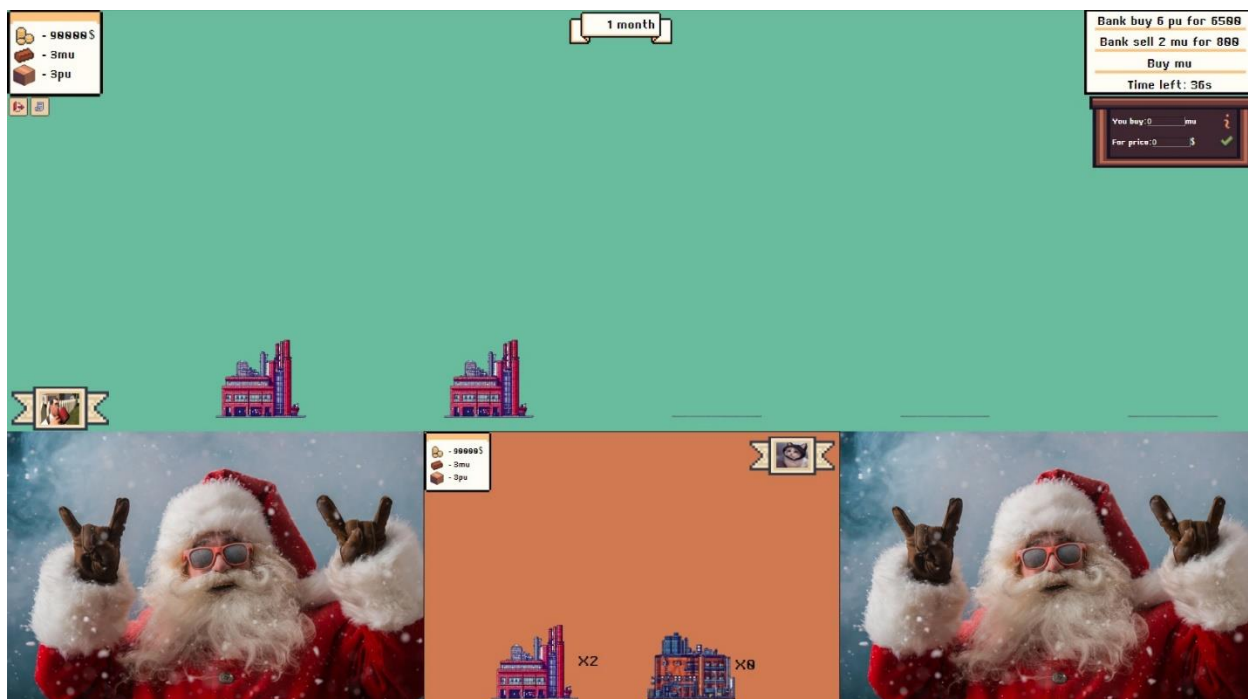


Рисунок 34 – Игровое окно на этапе закупки сырья

На рисунке 35 изображено игровое окно на этапе продажи продукции, где отображается доступное количество ЕПП, которое доступно игроку для продажи и стоимость.

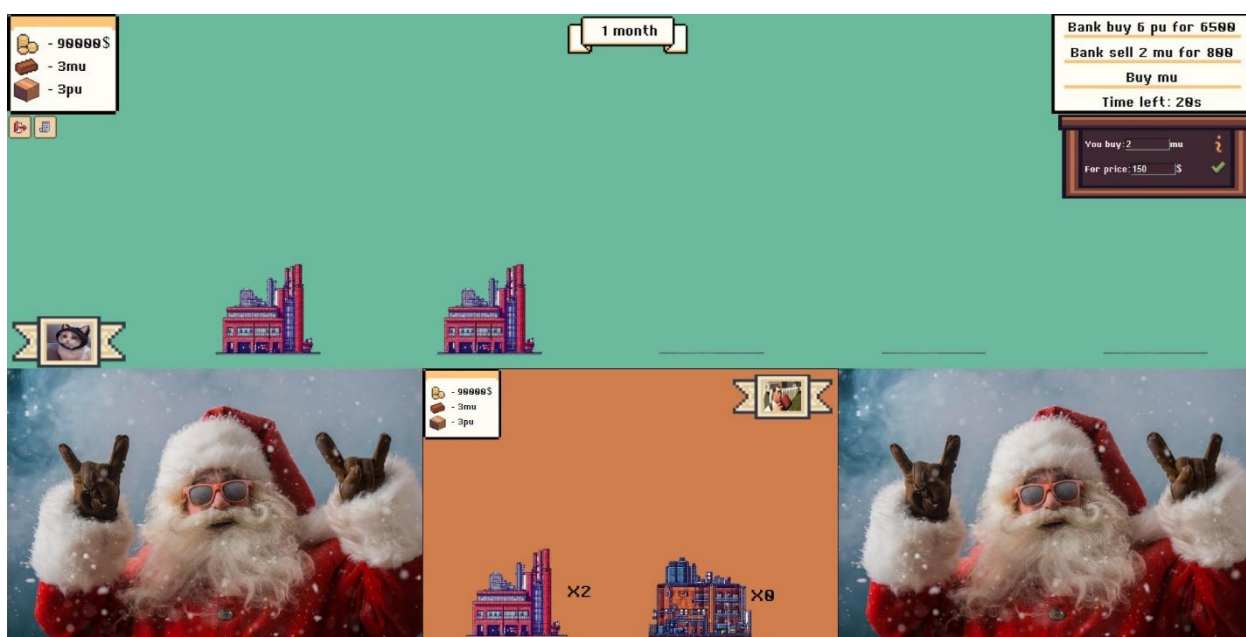


Рисунок 35 – Игровое окно на этапе продажи продукции

На рисунке 36 изображено игровое окно на этапе распределения сырья, где отображается информация о том, сколько продукции, на каком заводе и за какую стоимость можно изготовить.

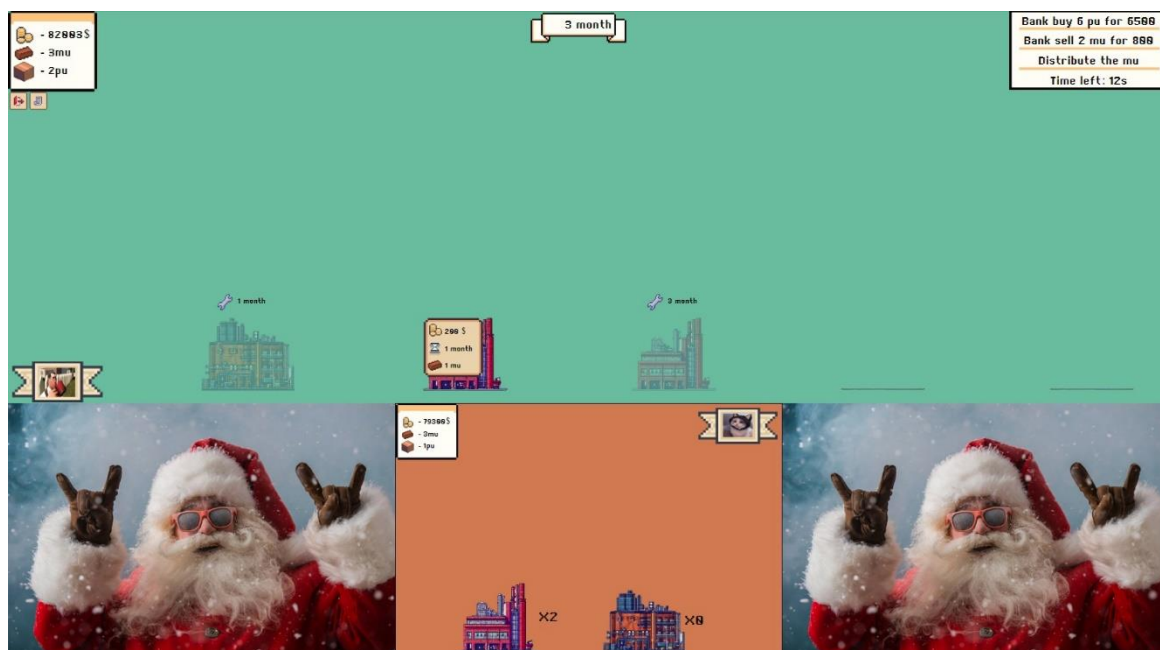


Рисунок 36 – Игровое окно на этапе распределения сырья

На рисунке 37 изображено игровое окно на этапе взятия ссуды, где отображается о сумме ссуды, которая доступна игроку на данный момент, и под залог какого завода она может быть взят.

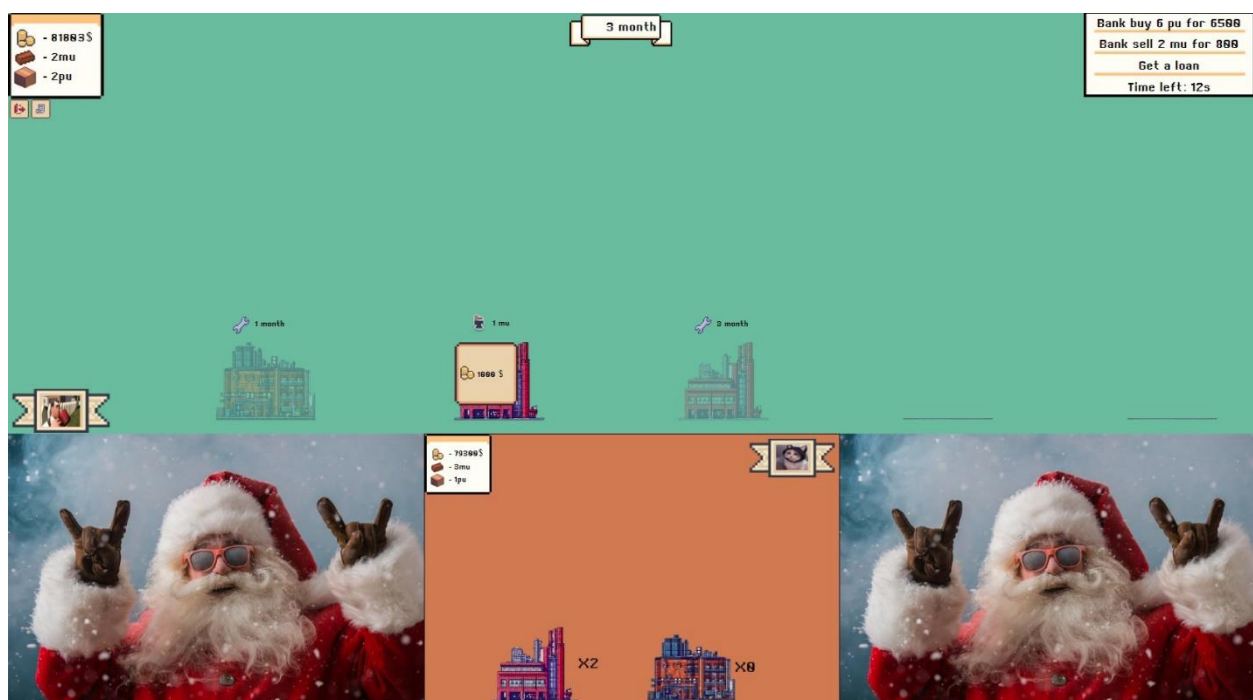


Рисунок 37 – Игровое окно на этапе взятия ссуды

На рисунке 38 приведено окно с информацией о разработчиках, а также руководство пользователя с правилами игры.

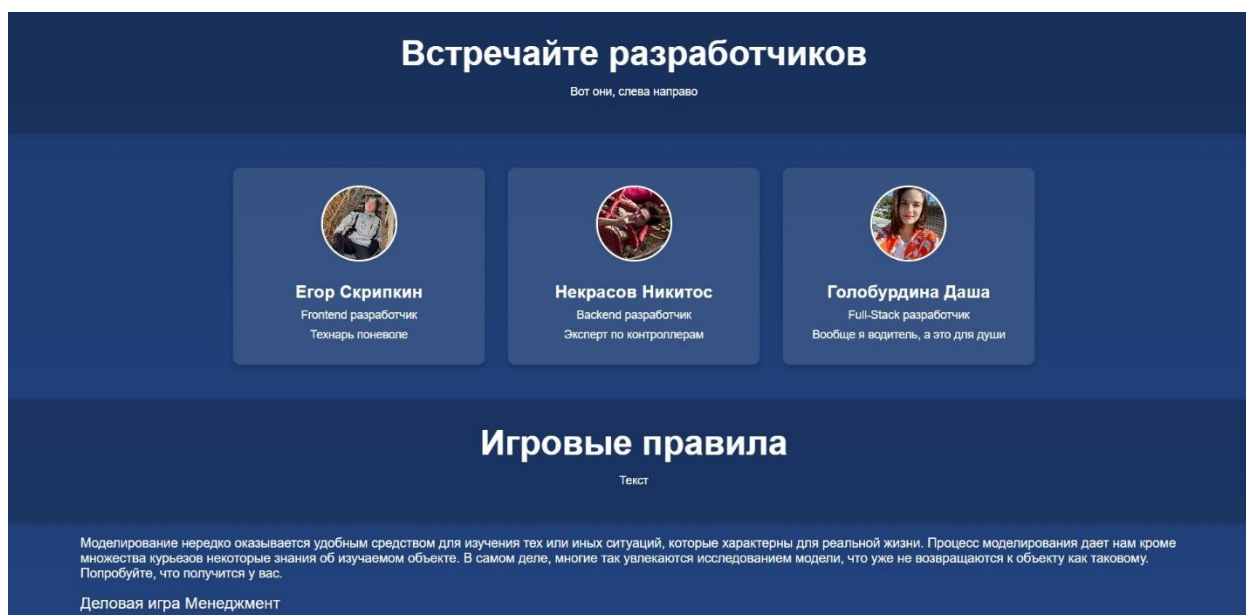


Рисунок 38 – Сведения о разработчиках

3.2 Диаграммы реализации

Диаграммы реализации предназначены для отображения состава компилируемых и выполняемых модулей системы, а также связей между ними.

Диаграммы реализации разделяются на два конкретных вида: диаграммы компонентов (component diagrams) и диаграммы развертывания (deployment diagrams).

3.2.1 Диаграмма компонентов

Диаграмма компонентов иллюстрирует, как различные программные компоненты взаимодействуют друг с другом.

Диаграмма компонентов позволяет представлять архитектурные решения и связи между частями системы, упрощая понимание её общего устройства.

На рисунке 39 приведена диаграмма компонентов, их описание приведено в таблице 6.

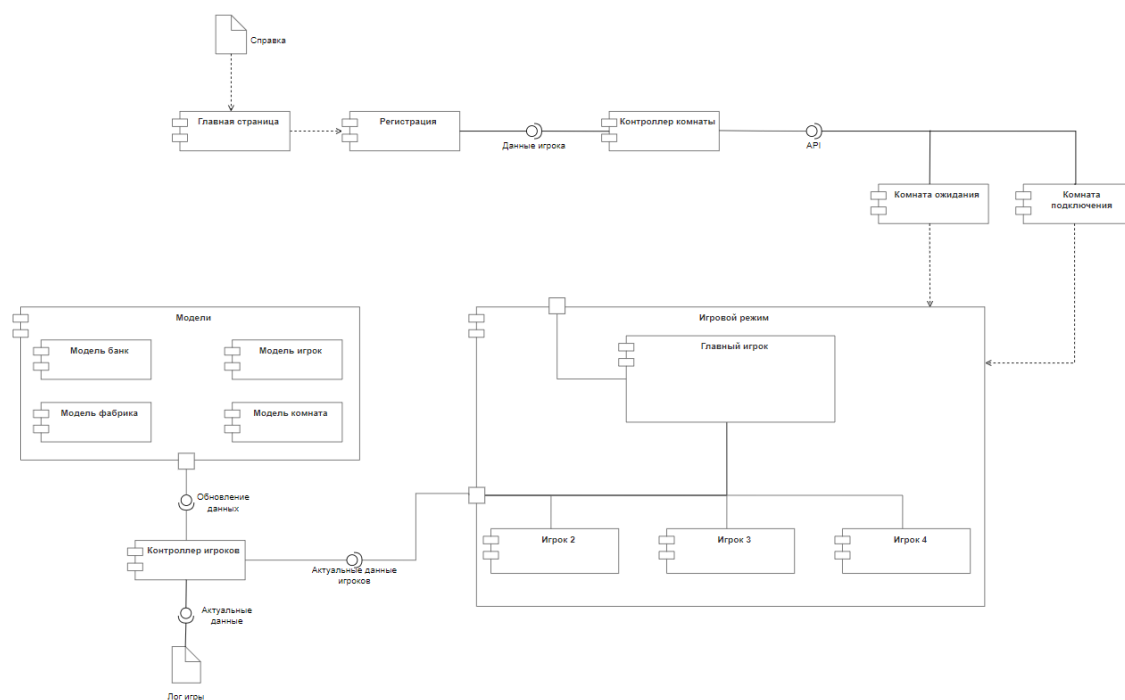


Рисунок 39 – Диаграмма компонентов системы

Таблица 6 – Описание компонентов системы

Название компонента	Назначение компонента	Подсистема
1	2	3
Главная страница	Позволяет игроку выбрать между созданием комнаты и присоединением к существующей, дает доступ к справочной информации.	Подсистема вывода данных
Регистрация	Получить данные о игровом нике и аватаре игрока	Подсистема регистрации
Контроллер комнаты	Сохранить данные игроков на сервере и отвечает за создание и подключение к комнате, генерирует код для подключения к данной комнате	Подсистема взаимодействия с клиентской частью
Комната ожидания	Отобразить код для подключения к комнате, осуществить мониторинг подключенных игроков и дать возможность главному игроку запустить игру при достаточном количестве игроков	Подсистема создания игровой сессии
Комната подключения	Произвести подключение к игровой комнате по коду	Подсистема гостевой авторизации

Продолжение таблицы 6

1	2	3
Игровой режим	Хранить и отображать актуальные данные игроков, выполнять, отправлять на сервер данные о выборе игроков и принимать актуальные данные	Подсистема визуализации
Главный игрок	Выполнять функции интерфейса текущего пользователя, принимать данные о выборе игрока и осуществлять их первичную валидацию	Подсистема выполнения ходов
Игрок N	Выводить актуальное состояние игроков конкурентов	Подсистема вывода данных
Контроллер игроков	Связывать клиентскую часть игры и серверную, предоставлять API для отправки запросов	Подсистема контроля игрового процесса
Модель банк	Управляет ресурсами и экономическими аспектами игровой комнаты. Она определяет доступное количество и цены ресурсов, обрабатывает заявки игроков на их покупку, а также контролирует уровень банка, изменяя его согласно вероятностной матрице.	Подсистема «Банк»
Модель фабрика	Представляет игровой завод и управляет его состоянием, уровнями и производственными процессами. Отвечает за строительство и модернизацию завода, обработку ресурсов, а также расчёт производственных результатов.	Подсистема контроля игрового процесса
Модель игрок	Управляет ресурсами игрока, отслеживает экономическое состояние и взаимодействует с игровыми механиками, такими как строительство, модернизация и залог заводов, обработка заявок на покупку или продажу ресурсов, а также управление ссудами.	Подсистема контроля игрового процесса

Продолжение таблицы 6

1	2	3
Модель комнаты	Управляет игровой комнатой, обеспечивая взаимодействие игроков, ход игры и распределение ресурсов	Подсистема контроля игрового процесса

3.2.2 Диаграмма развертывания

Диаграмма развертывания отображает, как программные компоненты и модули развертываются на физических узлах, таких как серверы, компьютеры, облачные платформы и другие устройства. Основная цель диаграммы развертывания – показать, как различные элементы системы взаимодействуют друг с другом в реальной среде и как они распределяются по физическим ресурсам.

На рисунке 40 приведена диаграмма развертывания системы.

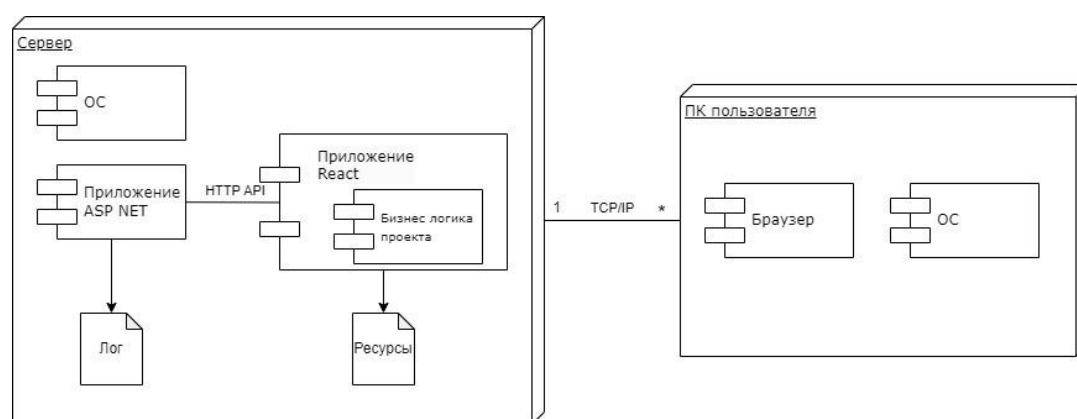


Рисунок 40 – Диаграмма развертывания системы

Данная диаграмма последовательности описывает взаимодействие между клиентским приложением, основанным на React, и серверным приложением, разработанным с использованием ASP.NET. В этом процессе участвуют несколько ключевых компонентов: ПК пользователя, на котором запускается приложение React, и браузер, через который пользователи взаимодействуют с интерфейсом.

Вся передача данных осуществляется с использованием протокола TCP/IP, который обеспечивает связь между клиентом и сервером. Сервер, на котором размещено приложение ASP.NET, отвечает за обработку запросов от клиентского приложения. Когда приложение React отправляет HTTP-запрос на сервер, приложение ASP.NET обрабатывает его и взаимодействует с HTTP

API, предоставляя необходимые данные. После завершения обработки сервер возвращает ответ в формате JSON.

Получив эти данные, приложение React отображает их в веб-браузере, обеспечивая пользователю доступ к актуальной информации. Параллельно с этим приложение ASP.NET записывает информацию о запросах в лог-файл, что позволяет отслеживать и анализировать работу сервиса.

3.2.3 Диаграмма классов

В соответствии со спецификацией, приведенной в п. 2.5.6, и с учетом выбранного языка программирования (см. п. 2.8.1) разработана диаграмма классов системы (этап реализации), приведенная на рисунке 41.

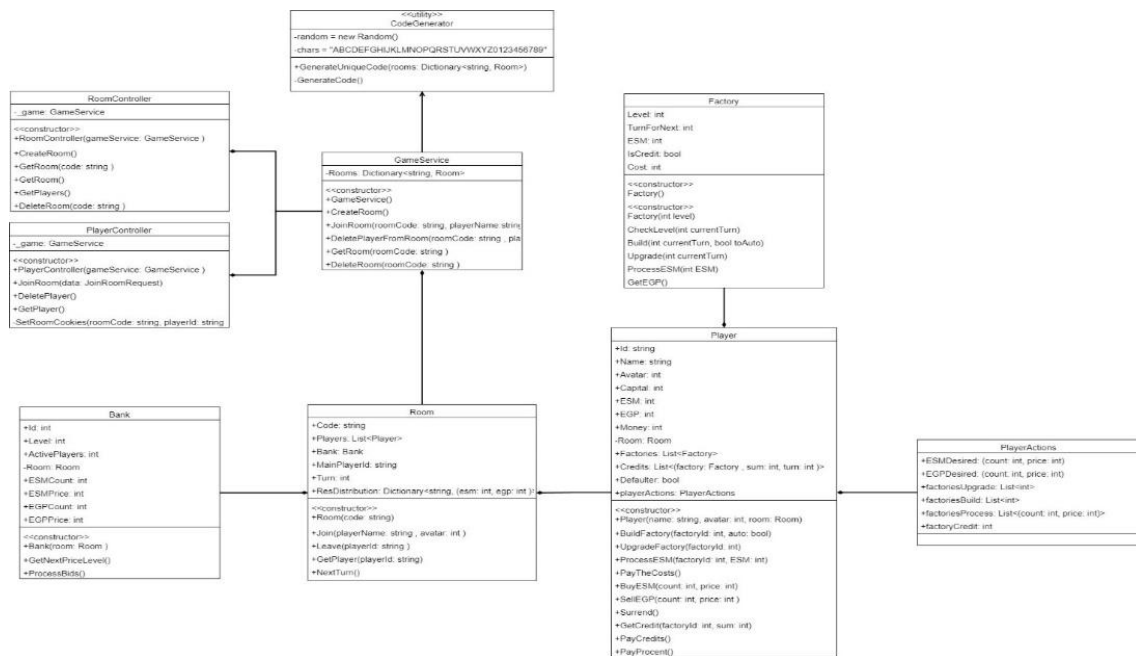


Рисунок 41 – Диаграмма классов системы (этап реализации)

На данной диаграмме представлено 7 классов:

- класс CodeGenerator отвечает за генерацию уникальных кодов для комнат с помощью метода GenerateUniqueCode();

- GameService: главный класс игры, управляющий комнатами и игроками, который содержит словарь Rooms для хранения всех комнат, и методы для создания новой комнаты (CreateRoom), удаления игрока из комнаты (DeletePlayerFromRoom) и получения комнаты по её коду (GetRoom);

- класс RoomController отвечает за управление комнатами, получения комнаты по её коду с помощью метода GetRoom и получения списка игроков в комнате, используя метод GetPlayers;
- класс PlayerController отвечает за управление игроками с помощью методов для присоединения к комнате JoinRoomData, удаления игрока при помощи DeletePlayer и отправки данных об игре методом SendRoomCookies;
- класс Factory предоставляет методы для создания новых фабрик: проверки уровня с помощью метода CheckLevel, запуска следующего хода, используя BuildNextTurn и обновления текущего хода при помощи UpgradeCurrentTurn;
- класс Room представляет комнату в игре, который содержит информацию о коде комнаты, игроках, фабриках и уровне;
- класс Player представляет игрока в игре, который содержит информацию о имени, аватарке, капитале, фабриках и кредитах.

3.3 Выбор и обоснование комплекса технических средств

3.3.1 Расчет объема занимаемой памяти

Расчет объема внешней памяти для сервера

Для расчета необходимого объема свободной внешней памяти, необходимой для функционирования серверной части системы, воспользуемся следующей формулой:

$$V_{\text{ЖД}} = V_{\text{ОС}} + V_{\text{ПР}} + V_{\text{СПО}} + V_{\text{ЛОГ}},$$

где $V_{\text{ОС}}$ – объем памяти, занимаемый операционной системой (операционная система Windows 11, $V_{\text{ОС}} = 25 \text{ Гб}$);

$V_{\text{ПР}}$ – объем памяти, занимаемый непосредственно файлами приложения ($V_{\text{ПР}} = 60 \text{ Мб}$);

$V_{\text{СПО}}$ – объем памяти, занимаемый сопутствующим программным обеспечением ($V_{\text{СПО}}$ в 280 Мб занимает Node Package Manager и 1 Гб занимает .NET);

$V_{\text{ЛОГ}}$ – объем памяти, занимаемый файлами лога игры ($V_{\text{ЛОГ}}$ не превышает 200 Мб);

Таким образом, суммарный объем внешней памяти составит:

$$V_{\text{ЖД}} = 25 \text{ Гб} + 60 \text{ Мб} + 280 \text{ Мб} + 1 \text{ Гб} + 200 \text{ Мб} \sim 26.7 \text{ Гб}.$$

Расчет объема ОЗУ

Для расчета необходимого объема ОЗУ воспользуемся следующей формулой:

$$V_{\text{ОЗУ}} = V_{\text{ОС}} + V_{\text{ПР}},$$

где $V_{\text{ОС}}$ – ОЗУ, занимаемое операционной системой (2 Гб);

$V_{\text{ПР}}$ – ОЗУ, которое займет само приложение (не более 2 Гб).

Суммарные объемы ОЗУ составит:

$$V_{\text{ОЗУ}} = 2 \text{ Гб} + 2 \text{ Гб} \sim 4 \text{ Гб}.$$

Таким образом, 4 Гб оперативной памяти можно считать минимально необходимым для функционирования серверной части системы.

Расчет объема внешней памяти для клиента

Для расчета необходимого объема свободной внешней памяти, необходимой для функционирования клиентской части системы, воспользуемся следующей формулой:

$$V_{\text{ЖД}} = V_{\text{ОС}} + V_{\text{браузера}},$$

где $V_{\text{ОС}}$ – объем памяти, занимаемый операционной системой (операционная система Windows 11, $V_{\text{ОС}} = 25 \text{ Гб}$);

$V_{\text{браузера}}$ – занимаемое браузером (не превышает 300 Мб);

Таким образом, суммарный объем внешней памяти составит:

$$V_{\text{ЖД}} = 25 \text{ Гб} + 300 \text{ Мб} \sim 25.3 \text{ Гб}.$$

Расчет объема ОЗУ для клиента

Для расчета необходимого объема ОЗУ воспользуемся следующей формулой:

$$V_{\text{ОЗУ}} = V_{\text{ОС}} + V_{\text{браузера}},$$

где $V_{\text{ОС}}$ – ОЗУ, занимаемое операционной системой (2 Гб);

$V_{\text{браузера}}$ – занимаемое браузером (не превышает 300 Мб);

Суммарные объемы ОЗУ составит:

$$V_{\text{ОЗУ}} = 2 \text{ Гб} + 300 \text{ Мб} = 2,3 \text{ Гб}.$$

Таким образом, 3 Гб оперативной памяти можно считать минимально необходимым для функционирования клиентской части системы.

3.3.2 Минимальные требования, предъявляемые к системе

Для корректного функционирования серверной части системы необходимо:

- тип ЭВМ: x86-64 совместимый;
- объем ОЗУ – не менее 4 Гб;
- объем свободного дискового пространства – не менее 26.7 Гб;
- процессор – Intel Core i3 2.0 ГГц или аналогичный;
- операционная система Windows Server 2016 или выше;
- Node Package Manager 18.x или 20.x.
- .Net 6 или 7.

Для корректного функционирования клиентской части системы необходимо:

- тип ЭВМ: x86-64 совместимый;
- процессор с тактовой частотой не менее 1.2 ГГц;

- объем ОЗУ – не менее 3 Гб;
- объем свободного дискового пространства – не менее 25.3 Гб;
- клавиатура или иное устройство ввода;
- мышь или иное манипулирующее устройство;
- операционная система: Windows 8 или выше, macOS 10.15 или выше, Linux с графической средой;
- браузер Google Chrome (версия 90 или выше), Mozilla Firefox (версия 88 или выше), Microsoft Edge (версия 91 или выше), Safari (версия 14 или выше).

ЗАКЛЮЧЕНИЕ

Во время курсового проектирования была разработана виртуальная экономическая система – игра «Менеджмент», в которой игроки принимают управленческие решения и реализуют стратегии для достижения финансового успеха. Система реализована в виде веб-приложения.

В первом разделе представлены ключевые термины предметной области, описаны характеристики существующих систем–аналогов, а также проведен их сравнительный анализ. На основе этого анализа составлена диаграмма объектов и сформулирована постановка задачи.

Во втором разделе была выбрана архитектура системы, разработаны структурная схема и функциональная спецификация. Описаны исключительные ситуации и предусмотрены реакции системы на них. Кроме того, созданы прототипы пользовательского интерфейса, а также построена логическая модель данных. Разработан информационно-логический проект по методологии UML, в который вошли все канонические диаграммы. Также был выбран набор программных средств для реализации системы.

Третий раздел посвящен описанию реализации системы. В нем описан пользовательский интерфейс, а также приведены диаграммы реализации. Произведены расчеты объема занимаемой памяти и определены минимальные системные требования.

Разработанная система позволяет моделировать различные аспекты экономической деятельности, участники могут ощутить себя в роли руководителей предприятий.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Особенность обучения менеджменту [Электронный ресурс]. URL: <https://www.cfin.ru/press/management/2001-5/11.shtml> (дата обращения: 28.09.2024).
- 2 Бизнес-симулятор [Электронный ресурс]. URL: https://en.wikipedia.org/wiki/Business_simulation_game (дата обращения: 28.09.2024).
- 3 RAD (программирование) [Электронный ресурс]. URL: [https://ru.wikipedia.org/wiki/RAD_\(программирование\)](https://ru.wikipedia.org/wiki/RAD_(программирование)) (дата обращения: 20.12.2024).
- 4 Уточнение модели предметной области в ООАД [Электронный ресурс]. URL: <https://www.geeksforgeeks.org/domain-model-refinement-in-ood/pdf> (дата обращения: 03.11.2024).
- 5 О проекте бизнес-курс [Электронный ресурс]. URL: <https://vkkb.ru/aboutbc.html> (дата обращения: 20.12.2024).
- 6 Monopoly-one играй в Монополию с друзьями онлайн [Электронный ресурс]. URL: <https://monopoly-one.com/>. (дата обращения: 24.12.2024).
- 7 Экономическая игра [Электронный ресурс]. URL: <https://virtonomica.ru/blogs/delovye-igry/> (дата обращения: 02.11.2024).
- 8 Системный анализ [Электронный ресурс]. URL: <https://victor-safronov.ru/systems-analysis/lectures/rodionov/04.html> (дата обращения: 20.12.2024).
- 9 Веб-приложение [Электронный ресурс]. URL: <https://ru.wikipedia.org/wiki/Веб-приложение> (дата обращения: 17.12.2024).
- 10 Зеленко Л.С. Методические указания к лабораторному практикуму по дисциплине «Программная инженерия». Самара: СГАУ, 2012-2024. 57 с.
- 11 Интерфейс (объектно-ориентированное программирование) [Электронный ресурс]. URL:

[https://ru.wikipedia.org/wiki/Интерфейс\(объектно-ориентированное_программирование\)](https://ru.wikipedia.org/wiki/Интерфейс(объектно-ориентированное_программирование)) (дата обращения: 20.12.2024).

12 UML [Электронный ресурс]. URL: <https://ru.wikipedia.org/wiki/UML> (дата обращения: 20.12.2024).

13 Лекция 2: Основные элементы языка UML [Электронный ресурс]. URL: <https://intuit.ru/studies/courses/32/32/lecture/1002?page=3> (дата обращения: 20.12.2024).

14 Лекция 4: Спецификация требований и рекомендации по написанию эффективных вариантов использования [Электронный ресурс]. URL: <https://victor-safronov.ru/systems-analysis/lectures/rodionov/04.html> (дата обращения: 20.12.2024).

15 C Sharp [Электронный ресурс]. URL: https://ru.wikipedia.org/wiki/C_Sharp (дата обращения: 20.12.2024).

16 Visual Studio Code [Электронный ресурс]. URL: https://ru.wikipedia.org/wiki/Visual_Studio_Code (дата обращения: 20.12.2024).

17 Виндовс – что такое [Электронный ресурс]. URL: <https://skyeng.ru/magazine/wiki/it-industriya/chto-takoe-vindows/> (дата обращения: 20.12.2024).

ПРИЛОЖЕНИЕ А

Руководство пользователя

А.1 Назначение системы

Система «Менеджмент» представляет собой виртуальную экономическую игру, в которой игроки выступают в роли руководителей предприятий, принимая управленческие решения для достижения финансового успеха. Игра реализована в виде веб-приложения и включает в себя множество ключевых функций.

Во-первых, система обеспечивает управление игроками, позволяя им подключаться к игровым сессиям и регистрироваться. Она контролирует время выполнения ходов и подачу заявок, а также отображает актуальную игровую ситуацию и состояние каждого игрока.

Во-вторых, система реализует игровую логику, что включает в себя правила игры и обработку действий игроков, изменяя состояние игры в соответствии с их решениями. Ведение учета активов и капитала каждого игрока также является важной частью этой функции.

Банк, как важный элемент игры, формирует и контролирует цены на готовую и сырьевую продукцию, а также рассматривает заявки игроков на их покупку и продажу.

Игроки имеют возможность выбирать доступные игровые действия в зависимости от ситуации, включая подачу заявок на получение сырья, переработку, продажу продукции, выбор ссуды и строительство новых фабрик.

Игра заканчивается либо после определенного числа кругов, либо при банкротстве всех игроков, кроме одного.

А.2 Условия работы системы

Для корректной работы системы необходимо наличие соответствующих программных и аппаратных средств:

- 1) Требования к техническому обеспечению:

- ЭВМ типа IBM PC;
- процессор типа x86 или x64 тактовой частоты 2000 МГц и выше;
- объем оперативной памяти (ОЗУ) — не менее 4 ГБ;
- объем свободного дискового пространства — не менее 26.4 Гб.

2) Требования к программному обеспечению:

- Windows Server 2016 или выше;
- установленный Node Package Manager 18.x или 20.x;
- установленный .Net 6 или 7.

Для корректной работы клиентской части системы необходимо наличие соответствующих программных и аппаратных средств:

1) Требования к техническому обеспечению:

- ЭВМ типа IBM PC;
- процессор типа x86 или x64 тактовой частоты 1200 МГц и выше;
- объем оперативной памяти (ОЗУ) — не менее 2 Гб;
- объем свободного дискового пространства – не менее 25.3 Гб;
- устройство ввода: клавиатура или иное устройство ввода;
- мышь или иное манипулирующее устройство.

2) Требования к программному обеспечению:

- операционная система: Windows 8 или выше, macOS 10.15 или выше, Linux с графической средой;
- установленная платформа Docker Desktop версии 3.3.0 и выше;
- браузер Google Chrome (версия 90 или выше), Mozilla Firefox (версия 88 или выше), Microsoft Edge (версия 91 или выше), Safari (версия 14 или выше).

А.3 Установка системы

Система поставляется в виде zip-архива. Данный файл необходимо распаковать в любую директорию на жестком диске.

Для запуска клиентской части необходимо открыть терминал в папке Power brokers и установить все модули при помощи команды `npm install`. После

чего необходимо запустить проект командой `npm run dev`. Клиентская часть будет доступна по адресу `http://127.0.0.1/`.

А.4 Работа с системой

А.4.1 Работа с системой в режиме пользователя

Чтобы начать игру необходимо зарегистрироваться. Для этого необходимо нажать кнопку «Connect» или «Create» на приветственной странице (см. рисунок А.1). Система откроет окно регистрации.



Рисунок А.1 – Приветственное окно

Вход в систему (регистрация)

Для регистрации необходимо выбрать аватар из предложенных и ввести никнейм, после чего нажать на кнопку продолжения «» (см. рисунок А.2). Будет открыто окно ожидания.

Для того, чтобы вернуться на главную страницу нужно нажать на кнопку возврата «».

Присоединение к игре

Для присоединения к игре по коду в окне ожидания необходимо ввести уникальный код в поле «», в случае создания новой игры ввод кода старшим игроком не требуется (см. рисунок А.3).



Рисунок А.2 – Окно регистрации

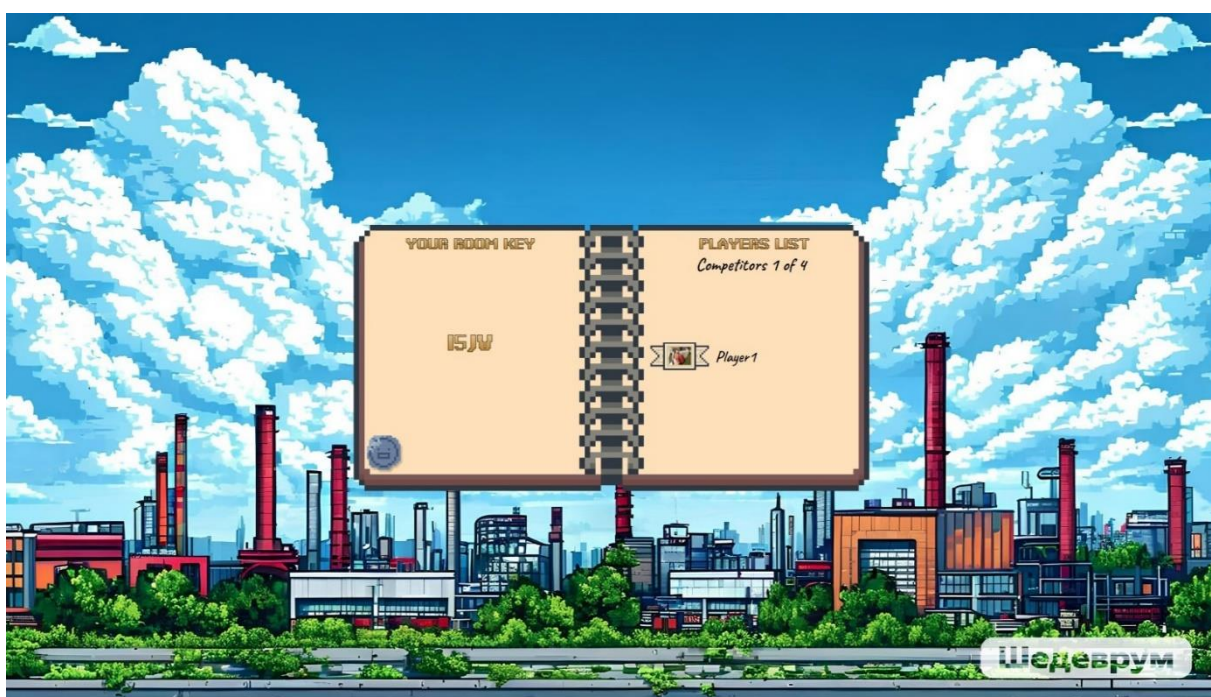


Рисунок А.3 – Окно ожидания

Как только минимальное количество игроков подключится, старший игрок должен будет начать игру с помощью кнопки продолжения.

В случае, если комната по заданному коду не найдена, то появляется всплывающее сообщение об ошибке (см.рисунок А.4).

Если игроков в комнате превысило допустимое количество, то также выведется сообщение об ошибке (см.рисунок А.5).

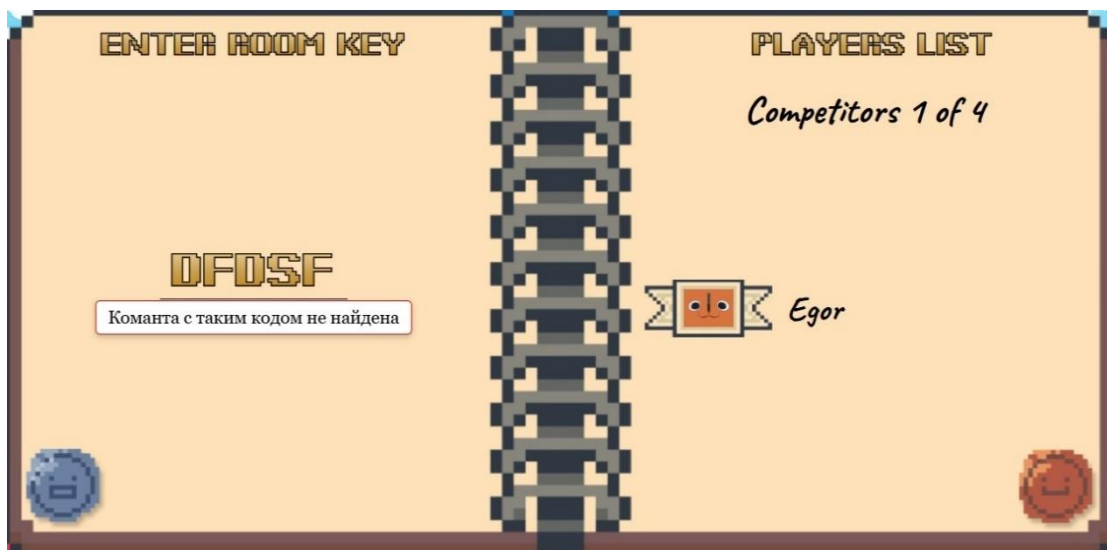


Рисунок А.4 – Окно ожидания с сообщением об ошибке

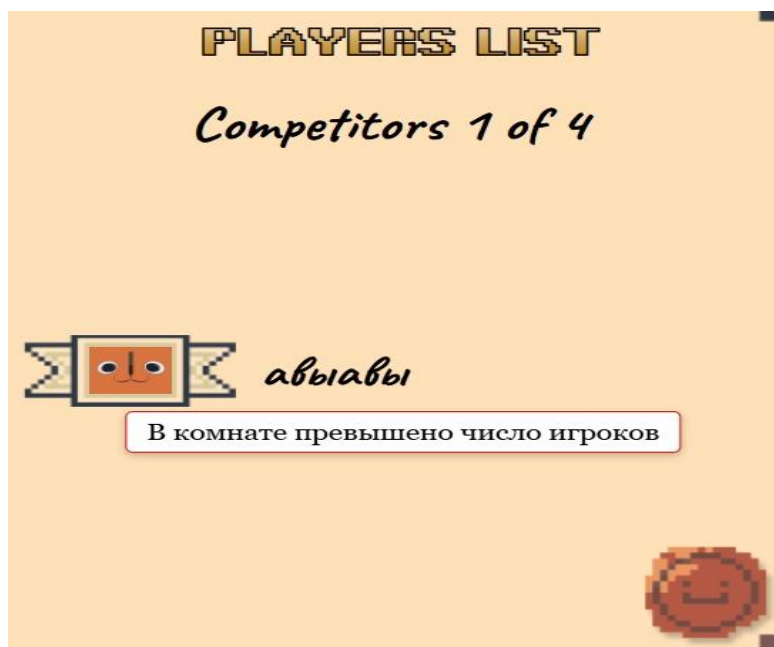


Рисунок А.5 – Окно ожидания с сообщением об ошибке

Игровой процесс

После запуска игры откроется игровое окно, которое является центральным элементом интерфейса (см.рисунок А.6). В верхней области отображаются данные о текущем игроке, включая его аватар, имя, состояние, количество ресурсов, а также количество и уровень фабрик. Номер текущего круга игры отображается сверху экрана.

В правом верхнем углу показаны информация о текущем ходе игры, включая текущую стадию, количество и стоимость ресурсов, а также оставшееся время для совершения хода.

В любой момент времени можно покинуть игру, тем самым признав себя банкротом, нажав на кнопку выхода «».

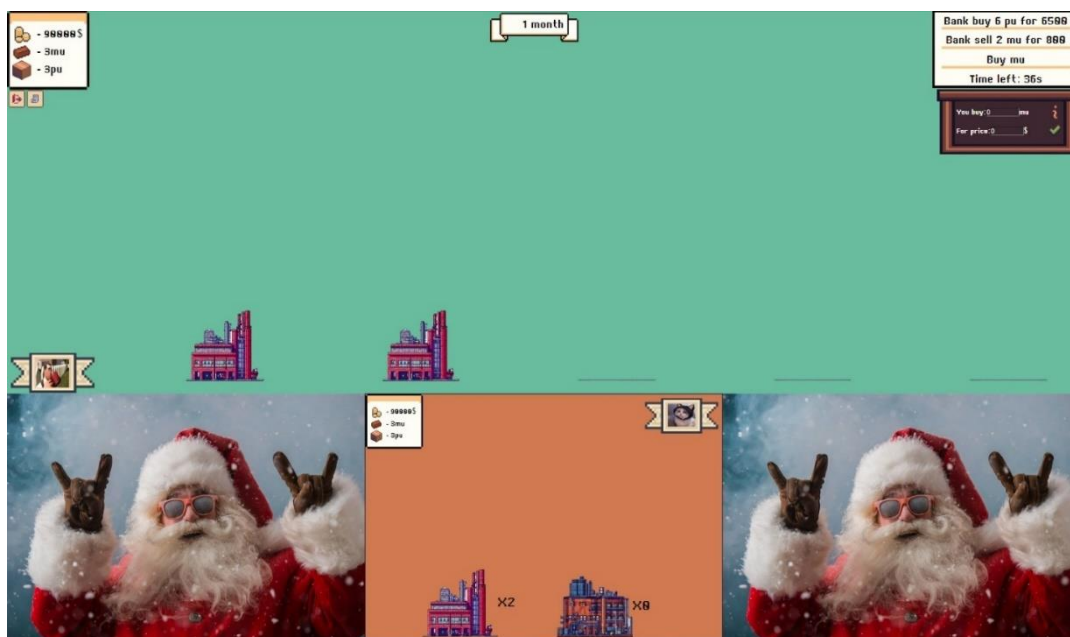


Рисунок А.6 – Игровое окно

На этапе закупки сырья для просмотра доступно количество ЕСМ и их стоимость. Игрок должен выбрать необходимые ресурсы, введя необходимое количество, указать предлагаемую стоимость и нажать на кнопку подтверждения заявки «» (см.рискнок А.7).



Рисунок А.7 – Поля для ввода

По истечении таймера этап завершается. Игрок должен помнить, что банку выгоднее продать ЕСМ за большую стоимость, поэтому в условиях ограниченности ЕСМ банк будет выбирать самую выгодную заявку среди всех заявок игроков. Остальные заявки будут игнорироваться. Игрок может отправить только одну заявку на покупку ЕСМ. В случае, если игрок вводит количество ЕСМ большее, чем имеется у банка или стоимость, превышающую состояние у игрока, то выводится сообщение об ошибке (см.рисунок А.8).



Рисунок А.8 – Сообщение об ошибке

Во время стадии продажи продукции отображается доступное количество ЕГП и их стоимость. Необходимо выбрать количество продукции для продажи и подтвердить сделку, нажав на кнопку «✓» (см.рисунок А.9).



Рисунок А.9 – Поля для ввода

Этап завершится либо когда все игроки отправят свои заявки, либо по истечении времени таймера. Игрок должен помнить, что банку выгоднее купить ЕГП за меньшую стоимость, поэтому в условиях ограниченности ЕГП банк будет выбирать самую выгодную заявку среди всех заявок игроков. Остальные заявки будут игнорироваться. Игрок может отправить только одну заявку на продажу ЕГП. В случае неверного ввода выводится сообщение об ошибке (см.рисунок А.10).

На этапе распределения сырья предоставлена информация о том, сколько продукции можно создать на каждом заводе и по какой стоимости (см.рисунок А.11). Нужно выбрать, как распределить ресурсы между своими заводами, нажав на кнопки с подходящими условиями.

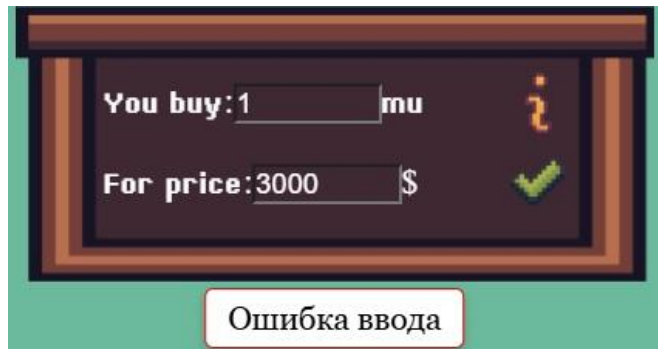


Рисунок А.10 – Сообщение об ошибке

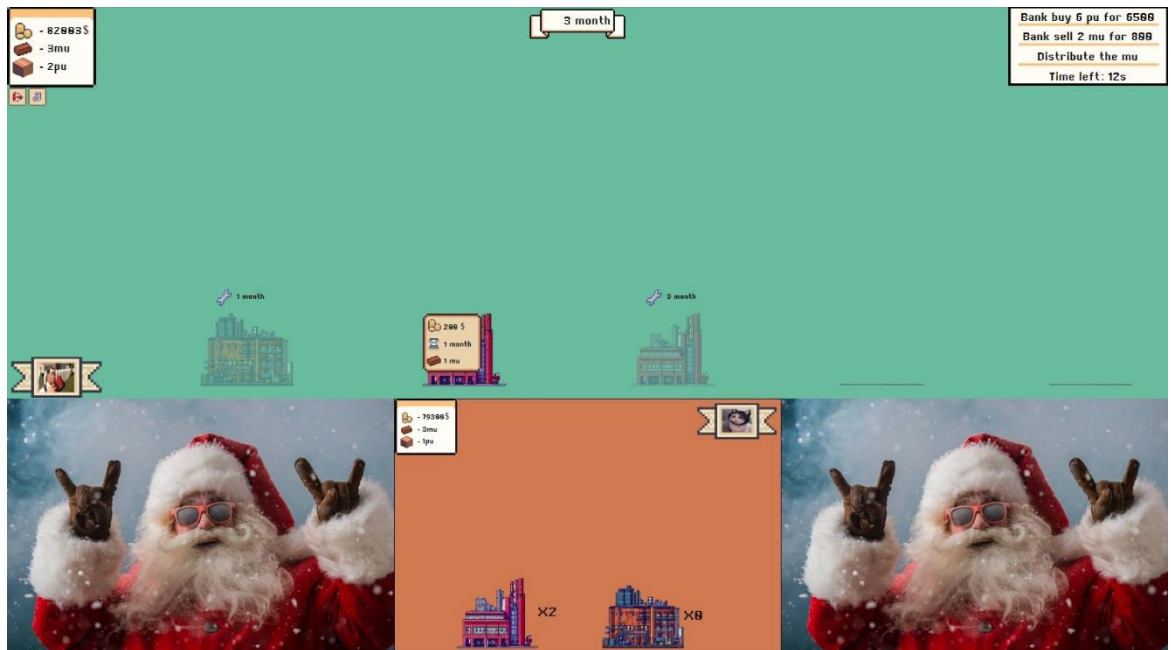


Рисунок А.11 – Игровое окно распределения сырья

В случае, если нажать на кнопку для переработки, но ресурсов для этого недостаточно, выводится сообщение об ошибке (см.рисунок А.12). По истечении времени этап завершается.



Рисунок А.12 – Сообщение об ошибке

На этапе взятия ссуды игрок сможет получить ссуду под залог в виде завода (см.рисунок А.13). Чтобы взять ссуду, необходимо выбрать нужный завод и подтвердить свое решение, нажав на кнопку подтверждения «». По истечении времени этап завершается.

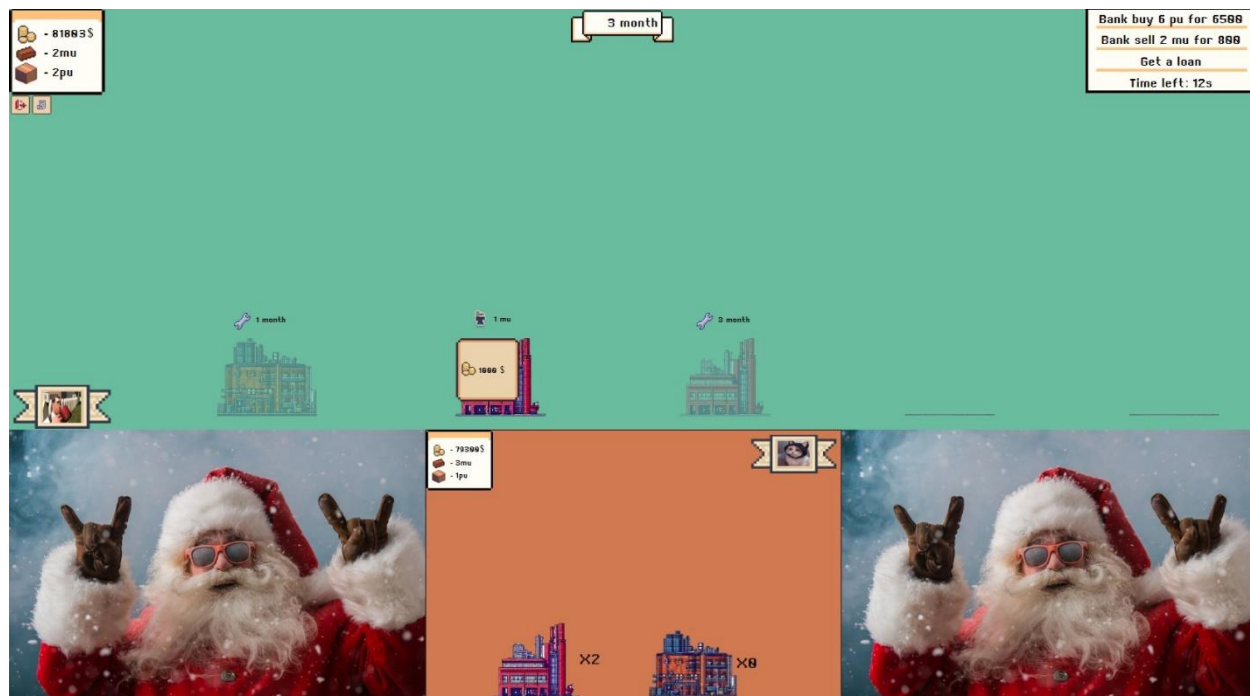


Рисунок А.13 – Игровое окно взятия ссуды

На этапе постройки фабрики игроку доступна информация о стоимости и времени постройки, а также тип завода (обычный или улучшенный). Для начала постройки необходимо нажать кнопку с подходящими условиями (см.рисунок А.14).

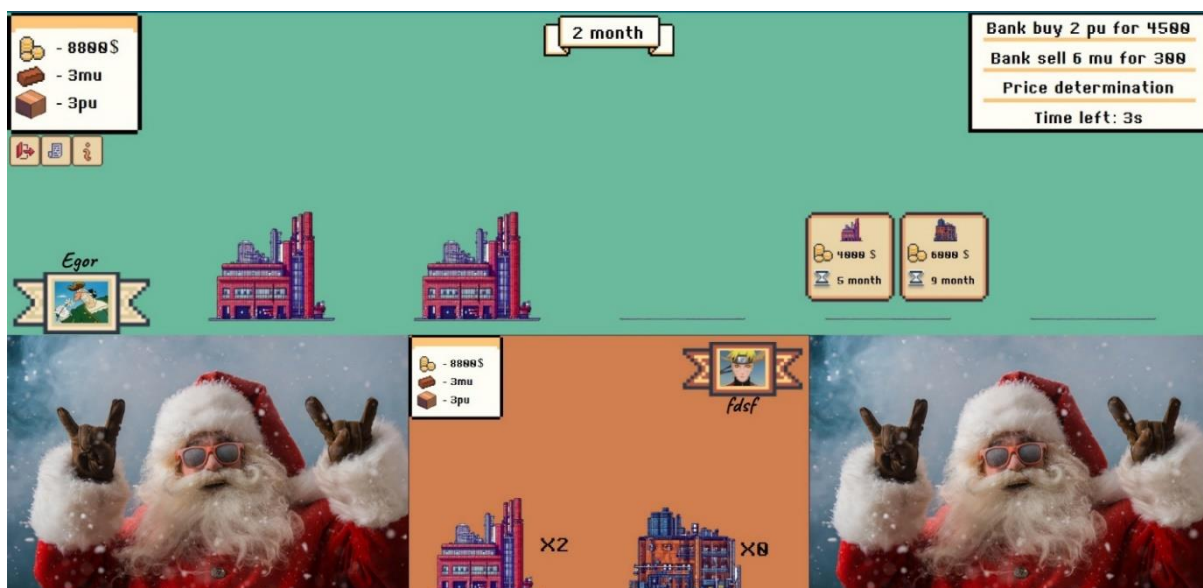


Рисунок А.13 – Игровое окно взятия ссуды

В случае, если выбрать постройку, для которой ресурсов недостаточно, то выводится сообщение с ошибкой (см.рисунок А.14).

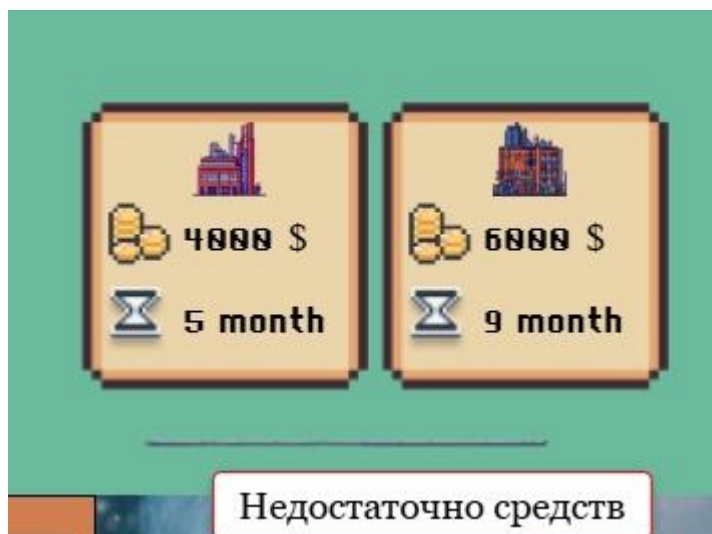


Рисунок А.14 – Сообщение об ошибке

В любой момент можно открыть окно с информацией о разработчиках и ознакомиться с руководством пользователя, нажав на кнопку получения справки «» (см.рисунок А.15).

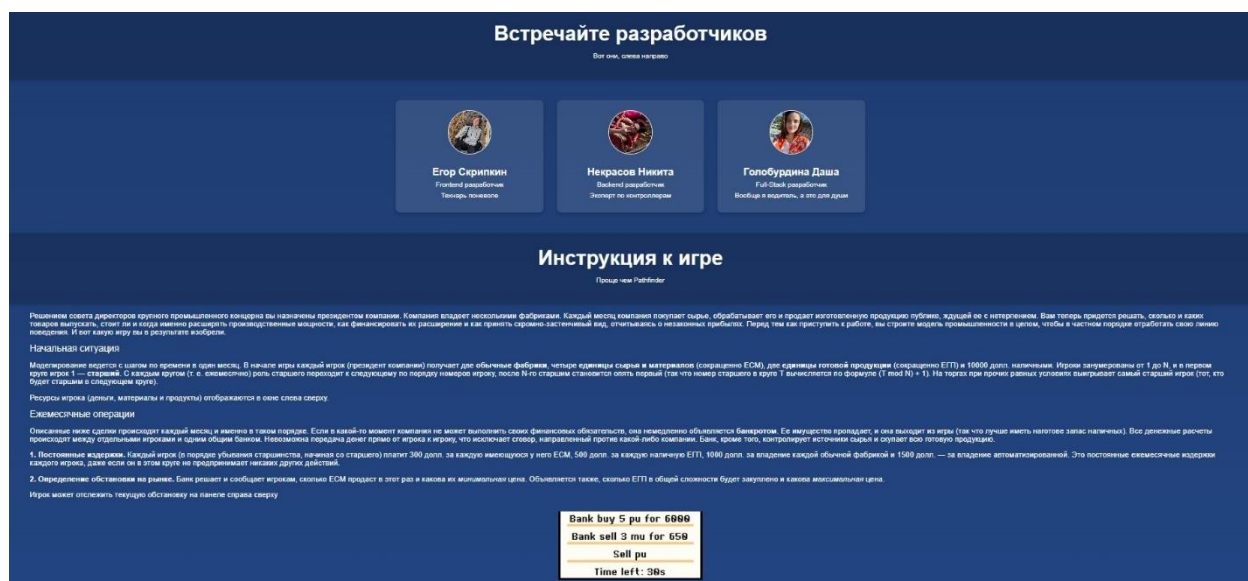


Рисунок А.15 – Окно со справочной информацией

Кроме того, нажав на кнопку «», можно посмотреть лог игры (см.рисунок А.16). Для того, чтобы закрыть окно с логом, необходимо нажать на кнопку «Подтвердить».

В случае победы в игре, выводится окно с поздравлением (см.рисунок А.17). А в случае проигрыша выводится окно с соответствующим сообщением (см.рисунок А.18).

ЗАПИСИ ИГРЫ

[14:00:30] Стадия 8: Получение кредитов.
[14:00:30] fdsf не имеет задолженностей по кредитам на этот месяц.
[14:00:30] Egor не имеет задолженностей по кредитам на этот месяц.
[14:00:30] Стадия 7: Выплата ссуд.
[14:00:24] fdsf не имеет кредитов.
[14:00:24] Egor не имеет кредитов.
[14:00:24] Стадия 6: Выплата ссудного процента.
[14:00:18] fdsf продал 0 ЕГП за 0\$.
[14:00:18] Заявка fdsf: 0 ЕГП за 0\$.
[14:00:18] Egor продал 0 ЕГП за 0\$.
[14:00:18] Заявка Egor: 0 ЕГП за 0\$.
[13:59:14] Стадия 5: Продажа продукции.
[13:58:45] Стадия 4: Производство продукции.
[13:58:45] fdsf приобрёл 0 ЕСМ за 0\$.
[13:58:45] Заявка fdsf: 0 ЕСМ за 0\$.
[13:58:45] Egor приобрёл 0 ЕСМ за 0\$.
[13:58:45] Заявка Egor: 0 ЕСМ за 0\$.
[13:57:40] Стадия 3: Обработка заявок на сырьё и материалы
[13:57:40] Стоимость ЕГП: 4500\$, Количество ЕГП: 2.
[13:57:40] Стоимость ЕСМ: 300\$, Количество ЕСМ: 6.
[13:57:40] Банк определил новый уровень цен - 5.
[13:57:40] Стадия 2: Определение обстановки на рынке.
[13:56:55] Стадия 3: Обработка заявок на сырьё и материалы
[13:56:55] Стоимость ЕГП: 5000\$, Количество ЕГП: 3.
[13:56:55] Стоимость ЕСМ: 400\$, Количество ЕСМ: 5.
[13:56:55] Банк определил новый уровень цен - 4.
[13:56:55] Стадия 2: Определение обстановки на рынке.
[13:55:52] Стадия 3: Обработка заявок на сырьё и материалы
[13:55:52] Стоимость ЕГП: 4500\$, Количество ЕГП: 2.
[13:55:52] Стоимость ЕСМ: 300\$, Количество ЕСМ: 6.

Подтвердить

Рисунок А.16 – Окно с логом игры

ПОЗДРАВЛЯЕМ, ВЫ ПОБЕДИЛИ!



Подтвердить

Рисунок А.17 – Окно победы

К СОЖАЛЕНИЮ, ВЫ ПРОИГРАЛИ



Подтвердить

Рисунок А.18 – Окно проигрыша

ПРИЛОЖЕНИЕ Б

Листинг модулей программы

```
using System;
namespace WebApplication2.Models
{
    public class Bank
    {
        //public int Id { get; set; }
        public int Level { get; set; }
        public int ActivePlayers
        {
            get
            {
                int active = 0;
                foreach (Player player in Room.Players)
                {
                    if (!player.Defaulter) active++;
                }
                return active;
            }
        }
        private Room Room { get; set; }
        public Bank(Room room)
        {
            Room = room;
            Level = 3;
        }
        public int ESMCount => Level switch
        {
            1 => (int)(ActivePlayers * 1.0),
            2 => (int)(ActivePlayers * 1.5),
            3 => (int)(ActivePlayers * 2.0),
            4 => (int)(ActivePlayers * 2.5),
            5 => (int)(ActivePlayers * 3.0),
            _ => throw new ArgumentOutOfRangeException("Level", "Уровень должен быть от 1 до 5")
        };
        public int ESMPPrice => Level switch
        {
            1 => 800,
            2 => 650,
            3 => 500,
            4 => 400,
            5 => 300,
            _ => throw new ArgumentOutOfRangeException("Level", "Уровень должен быть от 1 до 5")
        };
        public int EGPCount => Level switch
        {
            1 => (int)(ActivePlayers * 3.0),
            2 => (int)(ActivePlayers * 2.5),
            3 => (int)(ActivePlayers * 2.0),
            4 => (int)(ActivePlayers * 1.5),
            5 => (int)(ActivePlayers * 1.0),
            _ => throw new ArgumentOutOfRangeException("Level", "Уровень должен быть от 1 до 5")
        };
        public int EGPPPrice => Level switch
        {
            1 => 6500,
            2 => 6000,
            3 => 5500,
            4 => 5000,
```

```

5 => 4500,
_ => throw new ArgumentOutOfRangeException("Level", "Уровень должен быть от 1 до 5")
};
private static readonly double[,] transitionMatrix = {
    { 1.0 / 3, 1.0 / 3, 1.0 / 6, 1.0 / 12, 1.0 / 12 }, // Уровень 1
    { 1.0 / 4, 1.0 / 3, 1.0 / 4, 1.0 / 12, 1.0 / 12 }, // Уровень 2
    { 1.0 / 12, 1.0 / 6, 1.0 / 3, 1.0 / 4, 1.0 / 12 }, // Уровень 3
    { 1.0 / 12, 1.0 / 12, 1.0 / 4, 1.0 / 3, 1.0 / 4 }, // Уровень 4
    { 1.0 / 12, 1.0 / 12, 1.0 / 12, 1.0 / 3, 1.0 / 3 } // Уровень 5
};
public void NewPriceLevel()
{
    if (Level < 1 || Level > 5)
    {
        throw new ArgumentException("Уровень должен быть в диапазоне от 1 до 5.");
    }
    double randomValue = new Random().NextDouble(); // Генерируем случайное число [0, 1)
    double cumulativeProbability = 0.0;

    // Используем цикл с накоплением вероятностей
    for (int i = 0; i < 5; i++)
    {
        cumulativeProbability += transitionMatrix[Level - 1, i];
        if (randomValue <= cumulativeProbability)
        {
            Level = i + 1; // Уровни начинаются с 1
            return;
        }
    }
    throw new InvalidOperationException("Invalid transition matrix or probabilities.");
}
public Dictionary<Player, (int, int)> ProcessESMRequests()
{
    int eSMCount = ESMCount;

    // Сортируем игроков по ESMDesired.count
    SortPlayers(Room.Players, Room.MainPlayerId, x => x.actions.RequestedESM.price);

    // Обрабатываем заявки на ESM
    Dictionary<Player, (int count, int price)> result = new Dictionary<Player, (int, int)>();

    foreach (Player player in Room.Players)
    {
        if ((player.actions.RequestedESM.count > ESMCount) || (player.actions.RequestedESM.price <
ESMPrice) || (player.actions.RequestedESM.price * player.actions.RequestedESM.count > player.Money))
        {
            continue;
        }
        int receiveCount = Math.Min(player.actions.RequestedESM.count, eSMCount);
        int resPrice = 0;

        if (receiveCount > 0)
        {
            player.ESM += receiveCount;
            resPrice = receiveCount * player.actions.RequestedESM.price;
            player.Money -= resPrice;
            eSMCount -= receiveCount;
        }
        result.Add(player, (receiveCount, resPrice));
        if (eSMCount <= 0) break;
    }
    return result;
}

```

```

public Dictionary<Player, (int, int)> ProcessEGPRequests()
{
    int eGPCount = EGPCount;
    // Сортируем игроков по ESMDesired.count
    SortPlayers(Room.Players, Room.MainPlayerId, x => x.actions.RequestedEGP.price);
    // Обрабатываем заявки на ESM
    Dictionary<Player, (int count, int price)> result = new Dictionary<Player, (int, int)>();
    foreach (Player player in Room.Players)
    {
        if ((player.actions.RequestedEGP.count > player.EGP) || (player.actions.RequestedEGP.count >
EGPCount) || (player.actions.RequestedEGP.price > EGPPrice))
        {
            continue;
        }
        int receiveCount = Math.Min(player.actions.RequestedEGP.count, eGPCount);
        int resPrice = 0;

        if (receiveCount > 0)
        {
            player.EGP -= receiveCount;
            resPrice = receiveCount * player.actions.RequestedEGP.price;
            player.Money += resPrice;
            eGPCount -= receiveCount;
        }
        result.Add(player, (receiveCount, resPrice));
        if (eGPCount <= 0) break;
    }
    return result;
}

private void SortPlayers(List<Player> players, int mainPlayerId, Func<Player, int> keySelector)
{
    Random random = new Random();
    players.Sort((x, y) =>
    {
        int result = keySelector(x).CompareTo(keySelector(y));

        if (result == 0)
        {
            if (x.Id == mainPlayerId && y.Id != mainPlayerId)
            {
                return -1;
            }
            if (y.Id == mainPlayerId && x.Id != mainPlayerId)
            {
                return 1;
            }
            return random.Next(-1, 2);
        }
        return result;
    });
}

}
}
using static WebApplication2.GameSettings;
namespace WebApplication2.Models
{
    public class Factory
    {
        public int Level { get; private set; } // -1 - не построен,
        // 0 - строительство обычного завода,
        // 1 - строительство улучшенного завода,
        // 2 - обычный завод,
        // 3 - улучшение обычного завода,
        // 4 - улучшенный завод.
    }
}

```

```

public int TurnForNext { get; private set; } // Ход, на котором завод получит новый уровень
public int TurnForCredit { get; set; }
public int ESM { get; private set; }
public bool IsCredit { get; set; } // под залогом?
public int Cost => Level switch
{
    -1 => 0,
    0 or 1 or 2 or 3 => COST_FACTORY,
    4 => COST_AUTO_FACTORY,
    _ => throw new NotImplementedException()
};
public Factory()
{
    Level = -1;
    TurnForNext = 0;
    ESM = 0;
    IsCredit = false;
}
public Factory(int level)
{
    Level = level;
    TurnForNext = 0;
    ESM = 0;
    IsCredit = false;
}
public bool CheckLevel(int currentTurn)
{
    TurnForNext--;
    if (TurnForNext == 0) switch (Level)
    {
        case 0:
            Level = 2;
            TurnForNext = -1;
            return true;
        case 1 or 3:
            Level = 4;
            TurnForNext = -1;
            return true;
        default:
            break;
    }
    return false;
}
public bool Build(int currentTurn, bool toAuto)
{
    if (Level == -1)
        switch (toAuto)
        {
            case false:
                Level = 0;
                TurnForNext = FACTORY_BUILD_TIME;
                return true;
            case true:
                TurnForNext = FACTORY_AUTO_BUILD_TIME;
                Level = 1;
                return true;
        }
    return false;
}
public bool Upgrade(int currentTurn)
{
    switch (Level)

```

```

        {
            case 2:
                Level = 3;
                TurnForNext = FACTORY_UPGRADE_TIME;
                return true;
            default:
                break;
        }
        return false;
    }
    public int ProcessESM(int ESM)//-2 - не построен, -1 - нет места
    {
        switch (Level)
        {
            case 2:
                if ((this.ESM == 0) && (ESM >= 0) && (ESM <= 1))
                {
                    this.ESM += ESM;
                    return ESM;
                }
                return -1;
            case 4:
                if ((2 - this.ESM >= ESM) && (ESM >= 0) && (ESM <= 2))
                {
                    this.ESM += ESM;
                    return ESM;
                }
                return -1;
            default:
                return -2;
        }
        //if (Level < 2) this.ESM = 0;

        //if (ESM > 1 && Level == 4) this.ESM = 2;

        //else this.ESM = 1;

        //return this.ESM;
    }
    public int GetEGP()
    {
        int result = ESM;
        ESM = 0;
        return result;
    }
}
}namespace WebApplication2.Models
{
    public class Actions
    {
        public bool Surrend { get; set; }
        public (int count, int price) RequestedESM { get; set; }
        public (int count, int price) RequestedEGP { get; set; }
        public List<int> FactoriesUpgrade { get; set; }
        public List<(int id, bool auto)> FactoriesBuild { get; set; }
        public List<(int id, int esm)> FactoriesProcess { get; set; }
        public int FactoryCredit { get; set; }
        public Actions() {
            ClearActions();
        }
        public void ClearActions() {
            RequestedESM = (-1, -1);

```

```

        RequestedEGP = (-1, -1);
        FactoriesUpgrade = [];
        FactoriesBuild = [];
        FactoriesProcess = [];
        FactoryCredit = -1;
        Surrend = false;
    }
}
}using Microsoft.AspNetCore.Mvc;
using WebApplication2.Models;

namespace WebApplication2.Controllers
{
    [Route("api/room")]
    [ApiController]
    public class RoomController : ControllerBase
    {
        private readonly GameService _game;

        public RoomController(GameService gameService)
        {
            _game = gameService; // Внедрение зависимости через конструктор
        }

        [HttpPost]
        public ActionResult CreateRoom()
        {
            var room = _game.CreateRoom();

            // Возвращаем только код комнаты в теле ответа
            return CreatedAtAction(nameof(GetRoom), new { code = room.Code }, new { Code = room.Code
});
        }

        [HttpGet("{code}")]
        public IActionResult GetRoom(string code)
        {
            if (string.IsNullOrEmpty(code))
                return BadRequest("Код комнаты не может быть пустым.");

            var room = _game.GetRoom(code);
            if (room == null)
                return NotFound("Комната не найдена.");

            //var roomDto = new RoomDto
            //{
            //    Code = room.Code,
            //    Players = room.Players.Select(p => new PlayerDto { Name = p.Name, Avatar = p.Avatar
}).ToList()
            //};

            return Ok(room);
        }

        [HttpGet]
        public IActionResult GetRoom()
        {
            // Попытка получить значения roomId и playerId из куки
            if (Request.Cookies.TryGetValue("roomCode", out var roomCode))
            {
                var room = _game.GetRoom(roomCode);
                if (room == null)
                {

```

```

        return NotFound("Комната не найдена");
    }
    return Ok(room);
}

return BadRequest("Куки не найдены или неверные данные");
}
[HttpGet("players")]
public IActionResult GetPlayers()
{
    if (Request.Cookies.TryGetValue("roomCode", out var roomCode))
    {
        var room = _game.GetRoom(roomCode);
        if (room == null)
        {
            return NotFound("Комната не найдена");
        }
        return Ok(room.Players);
    }

    return BadRequest("Куки не найдены или неверные данные");
}
[HttpGet("log")]
public IActionResult Log()
{
    if (Request.Cookies.TryGetValue("roomCode", out var roomCode))
    {
        var room = _game.GetRoom(roomCode);
        if (room == null)
        {
            return NotFound("Комната не найдена");
        }
        return Ok(room.Log);
    }
    return BadRequest("Куки не найдены или неверные данные");
}

[HttpDelete("{code}")]
public IActionResult DeleteRoom(string code)
{
    if (string.IsNullOrEmpty(code))
        return BadRequest("Код комнаты не может быть пустым.");

    var success = _game.DeleteRoom(code);
    if (!success)
        return NotFound("Комната не найдена.");

    return Ok($"Комната {code} удалена.");
}
}

```